

HANDS-ON LABS

FortiCNAPP Workshop: AWS Integration

Built 2 October 2026

github.com/andrewbearsley/forticnapp-workshop-aws-automated

The labs need a live AWS account and the FortiCNAPP console. Use this copy to read ahead and follow along. The GitHub repository always has the latest version.

Contents

Overview	3
Lab 1 What FortiCNAPP Sees	6
Lab 2 Get Temporary AWS Credentials	18
Lab 3 Onboard AWS with Automated Configuration	24
Lab 4 Add CloudTrail with CloudFormation	35
Lab 5 Install Linux Agent	40
Lab 6 Install Windows Agent	49
Lab 7 Install the Lacework CLI	58
Lab 8 Code Security for Infrastructure as Code (IaC)	62
Lab 9 Code Security for Applications (SCA)	64
Lab 10 Clean Up Workshop Resources	67
 OPTIONAL: INTEGRATE AWS VIA INFRASTRUCTURE AS CODE	
Lab 11 Install Terraform	76
Lab 12 Install Integrations via Terraform	80
Lab 13 Scripted Cleanup of All Workshop Resources	86

Overview

Let's connect FortiCNAPP to an AWS account, then see what it finds!

What FortiCNAPP is

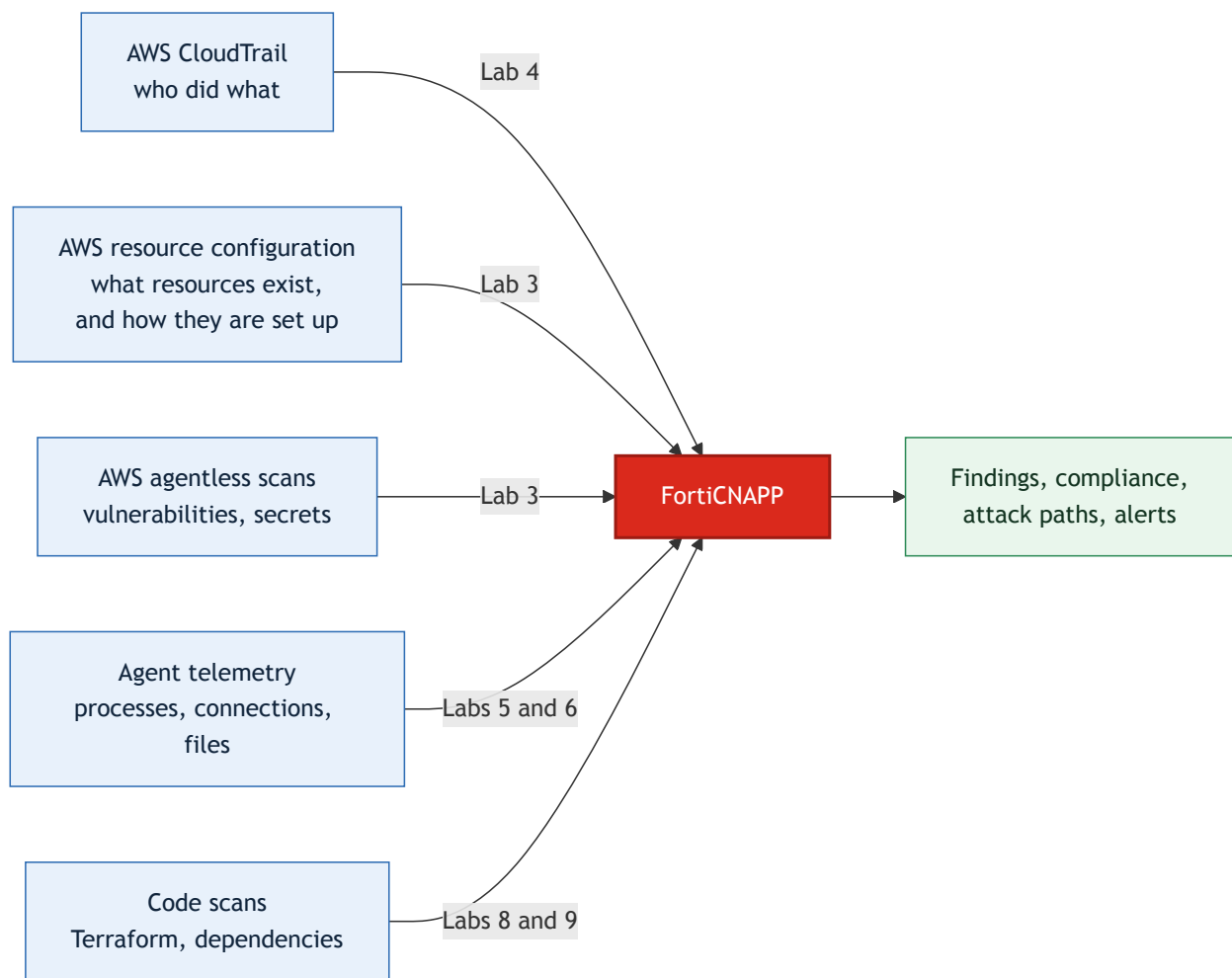
Fortinet's cloud-native application protection platform, formerly Lacework. One platform securing the whole code-to-cloud lifecycle, from the Terraform in a pull request to the workload it becomes:

	Answers
Posture	Is anything misconfigured, exposed, or failing a compliance framework?
Workloads	What is running on my hosts and containers, and is it behaving normally?
Identities	Who can do what in this account, and who actually does?
Code security	Would this have been a problem before it ever deployed?

FortiCNAPP correlates across all four rather than alerting on each in isolation. That is what lets it catch zero-day activity and compromised credentials from behaviour rather than from a signature, while raising far fewer alerts.

The practical effect is fewer people chasing noise: cloud architects see what to fix and in what order, risk teams get compliance evidence without asking for it, and threat teams get a short list worth investigating.

What you are building



Hands-on lessons

Lab	What you do	Why it matters
1	Explore a populated console	See FortiCNAPP in action
2	Get temporary AWS credentials	FortiCNAPP needs permission to build, briefly
3	Onboard the account with the wizard	Configuration and agentless in one pass
4	Add CloudTrail with CloudFormation	Threat detection, by a method the wizard cannot use here
5	Install the Linux agent	Continuous visibility, not periodic snapshots
6	Create a Windows agent token, then optionally install the agent	Same idea, different OS
7	Install the Lacework CLI	Needed by Labs 8 and 9
8	Scan Terraform for misconfigurations	Catch it before it reaches AWS
9	Scan an app for vulnerable dependencies	Catch them before they reach AWS

Lab	What you do	Why it matters
10	Clean up	Leaving cloud resources running costs money

Optional: integrate AWS via infrastructure as code

Take these to onboard through a pipeline rather than a wizard.

Lab	What you do	Why it matters
11	Install Terraform	The wizard writes Terraform, so you need it to run the same thing yourself
12	Onboard AWS via Terraform	Onboarding becomes reviewable code, not a set of console clicks
13	Scripted cleanup	One script removes everything the workshop created

Short on time?

You have	Run
90 minutes	Labs 1 to 4, then 10
Half a day	Labs 1 to 10

Prerequisites

- An AWS account with administrator access
- FortiCNAPP console access, tenant **FORTINETAPACDEMO**
- A browser
- An RDP client, for the optional part of Lab 6: Remote Desktop Connection on Windows, **Windows App** on a Mac

Resources

- [FortiCNAPP documentation](#)
- [Automated configuration](#)

Contributing

Questions or improvements, open an issue or a pull request.

Lab 1: What FortiCNAPP Sees

Objectives

Before you connect anything, look at an environment that is already connected and full of real findings.

By the end you will know what the later labs build towards. And roughly where things live in the console.

Prerequisites

- Your email address added to the FortiCNAPP demo environment

Important

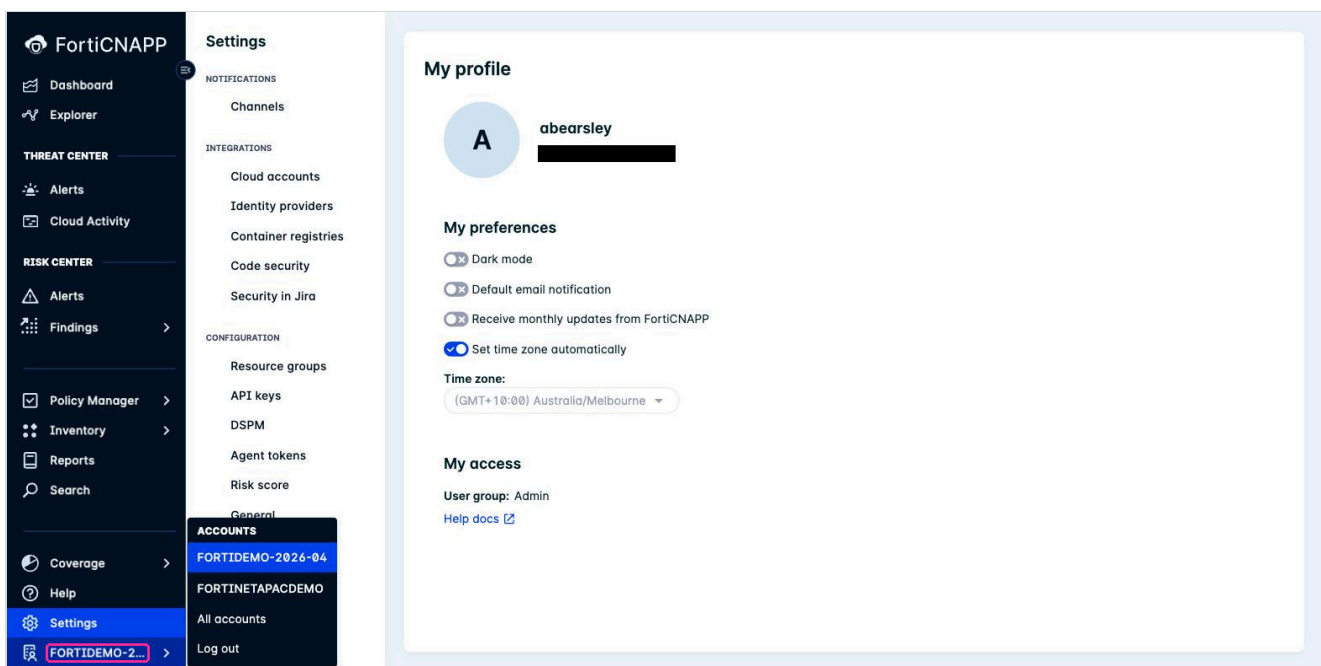
FortiCNAPP is multi-tenanted. See the tenant name at the bottom left navigation.

Lab 1 takes you through a **read-only tenant (FORTIDEMO)**, populated with vulnerable applications and signs of compromise.

Lab 3 onwards is a **sandpit tenant (FORTINETAPACDEMO)**, where you onboard your own AWS account.

Step 1: Sign in and pick the tenant

1. Go to <https://partner-demo.lacework.net/>
2. Enter your email address and click **Get sign in link**.
3. Open the link from your email.
4. If an onboarding wizard appears, choose **Go to platform**.
5. At the **bottom of the left navigation**, click the account name and select **FORTIDEMO-2026-04**.



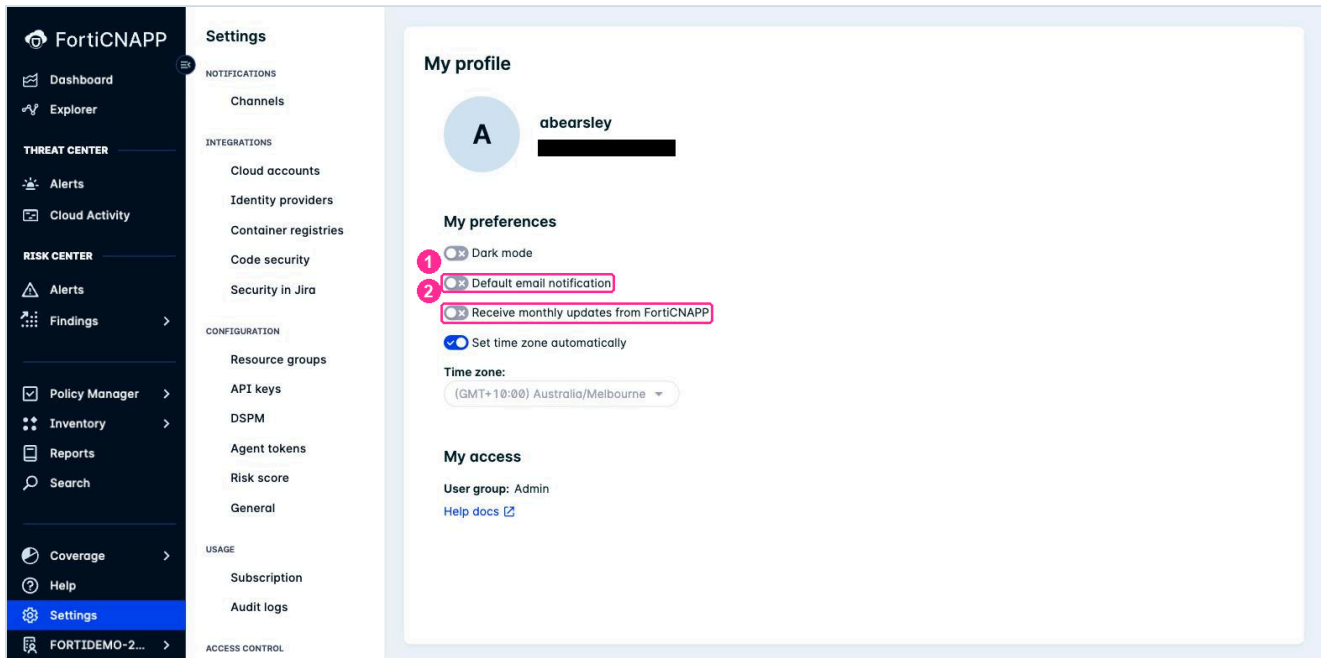
Checkpoint: the bottom of the left navigation reads **FORTIDEMO-2026-04** .

Step 2: Turn off email notifications

Do this now, before you go any further. This tenant is noisy, and by default it will email you about it.

Go to **Settings > My profile**, then under **My preferences**:

1. Turn **off Default email notification**.
2. Turn **off Receive monthly updates from FortiCNAPP**.

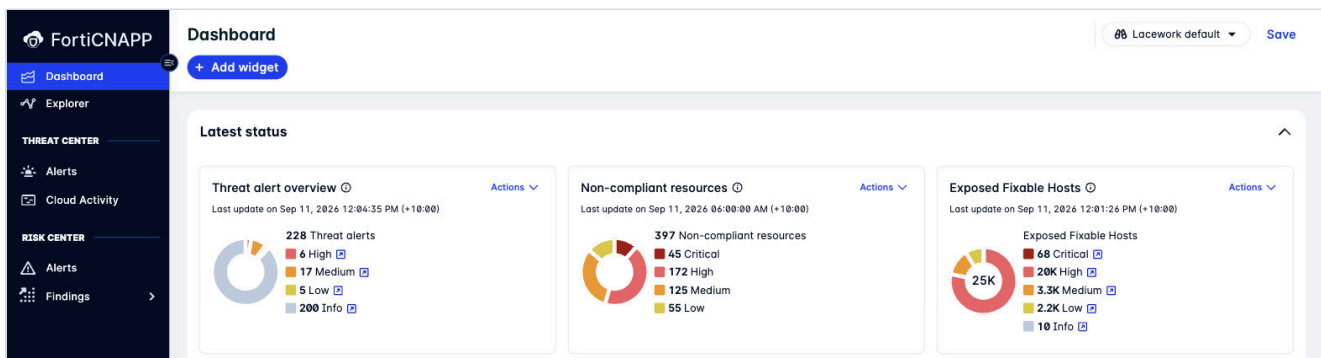


These are **your** preferences only. You are not changing anything for anyone else in the tenant.

Repeat this in **Lab 3**, the first time you enter FORTINETAPACDEMO. The setting is per tenant.

Step 3: Let's explore FortiCNAPP!

For a high-level view, go to **Dashboard**.



Widget	What it counts
Threat alert overview	Something is happening that looks like an attack
Non-compliant resources	Configuration that fails a benchmark
Exposed Fixable Hosts	Internet-reachable hosts with a patchable vulnerability

Checkpoint: you can read a number off each of the three widgets.

Worth pausing on **Exposed Fixable Hosts**. Not "hosts with vulnerabilities", which is every host. Exposed, and fixable. That is the list you would actually work through on a Monday morning.

Step 4: Show me the exposed hosts

Explorer answers questions about how things connect, rather than listing them.

1. Go to **Explorer**.
2. Click the **query** button to the right of the search bar. It is labelled with the current query, so it usually reads **Show Hosts**. That opens the **Query Builder**.
3. Under **SHOW**, click **Select a return type**, choose **Hosts**, then click **Add clause**.
4. Click **Add clause** again, choose **Internet Exposed**, leave **is equal to** set to **True**, and click **Add clause**.
5. Click **Search Results**.

The screenshot shows the FortiCNAPP Explorer interface. On the left is a dark sidebar with navigation options: Dashboard, Explorer (selected), Threat Center, Risk Center, Policy Manager, Inventory, Reports, Search, Coverage, Help, Settings, and FORTIDEMO-2... The main area is titled 'Explorer' and contains a search bar with the text 'Search saved queries or try "critical s3 with iam access"'. Below the search bar is a 'Save query' button and a query preview: 'Show Hosts that meet conditions: (Internet Exposed true)'. To the right of the preview is a timestamp: 'Last updated on 9/11/2026 7:00 AM (+10:00)'. Below this is a section titled 'Query results (21)' with a 'Sort: 1' dropdown. The results are displayed in a table with columns: Action, Resource Id, Cloud Provider, Alerts, Attack Paths, Compliance Violations, Public IP Addresses, and Resource Tags. The table contains 8 rows of data. The first row has a 'Graph' button next to the 'Action' column. The second row has a 'Details' button next to the 'Resource Id' column. The third row has a 'Graph' button next to the 'Action' column. The fourth row has a 'Details' button next to the 'Resource Id' column. The fifth row has a 'Graph' button next to the 'Action' column. The sixth row has a 'Details' button next to the 'Resource Id' column. The seventh row has a 'Graph' button next to the 'Action' column. The eighth row has a 'Details' button next to the 'Resource Id' column. At the bottom of the table is a 'Display: 50' dropdown and a pagination indicator '1 - 21 of 21'.

Action	Resource Id	Cloud Provider	Alerts	Attack Paths	Compliance Violations	Public IP Addresses	Resource Tags
Graph	i-0ec084e06c5cddb04	AWS	0	0	0		Name: FortiGateVM F
Details	i-0ec3a4a22ebc9e207	AWS	0	0	0	18.233.29.29	Name: FortiGateVM F
Graph	i-046a96003e752e3f6	AWS	1	0	3		Name: primary aw
Details	activity-generator	AWS	0	1	3	35.231.254.24	type: activity-gener...
Graph	gke-sharedgke-default-node-pool-e9093666-nrh5	AWS	0	1	0	34.24.191.147	deployment: ticketin.
Details	activity-generator-machine	AWS	0	1	4	20.12.208.170	
Graph	vote-traffic-machine	AWS	0	1	4	20.29.93.124	
Details	i-00f8fc270e530d854	AWS	0	1	5	54.82.232.173	Name: ecom-traffic
Graph	i-0385aae9bd1a8d16b	AWS	2	0	3		Name: primary aw

Every one of these hosts can be reached from the internet. The columns beside them say how much trouble each one is in.

6. Pick a row with a non-zero number under **Alerts** or **Attack Paths**, and click **Graph**.
7. Zoom in with the **+** control.



Checkpoint: you can trace the red line from the internet gateway to the host.

Read it left to right: internet gateway, load balancer, network interface, security group, then the host. That red line **is** the exposure, drawn as the actual chain of AWS resources that permits it. Hanging underneath are the host's vulnerabilities, compliance violations and alerts.

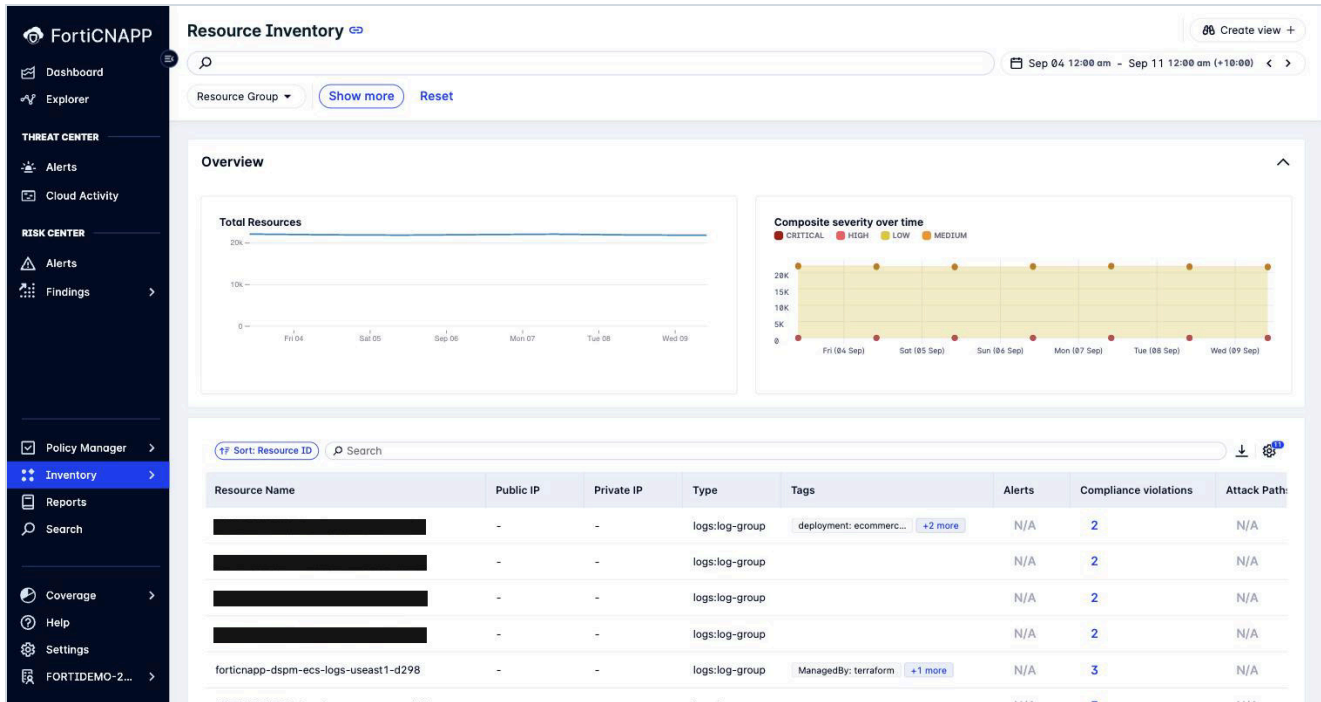
This is the context an exposed host needs before you can act on it. A vulnerability list tells you what to patch. This tells you what is reachable, by which route, and which link in that chain you could break instead.

Step 5: What do we have in our cloud estate?

What exists is one question. What it actually does is another. Separate views for each.

What exists

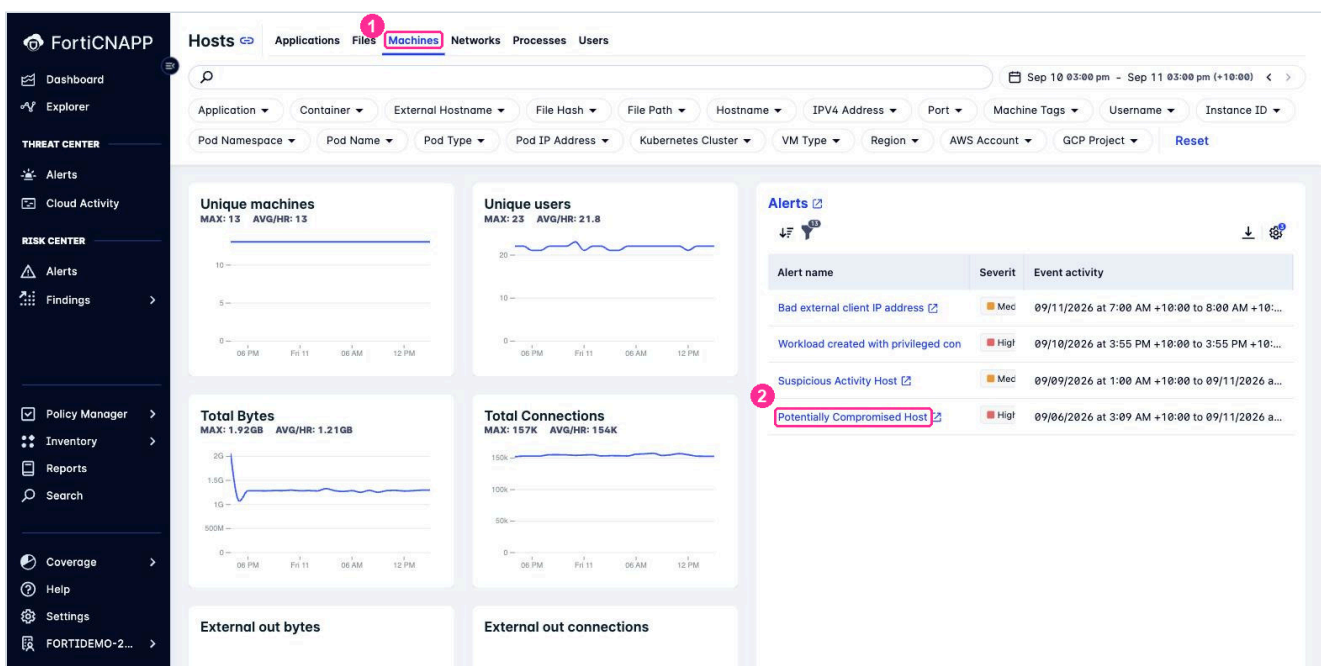
Go to **Inventory** > **Resource Inventory**.



In a data centre you know what is in the rack because you put it there. In a cloud account, anyone with credentials can create something at three in the morning, so asking the provider is the only way to know.

What it does

Go to **Inventory > Hosts**, then the **Machines** tab.



Look at the tabs: **Applications, Files, Machines, Networks, Processes, Users**. Every one is a record of behaviour over time, not a snapshot.

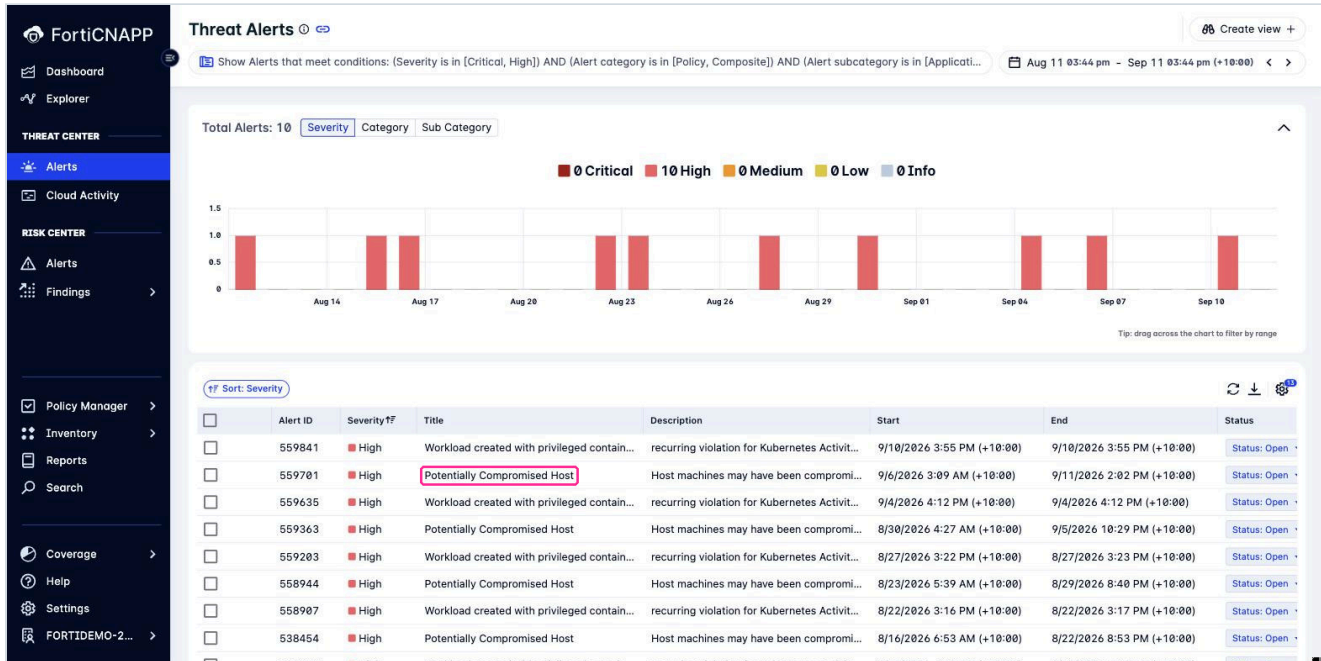
Checkpoint: you can read **MAX** and **AVG/HR** off the Unique machines widget.

This is the polygraph. FortiCNAPP watches what normally runs, who normally logs in, and what normally talks to what, then builds a baseline per host. It does not need a signature for an attack it has never seen. It needs to know that this machine has never done this before.

That is why the agent in Labs 5 and 6 matters. A scan tells you what is installed. Only something watching continuously can tell you that behaviour changed.

Step 6: What is happening right now?

Go to **Threat Center > Alerts**.



The view opens on **Critical and High, last month**.

1. Open an alert titled **Potentially Compromised Host**.
2. Read the description and the affected resource.
3. Look at **Threat Center > Cloud Activity**, the CloudTrail record of who did what.

Checkpoint: you have opened one alert and can say which host it refers to.

What you just opened

Potentially Compromised Host is a composite alert. Worth stopping on, because it is not a rule firing.

Most alerts are one condition being met. A composite alert is assembled from several signals that only mean something together. No single one of them would have been worth waking anyone for.

Read the description again with that in mind. A process nobody has run before. A connection to somewhere this host has never talked to. A file changed that never changes. Any one of those is a Tuesday. Together, on the same host, in the same window, they are how a compromise actually looks.

This is the polygraph from Step 5 doing its job. You cannot assemble a composite alert without knowing what normal looked like first, which is what the agent in Labs 5 and 6 gives you.

Another one, if you want to see it

Potentially Compromised AWS Keys is the identity version: the same idea, pointed at credentials rather than hosts. A key used from somewhere new is odd. A key used from somewhere new, at an unusual hour, to list resources it has never touched, is how a stolen credential behaves.

This one may not be in the demo tenant on the day. The default view shows Critical and High for the last month, and that window often contains only **Potentially Compromised Host** and **Workload created with privileged containers**. If you cannot find it, widen the severity filter or the time range. Nothing is broken.

Composite alerts are rare by design, so you have to go looking:

1. Set **Alert category** to **Composite** only.
2. Set the date range to the **last 6 months**. A month will not find one.

That one comes from CloudTrail, which is why Lab 4 onboards it.

There is a second inbox at **Risk Center > Alerts**. Threat alerts say something is happening. Risk alerts say something is dangerous. Worth knowing both exist.

Step 7: What do I fix first: the toxic combinations

Go to **Risk Center > Findings > Attack Path**.

The screenshot shows the FortiCNAPP interface with the **Attack Path** view selected. The left sidebar contains navigation options like Dashboard, Explorer, Threat Center, Risk Center, Policy Manager, Inventory, Reports, Search, Coverage, Help, Settings, and FORTIDEMO-2. The main content area displays four panels:

- Top risky hosts**: A table listing hosts with their Account/Project/Subscriptions and Vulnerability Observations. Hosts include cloud-activity, ecom-traffic, activity-generator-machine, vote-traffic-machine, and datalayer0.
- Top risky container images**: A table listing container images with their Account/Project/Subscriptions. Images include docker.io/hwgithub0/ecommerce-website:latest, docker.io/library/postgres:latest, docker.io/library/docker:24.0.0-rc.1-dind, registry.k8s.io/ingress-nginx/controller:sha256..., and docker.io/hwgithub0/ticketing-bff:latest.
- Top risky paths with exposed secrets**: A table listing secret types, secret identifiers, hosts, and accounts. The entry shows an ssh_private_key secret on host ip-172-16-1-55.
- Top risky data assets**: A table listing data assets, resource types, and accounts. Assets include order, order, order, and order, all associated with RDS resources.

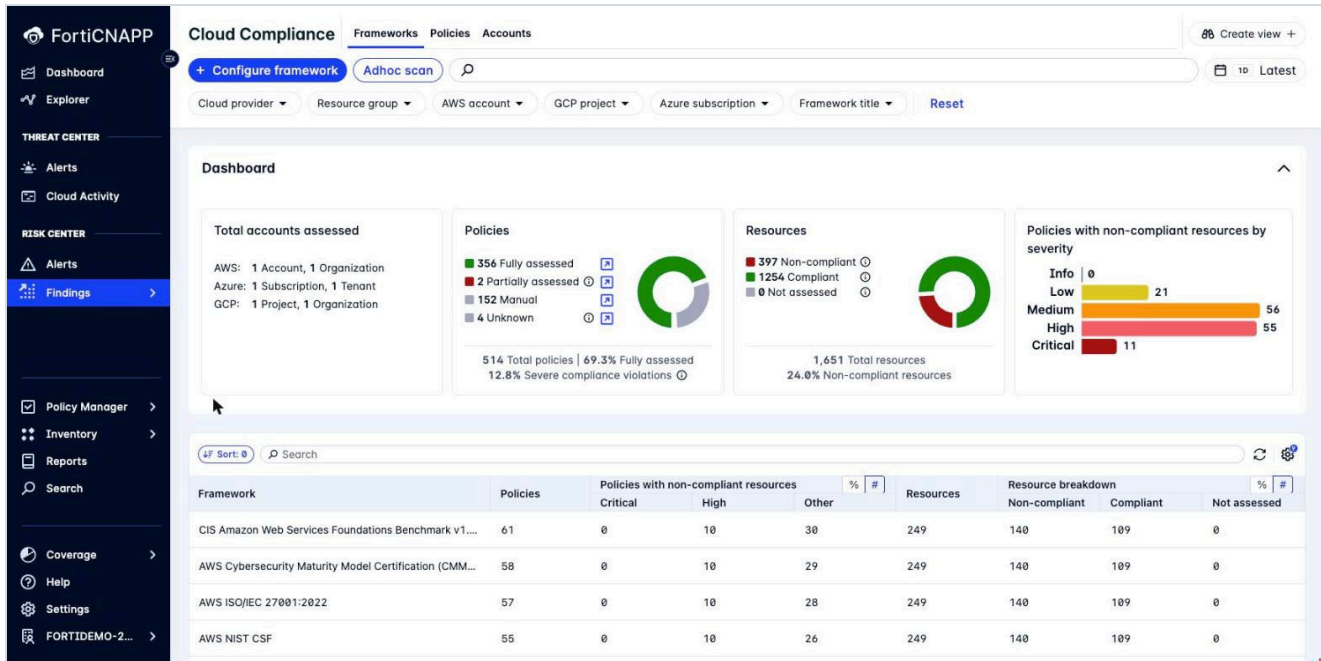
Attack Path only lists combinations that are genuinely reachable and genuinely damaging: a vulnerable host that is internet-facing **and** holds credentials to something valuable.

Checkpoint: find the entry under **Top risky paths with exposed secrets**. That is a private key sitting on a reachable host.

One vulnerability on an isolated box is a ticket. The same vulnerability on an internet-facing box with a key to the database is an incident.

Step 8: What do I fix first: compliance

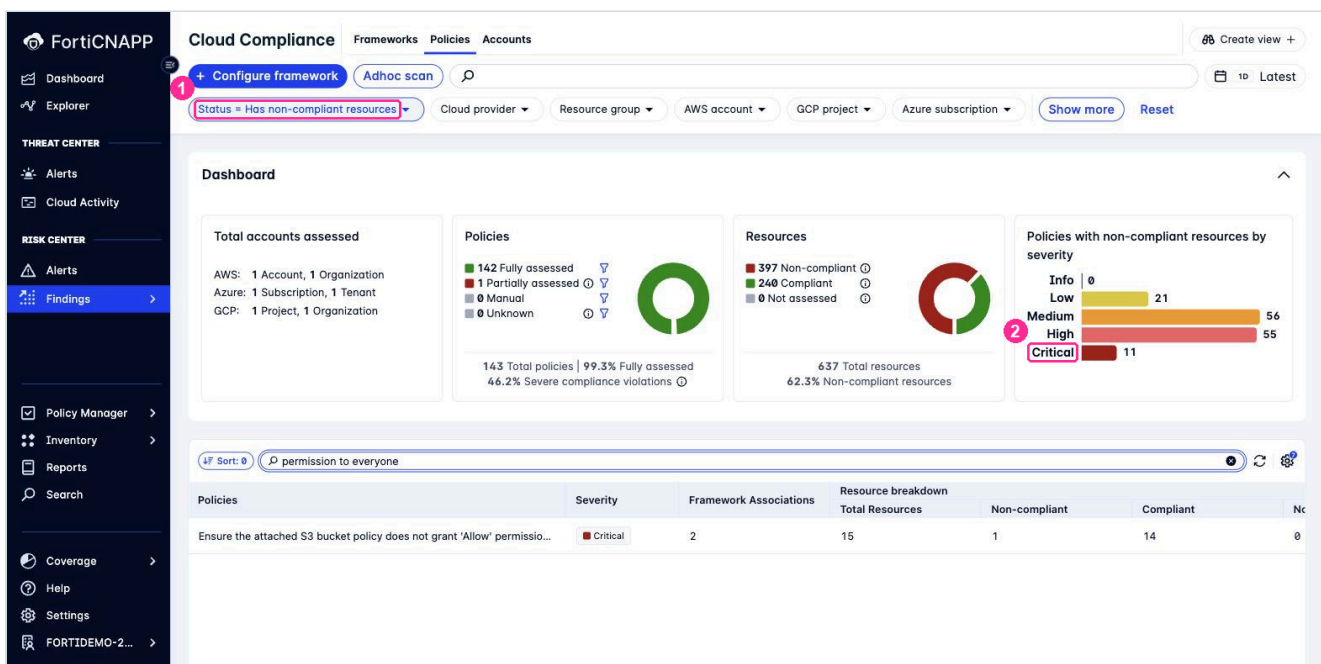
Go to **Risk Center > Findings > Compliance > Cloud**.



The **Frameworks** tab scores the same findings against CIS, ISO/IEC 27001, NIST CSF, SOC 2 and the rest.

Now find a specific failure.

1. Click the **Policies** tab. It opens already filtered to **Status = Has non-compliant resources**, which is the only filter that matters when you are deciding what to do today.
2. Search for `permission to everyone`.



3. Open **Ensure the attached S3 bucket policy does not grant 'Allow' permission to everyone**. It is **Critical**.

FortiCNAPP Policies > Policy Details

Ensure the attached S3 bucket policy does not grant 'Allow' permission to everyone Critical AWS Fully assessed

Last evaluated on 09/11/2026 at 9:24 AM +10:00

1 [View context](#)

2 15 Total resources

- 1 Non-compliant
- 14 Compliant
- 0 Not assessed
- 0 Excepted

6.7% non-compliant resources

Resource	Region	Account	Status
arn:aws:s3:::unhackable-mcp-bucket-324752	us-east-1	Account: [REDACTED]	Non-compliant

Checkpoint: 15 resources assessed, exactly 1 non-compliant, and you can see which bucket it is.

One bucket in fifteen, with a policy granting access to everyone. Note the bucket's name while you are here. Confidence is not a control.

4. Click **View context**, top left.

FORTINET DOCUMENT LIBRARY

Products Best Practices Hardware Guides Products A-Z

Search Document

Home > FortiCNAPP > Lacework FortiCNAPP Policies

lacework-global-140

Ensure the attached S3 bucket policy does not grant 'Allow' permission to everyone

Description

An S3 bucket has a policy which allows permission to everyone, and the Block Public Access setting 'RestrictPublicBuckets' is not enabled at the bucket or account level. It is a best practice to restrict access to specific principals, and to enable the Block Public Access setting 'RestrictPublicBuckets' for buckets with policies granting access to everyone.

Remediation

Perform the following to remove permissions for everyone from the S3 bucket:

1. Sign in to the AWS Management Console.
2. Select Services.
3. Select S3.
4. Select the bucket to change.
5. Go to Permissions.
6. Go to Bucket Policy and select Edit.
7. Check for any Statement with Effect set to 'Allow', Principal set to '*' or 'AWS*', and no conditions.
8. To remove access, delete the Statement.
9. To limit access to a specific AWS account or Identity and Access Management (IAM) user, replace the unrestricted Principal

Checkpoint: you are reading the **Description** and a numbered **Remediation** for that exact policy.

Every policy links straight to what it means and the steps to fix it, so a finding is never just a red number. Hand this page to whoever owns the bucket.

Step 9: What do I fix first: identities

Go to **Risk Center > Findings > Identities**, then the **Top Identity Risks** tab.

Click the top identity and read the **Risks** panel on the right.

Checkpoint: find an identity that allows full admin, and check its **Last used time**.

A role that can do everything and has never been used is the cheapest fix in cloud security. Nothing breaks when you remove it. One less thing for an attacker to find. Identity is usually the shortest path from a foothold to real damage.

Step 10: What do I fix first: vulnerabilities

Go to **Risk Center > Findings > Vulnerabilities**, then the **Top items** tab.

The widgets answer different questions:

Widget	Answers
Top vulnerabilities by impacted hosts	Which CVE is on the most machines
Top fixable packages	Which single upgrade closes the most CVEs

Checkpoint: find a package where one **Fix version** clears several CVEs at once.

The second widget is the one to work from. Patching by CVE is endless. Patching by package is finite. The **Fix version** column tells you exactly where to get to.

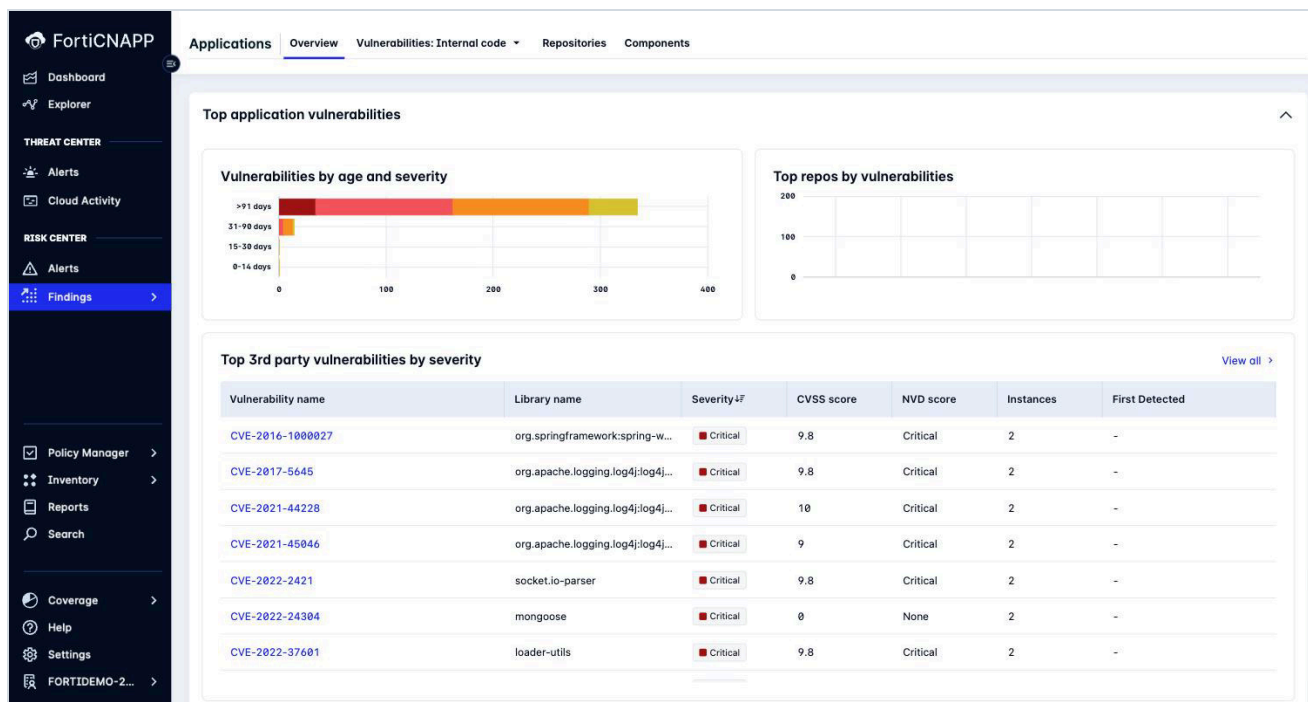
Step 11: Could I have caught it earlier?

Go to **Risk Center > Findings > Code Security**.

Infrastructure (IaC) scans the Terraform and CloudFormation that builds the infrastructure.

The screenshot shows the FortiCNAPP interface with the 'Infrastructure as code' section selected. The 'Assessments' tab is active, displaying a table of repositories. The table columns are: Repository, Languages detected, Last scan, Results (C, H, M, L counts), and Actions. The results are displayed as colored boxes: C (Critical), H (High), M (Medium), and L (Low). The table lists several repositories, including partner-deploy/voteapp, partner-deploy/commerce-deploy, partner-deploy/ticketing, partner-deploy/unhackable-vote-app, partner-deploy/unhackable-commerce-app, partner-deploy/mcp-server, and partner-deploy/commerce. Each row shows the languages detected (e.g., cloudformation, terraform, node) and the results of the latest scan (e.g., C: 12, H: 34, M: 82, L: 103).

Applications scans what your code depends on.



Checkpoint: find a Log4j CVE in the Applications list. You already know that one.

Everything up to this point finds problems that are already running. This finds them in the code, before they exist in AWS. You do both in Labs 8 and 9.

What did we do here?

We went from "how bad is it" to "how would I have prevented it", which is the same path you walk during a real incident.

Hold on to those questions, because the rest of the workshop answers them:

Question	You built it in
What have I got, and is it compliant	Lab 3
What is happening on my workloads	Labs 5 and 6
Could I have caught it in code	Labs 8 and 9

Next: [Lab 2: Get Temporary AWS Credentials.](#)

Lab 2: Get Temporary AWS Credentials

Objectives

In Lab 3, FortiCNAPP builds your integration for you: IAM roles, buckets, keys, queues, a CloudTrail trail. To do that it needs permission to create things in your AWS account.

You are not handing over a permanent key. You are issuing a **visitor pass that expires in an hour**. FortiCNAPP uses it once to build the integration, then throws it away and runs on the read-only role it created. That role is what stays.

This lab gets you that pass, so the wizard in Lab 3 does not sit waiting while you go and find it.

Prerequisites

- An AWS account with administrator access
- Either AWS IAM Identity Center access, or an IAM user that can create a role

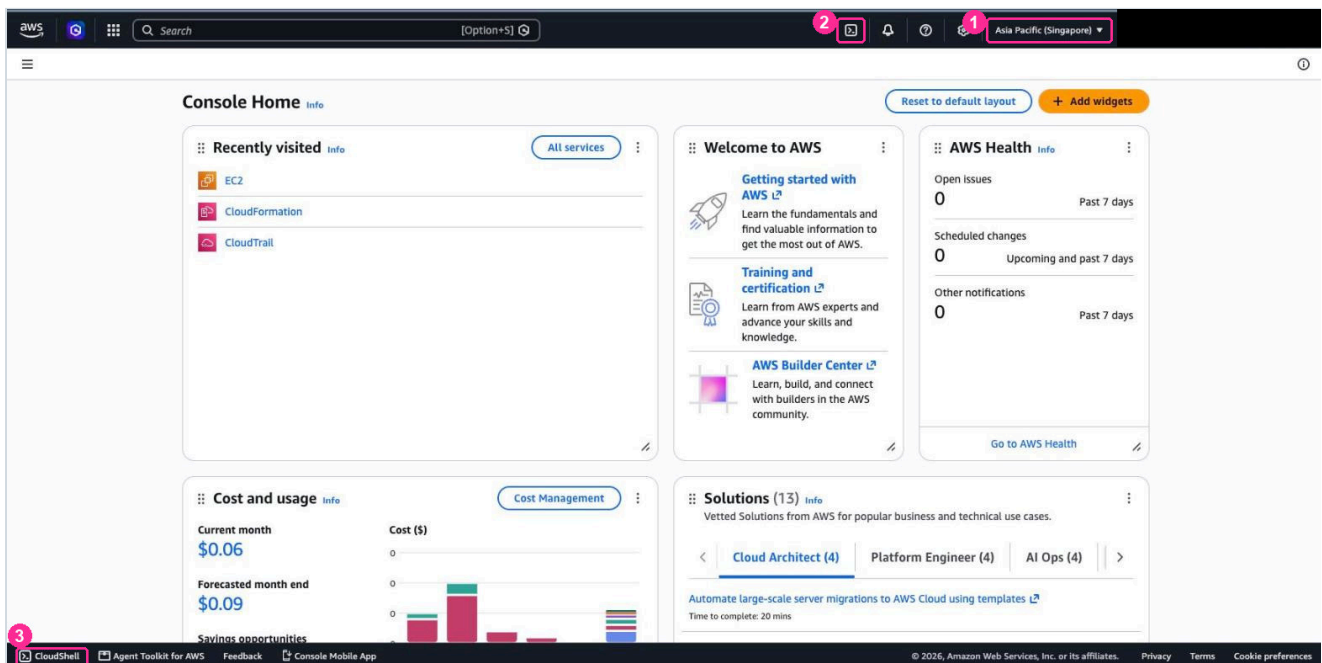
Which method do you need?

Not a preference. It depends on how you signed in to AWS.

Ask AWS who you are. You do that in **CloudShell**, a Linux terminal built into the console with your credentials already loaded.

Opening CloudShell

1. Check the **region selector** reads **Asia Pacific (Singapore)**.
2. Click the **>_** icon in the top bar. **CloudShell** at the bottom left does the same thing.



CloudShell opens as a panel across the bottom. First launch takes a minute, and the panel stays black until it is ready.

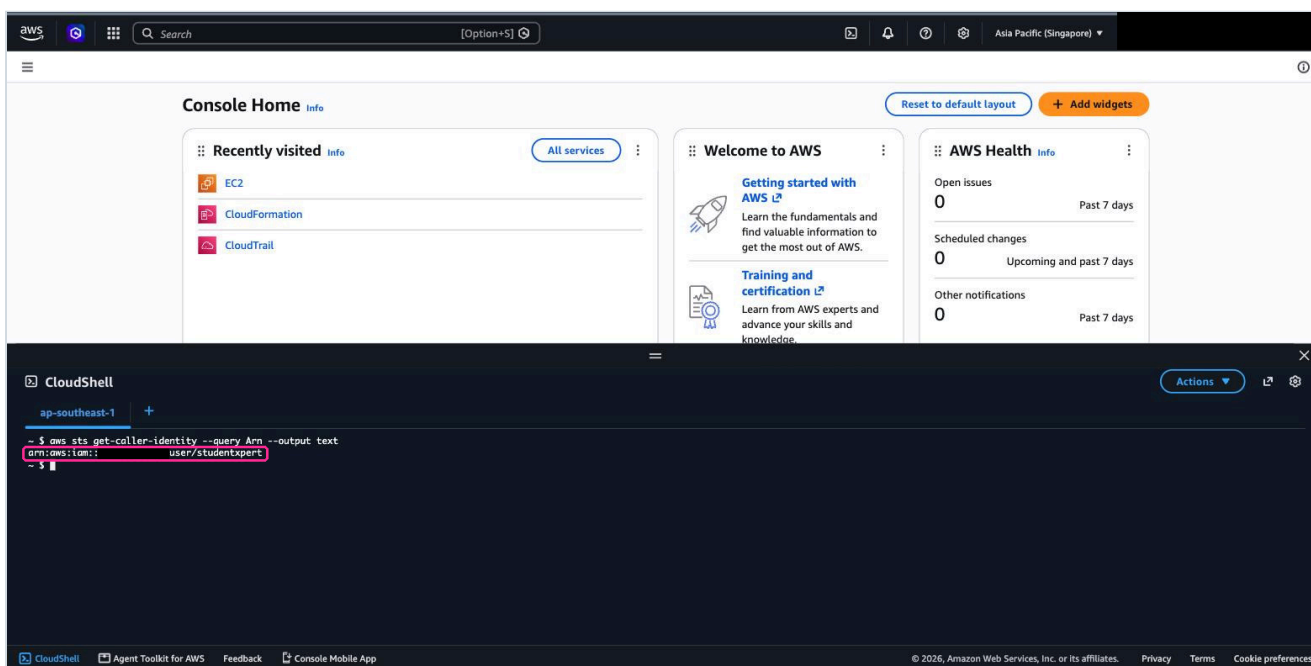
Important

The first launch in a region shows a "Welcome to AWS CloudShell" dialog. Close it before you type. Anything typed while it is open is swallowed, and the command appears to have done nothing.

Who are you?

At the `~ $` prompt:

```
aws sts get-caller-identity --query Arn --output text
```



Read the ARN it prints:

If it says	You are	Go to
<code>arn:aws:sts::<id>:assumed-role/AWSReservedSSO_...</code>	Signed in through Identity Center	Method A
<code>arn:aws:iam::<id>:user/<name></code>	A plain IAM user	Method B

Checkpoint: you know which of the two methods you are following.

Method A: AWS IAM Identity Center

You will not be in CloudShell for this one. The values come straight out of the access portal.

1. Open your AWS access portal.
2. Go to the **Accounts** tab and select the account you want to integrate.
3. Click **Access keys** next to the role you will use.
4. Choose **Option 3: Use individual values in your AWS service client**.
5. Copy all three values somewhere you can paste from:

- **AccessKeyId**
- **SecretAccessKey**
- **SessionToken**

Checkpoint: you have three values, and the first starts with `ASIA` .

Skip to [Lab 3](#).

Method B: Create a role, then assume it

You are an administrator in your own lab account, so you can create the role yourself. Nobody needs to provision it for you.

Run all of this in CloudShell.

Warning

Only run Method B if the ARN said `:user/` . The trust policy below names an IAM user, and an Identity Center session is not one, so `assume-role` is denied. The role and the policy get created first, so you only find out on the third command.

1. Create the role

```
ACCT=$(aws sts get-caller-identity --query Account --output text)
USER=$(aws sts get-caller-identity --query Arn --output text | awk -F/ '{print $NF}')

cat > trust.json <<JSON
{"Version":"2012-10-17","Statement":[{"
  "Effect":"Allow",
  "Principal":{"AWS":"arn:aws:iam::${ACCT}:user/${USER}"},
  "Action":["sts:AssumeRole"]}]}
JSON

aws iam create-role --role-name forticnapp-onboarding \
  --assume-role-policy-document file://trust.json
```

The trust policy says one thing: *this role may be assumed by me, and nobody else.*

2. Give it permissions

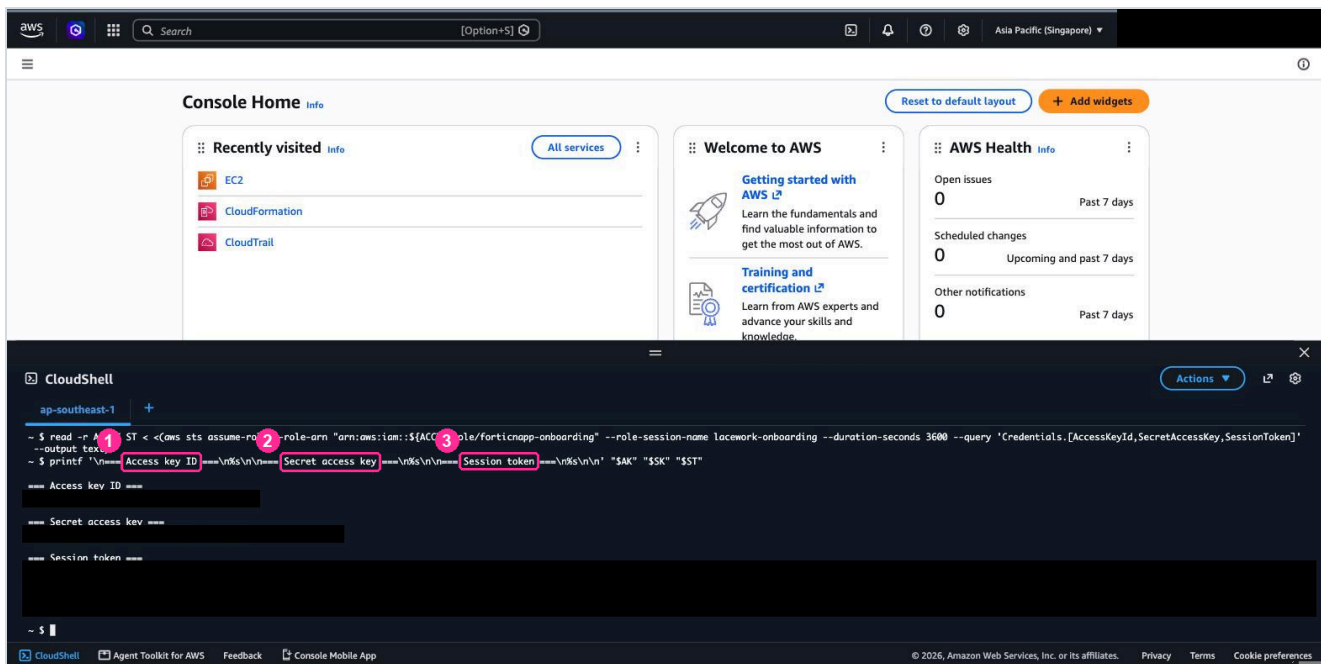
```
aws iam attach-role-policy --role-name forticnapp-onboarding \
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess
```

3. Assume it

Wait about ten seconds for the role to propagate, then:

```
sleep 10
read -r AK SK ST <<(aws sts assume-role \
  --role-arn "arn:aws:iam::${ACCT}:role/forticnapp-onboarding" \
  --role-session-name lacework-onboarding --duration-seconds 3600 \
  --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]' --output text)

printf '\n=== Access key ID ===\n%s\n\n=== Secret access key ===\n%s\n\n=== Session token\n\n\n' "$AK" "$SK" "$ST"
```



Checkpoint: three labelled blocks print, and the access key ID starts with `ASIA` .

To copy a value, select it with the mouse and press **Cmd+C**, or **Ctrl+Shift+C** on Windows and Linux. Select from the start of the value to the end of the line, not the label.

Important

Go straight into Lab 3. The clock starts now, not when you open the wizard. If you stop for a break between here and the Configure step, the credentials expire and you start Lab 2 again. The failure surfaces as `ExpiredToken` two steps into Lab 3, not when you paste.

Caution

Paste these into the Lab 3 wizard only. Not into chat, not into a shared document, and take care if your screen is being shared. They are live credentials for an hour.

Important

Do not raise `--duration-seconds` above 3600. CloudShell may already be running as a role session. AWS caps role chaining at one hour.

Permissions

`AdministratorAccess` keeps the lab moving. In production, use least privilege instead.

The **Authorization Guide** panel in the Lab 3 wizard (click **Open Guide** on the Authorize step) offers four ready-made policy documents:

File	Covers
<code>aws_config_policy.json</code>	Configuration
<code>aws_cloudtrail_policy.json</code>	CloudTrail
<code>aws_agentless_policy.json</code>	Agentless Workload Scanning

File	Covers
aws_eks_auditlog_policy.json	EKS audit log

FortiCNAPP Settings

Integrate cloud account - AWS

Step 2 of 4 • Authorize

Temporary credentials required
 Lacework FortiCNAPP needs your authorization to create the IAM roles, permissions, and cloud resources required to monitor your cloud accounts. Please use [AWS IAM Identity Center](#) or [AWS CLI](#) to obtain temporary credentials. [Open Guide](#) for more instructions.

☒ Enable organization level integration

☒ Simulate IAM permissions

Access key ID

Secret access key

Session token

Default region

Authorization Guide

Obtaining temporary credentials

Using AWS IAM Identity Center:

- Go to your AWS access portal.
- Select the account to integrate with Lacework.
- Click "Access keys" next to the role you'll use.
- Copy the AccessKeyId, SecretAccessKey, and SessionToken into the fields.

Using an IAM user:

- Ensure you have an IAM user with the required permissions.
- Install the [AWS CLI](#) on your machine.
- Use the AWS CLI to login to the AWS account that you want to integrate with Lacework.
- Enable a MFA device for the IAM user.
- Run the following AWS CLI command to obtain the serial number of your MFA device.
`aws iam list-mfa-devices --user-name YourIAMUserName`
- Run the following AWS CLI command to

Select method > **Authorize** > Configure > Review & Deploy

Cancel Next

If you plan to use **Simulate IAM permissions** in Lab 3, the credentials also need `iam:SimulatePrincipalPolicy`, plus `iam:GetRole` for assumed roles.

If it goes wrong in Lab 3

Symptom	Cause
<code>InvalidClientTokenId</code> at Configure, Task 2	You used plain <code>aws sts get-session-token</code> . AWS blocks IAM API calls from those credentials unless MFA was included, and discovery calls IAM. Use a role session.
Session token is required	You pasted a long-lived AKIA key. The wizard needs a session token; static keys are rejected.
Cannot call <code>GetSessionToken</code> with session credentials	You are federated. Use Method A.
<code>is not authorized to perform: sts:AssumeRole</code> at Method B step 3	You are signed in through Identity Center and ran Method B. Use Method A. See the warning below.
<code>ExpiredToken</code> : The security token included in the request is expired at Configure, Task 2	Your credentials aged out between Lab 2 and discovery. Run Method A or B again and restart the wizard.
Credentials expired mid-wizard	They last an hour. Run Method A or B again.

What did we do here?

We issued a short-lived credential and nothing else.

This is the step that makes automated configuration safe to use on a production account. The CloudFormation path never asks for a credential, because you launch each stack yourself, and it costs you two console walk-

throughs. Here you trade one bounded hour of access for a wizard that does the work, plus a preflight permission check you would not otherwise get.

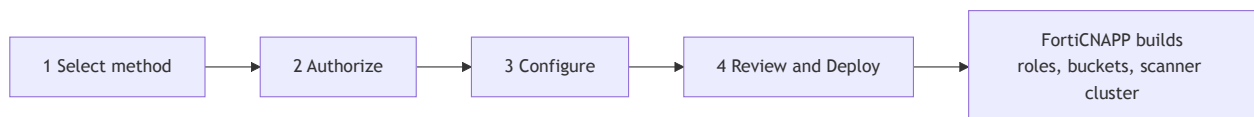
Next: [Lab 3: Onboard AWS with Automated Configuration](#).

Lab 3: Onboard AWS with Automated Configuration

Objectives

In this lab, you connect your AWS account to FortiCNAPP, so that the findings you toured in Lab 1 start appearing for **your** account.

You hand over the hour-long credential from Lab 2. FortiCNAPP then does the work: it checks your permissions, writes a Terraform plan for your account, applies it, and registers the result. You watch, you do not write any of it.



Work through the screens, then wait five to ten minutes while it builds.

Worth knowing before you start:

It creates real AWS resources	IAM roles, and the ECS cluster and networking the agentless scanner needs.
The credential expires	One hour from when you made it. If the wizard sits idle, it will fail partway.

Prerequisites

- Completed [Lab 2: Get Temporary AWS Credentials](#)
- Temporary AWS credentials that have not expired
- FortiCNAPP console access, tenant **FORTINETAPACDEMO**

Lab steps

Step 1: Open Cloud Accounts

1. Log into the FortiCNAPP console at <https://partner-demo.lacework.net/>
2. Confirm the tenant selector at the bottom left shows **FORTINETAPACDEMO**.

Important

If the switch does not take, refresh the browser. The selector sometimes reports the new tenant while the page still shows the old one's data. A reload settles it.

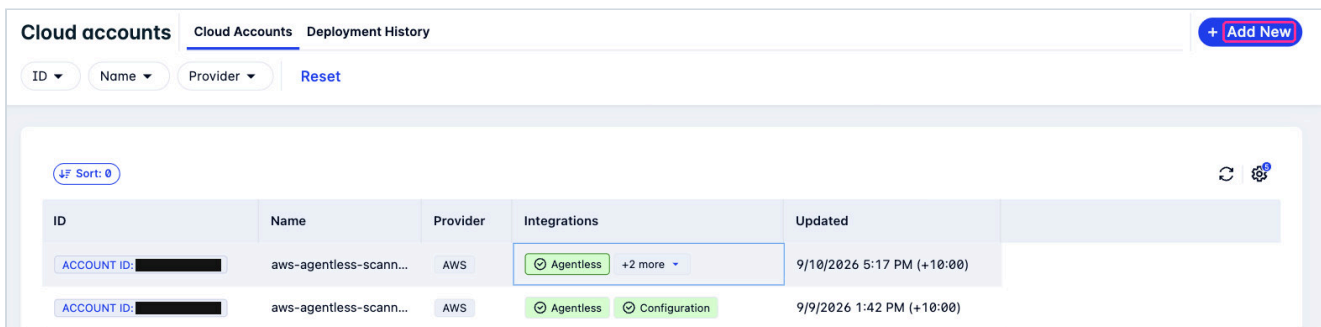
3. **Turn off email notifications for this tenant too.** Go to **Settings > My profile**, then turn **off Default email notification** and **Receive monthly updates from FortiCNAPP**.

The setting is per tenant, so switching tenants does not carry it across. Skip this and your own onboarding will email you about itself for the rest of the day.

Check: both toggles show Off before you go on. This tenant is shared. With the setting on, you also get the alert emails from every later class that onboards accounts here.

4. Navigate to **Settings > Integrations > Cloud accounts**.

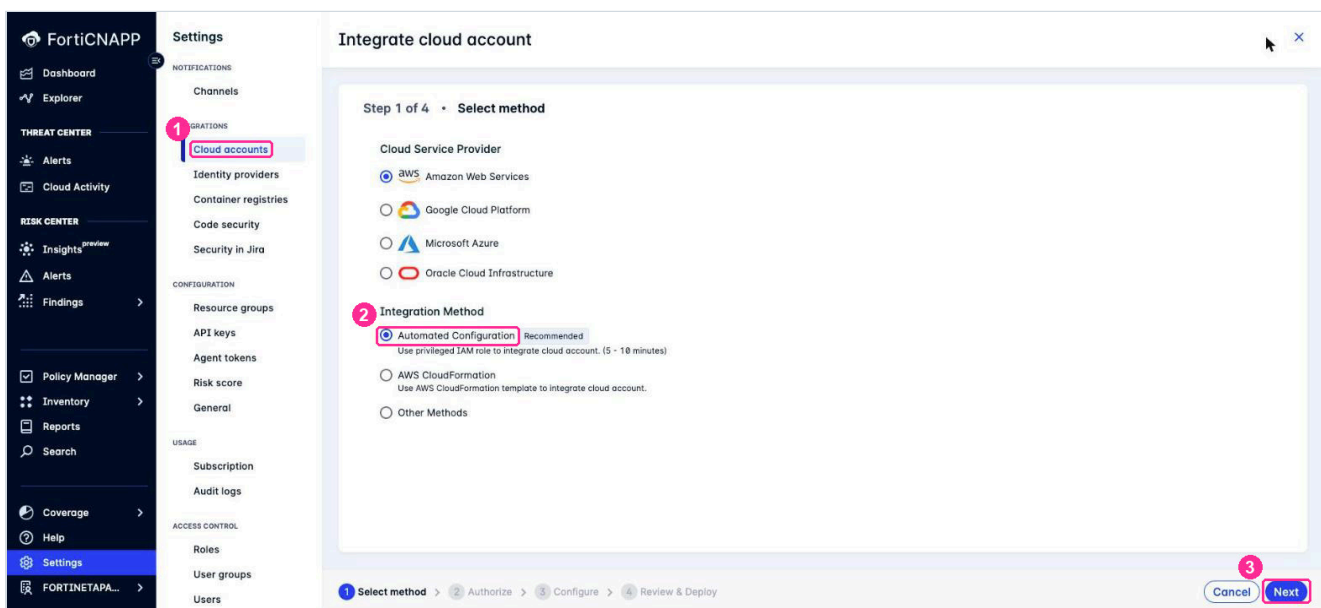
5. Click **Add New**, top right.



Checkpoint: the Integrate cloud account wizard opens on Step 1 of 4.

Step 2: Select the Method (Step 1 of 4)

1. Under **Cloud Service Provider**, select **Amazon Web Services**.
2. Under **Integration Method**, select **Automated Configuration**.
3. Click **Next**.



You just picked one of three ways to do the same job. The end state is identical.

Method	What you do	Pick it when
Automated Configuration	Hand over short-lived credentials, FortiCNAPP builds everything	Default. Least typing, and the console marks it Recommended.
AWS CloudFormation	Launch a stack per integration, set the parameters yourself	You want to read the template before anything is created
Other Methods	Take the Terraform and run it yourself	You want onboarding in a pipeline, reviewed in a pull request

Labs 11 and 12 do the Terraform route, so you can compare the two directly.

Step 3: Authorize (Step 2 of 4)

This screen takes the credentials from Lab 2.

1. Leave **Enable organization level integration** off.

Turning it on onboards every account in your AWS organization in one pass, using CloudFormation StackSets. That is the right choice for an account inside an AWS Organization. For this workshop, leave it off and integrate a single account.

2. Turn **Simulate IAM permissions** on.

This runs an AWS IAM policy simulation during discovery. It catches denials from **service control policies and permission boundaries before deployment**, which is the usual reason onboarding fails in an enterprise AWS organization. The simulation needs `iam:SimulatePrincipalPolicy` on your credentials, plus `iam:GetRole` if you assumed a role.

3. Enter your credentials from Lab 2:

- **Access key ID**
- **Secret access key**
- **Session token**

4. In **Default region**, select the region where FortiCNAPP will deploy its resources, for example **ap-southeast-1** for Asia Pacific (Singapore).

Region matters. Set it to the region where your workloads run. Get it wrong and agentless scanning looks in an empty region, finds nothing, and reports nothing. It does not warn you.

5. Click **Next**.

Checkpoint: the wizard moves to Step 3 of 4. If it refuses, your credential is the problem, not the form. See the failure table at the end of [Lab 2](#).

Need the credential instructions again? Click **Open Guide** in the blue banner. The Authorization Guide panel covers all three credential methods and offers the least-privilege IAM policy files for download.

Step 4: Configure (Step 3 of 4)

Task 1 of 3 lists four integration types, each a toggle that starts off:

Toggle	Tagged as	What it gives you
Agentless Workload Scanning	Cloud Vulnerability Management	Vulnerability and secret scanning with no agent on the instance
Configuration	Cloud Security Posture Management	Resource inventory, compliance assessment, risk analysis
CloudTrail	Cloud Threat Detection	AWS CloudTrail ingestion. Creates its own trail by default.
EKS Audit Log	Kubernetes Security Posture Management	EKS audit log ingestion. Leave it off unless this account runs EKS.

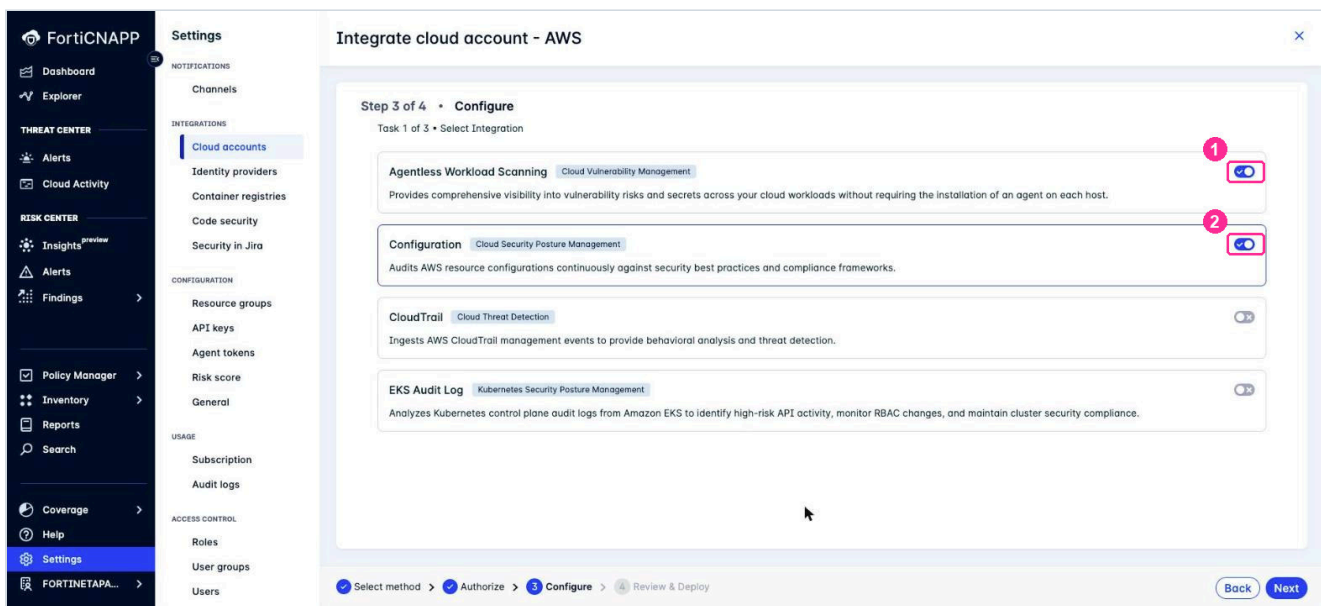
Turn on two toggles, and only two:

1. **Agentless Workload Scanning**
2. **Configuration**

Leave the CloudTrail and EKS Audit Log toggles off.

Important

CloudTrail comes next, in Lab 4. It uses CloudFormation instead, for reasons that lab explains. Leave the toggle off here.



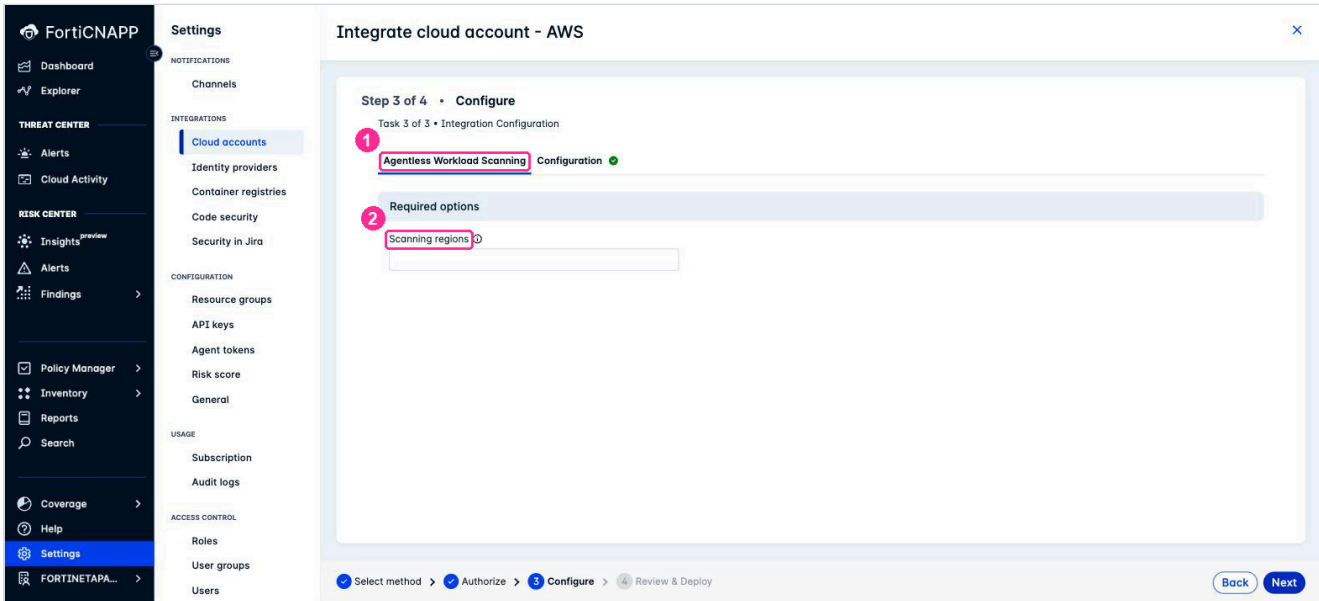
Two toggles here, the third arrives in Lab 4. Posture and vulnerability scanning from the wizard, threat detection from a CloudFormation template.

Set the agentless scanning regions

1. On the **Agentless Workload Scanning** tab, set **Scanning regions** to the region where your workloads run, for example **ap-southeast-1**.

Important

Scanning regions is required and starts empty. The wizard will not let you leave this step until you set it, and the error only appears once you try to advance.



2. On the **Configuration** tab, leave **Advanced options** alone.

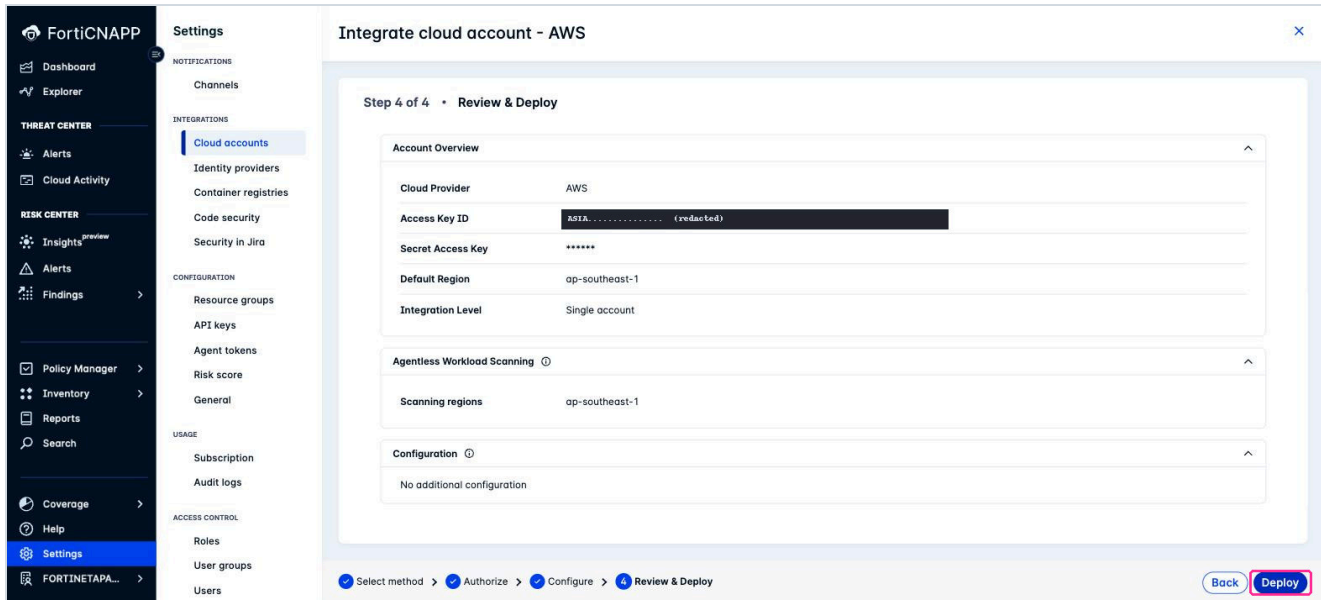
There are only two tabs. A tab appears per selected integration type, so CloudTrail and EKS Audit Log have none here.

The **Configuration** tab's **Advanced options** panel holds one setting, a **Use an existing IAM role** toggle, off by default. Turn it on when the account already has a Lacework cross-account role you want to keep. Automated configuration does not detect an existing role on its own.

Step 5: Review and Deploy (Step 4 of 4)

This step is a read-only summary. **Account Overview** repeats your credentials, region and integration level, then one panel per selected integration type repeats its settings.

1. Read **Account Overview** and confirm the region and integration level.
2. Read each integration panel.
3. Click **Deploy**.



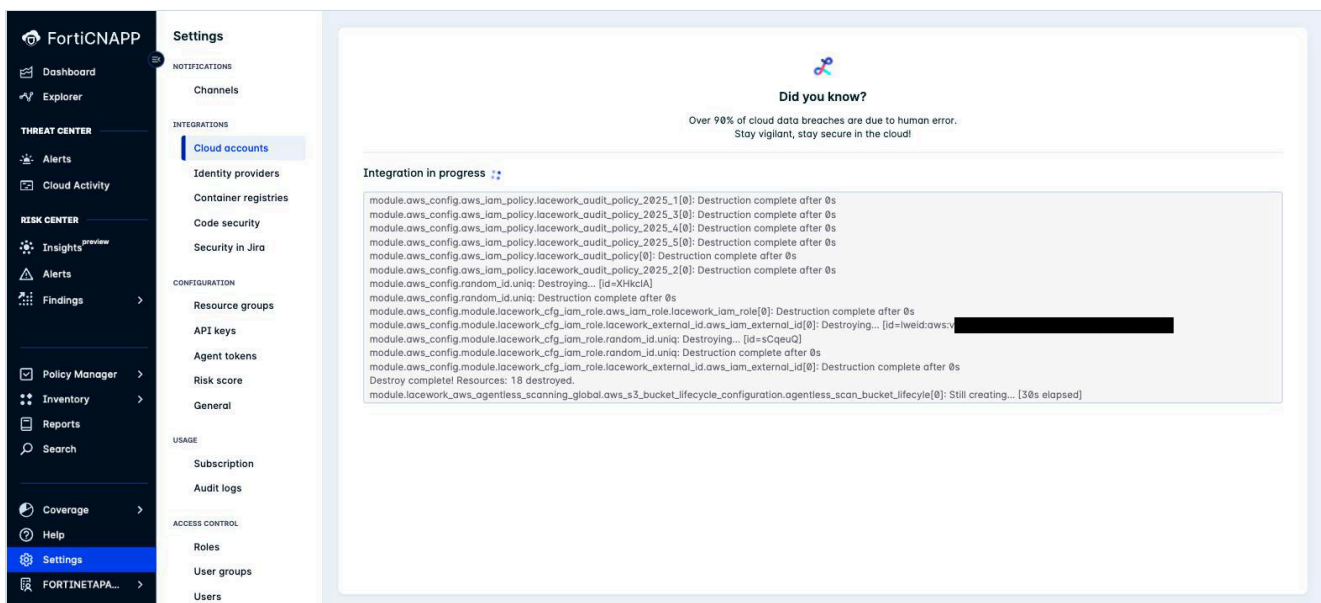
The panels collapse, they do not edit. Clicking a value does nothing. To change a setting, click **Back** and return to the Configure step.

Nothing has been created yet. Everything up to this point was a dry run against your account. The permission check already ran, in Task 2 of the Configure step, so a missing permission stops you there with an empty account rather than halfway through a deployment with half the resources built.

Step 6: Watch the Deployment

FortiCNAPP generates a Terraform plan, applies it, and registers each integration. It creates IAM roles and policies, storage, encryption keys and messaging services. Where it needs logic that infrastructure provisioning cannot express, such as registering the integration itself, it deploys short-lived Lambda helpers and then removes them.

Wait for all selected integration types to complete. Allow 5 to 10 minutes. This is a good moment to stretch.



Checkpoint: every integration type you selected reads **SUCCEEDED**. If any reads **FAILED**, read the next box before you retry anything.

Every run is recorded under **Settings > Integrations > Cloud accounts > Deployment History**, successes and failures alike. That is where you troubleshoot a failed onboarding.

Step 7: Review What Was Created

The wizard closes on its own and leaves you on the deployment record.

1. Expand each integration type to view the cloud resources created for it.
2. Click **Cloud Accounts** in the breadcrumb when you have finished reading.

You can return to this record at any time. Go to **Settings > Integrations > Cloud accounts** and select the **Deployment History** tab.

Cloud Accounts > Deployment History > Deployment 1391

Deployment 1391

AWS Automated Configuration

Account ID: [REDACTED] Created: 9/10/2026 5:15 PM (+10:00)

[Integrations](#)

Overview

Account ID [REDACTED]

Caller Identity [REDACTED]:65:assumed-role/forticnapp-onboarding/lacework-onboarding

Integrations

Agentless Workload Scanning	GUID: FORTINET_94E7F221...	↓ Terraform files	SUCCEEDED ▾
Configuration	GUID: [REDACTED]	↓ Terraform files	SUCCEEDED ▾

The record repays reading properly:

- **Caller identity:** the exact AWS principal used to deploy. This is what answers "who created these resources" six months later.
- **Per-integration status:** SUCCEEDED or FAILED, integration by integration.
- **Resources:** every resource created, by ARN, name and type.
- **Terraform files:** download what FortiCNAPP generated for each integration.

That last one matters. **Automated configuration is not a black box.** It writes Terraform, and you can take that Terraform away. Onboard with the wizard, download the generated files, and commit them to your own repository.

Every resource is tagged with two keys, `lacework_tag` and `lacework_integration`. The deployment record shows the values for your run.

Remember these tags. Lab 10 uses them for cleanup.

Step 8: Verify the Integrations

1. Return to **Settings > Integrations > Cloud accounts**.
2. Confirm your AWS account ID appears in the list.
3. Confirm the **Integrations** column shows **Configuration** and **Agentless**. CloudTrail is not there yet. Lab 4 adds it with CloudFormation.

Step 9: Confirm the AWS Side

Switch to the AWS Console and confirm the resources exist.

1. Go to **IAM > Roles** and search for `lw-`. The Configuration integration's cross-account role is named `lw-iam-` plus a random suffix. Searching for `lacework` returns the four agentless roles instead.
2. Open the `lw-iam-` role and select the **Trust relationships** tab. The trusted principal is a role in the FortiCNAPP AWS account, and the `StringEquals` condition on it is the external ID.
3. Go to **Amazon ECS > Clusters**. Confirm the agentless scanner cluster exists. Its name starts `lacework-agentless-scanning-cluster-`.

Two integrations, two naming schemes. Configuration creates one role and a set of `lwaudit-policy-` policies. Agentless creates four roles, an ECS cluster, and its own networking. Both carry the `lacework_tag` key, which is how Lab 10 finds them.

Troubleshooting

All of these came up while building the lab.

Discovery fails on an AWS Organization trail

```
TrailNotFoundException: Unknown trail: aws-controltower-BaselineCloudTrail
for the user: <your account id>
```

Cause: the account is a member of an AWS Organization with an organization-wide CloudTrail, often created by Control Tower. A member account sees that trail as a shadow trail, and AWS requires the full trail ARN to look one up rather than the name.

What to do: this is why Step 4 has you leave the CloudTrail toggle off. [Lab 4](#) adds it with CloudFormation, which has no discovery step for the organization trail to break.

For CloudTrail coverage across a whole organization, run an **organization level** integration from the management account.

The screenshot shows the FortiCNAPP Settings interface. On the left is a navigation menu with sections: THREAT CENTER (Alerts, Cloud Activity), RISK CENTER (Insights, Alerts, Findings), Policy Manager, Inventory, Reports, Search, Coverage, and Help. The 'Settings' option is selected. The main panel is titled 'Integrate cloud account - AWS' and shows 'Step 3 of 4 - Configure'. A red error box states: 'Failed to proceed operation error CloudTrail: GetTrail, https response error StatusCode: 400, RequestID: cea8997d-2508-48f3-bac2-673b5972b258, TrailNotFoundException: Unknown trail: aws-controltower-BaselineCloudTrail for the user: [redacted]'. Below the error is a 'Return and try again' button. At the bottom, a progress bar shows steps: Select method, Authorize, Configure (current), and Review & Deploy. 'Back' and 'Next' buttons are also present.

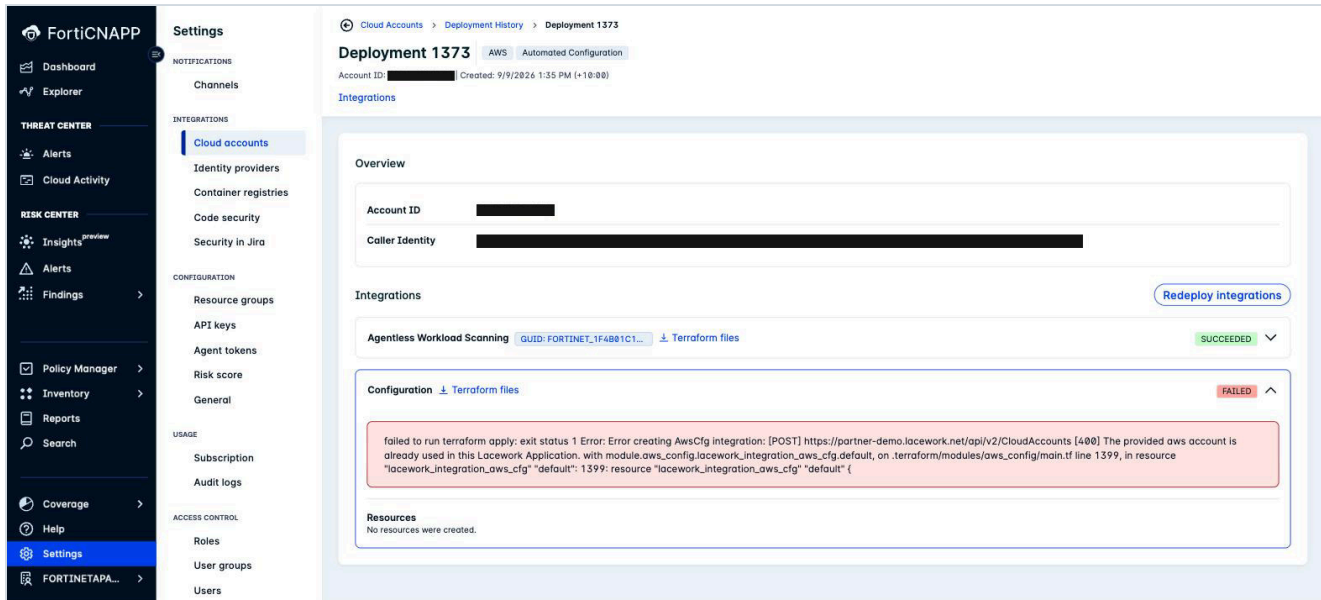
The AWS account already has that integration type

Error creating AwsCfg integration: [400]
The provided aws account is already used in this Lacework Application.

Cause: a Configuration integration already exists for this AWS account in this FortiCNAPP tenant.

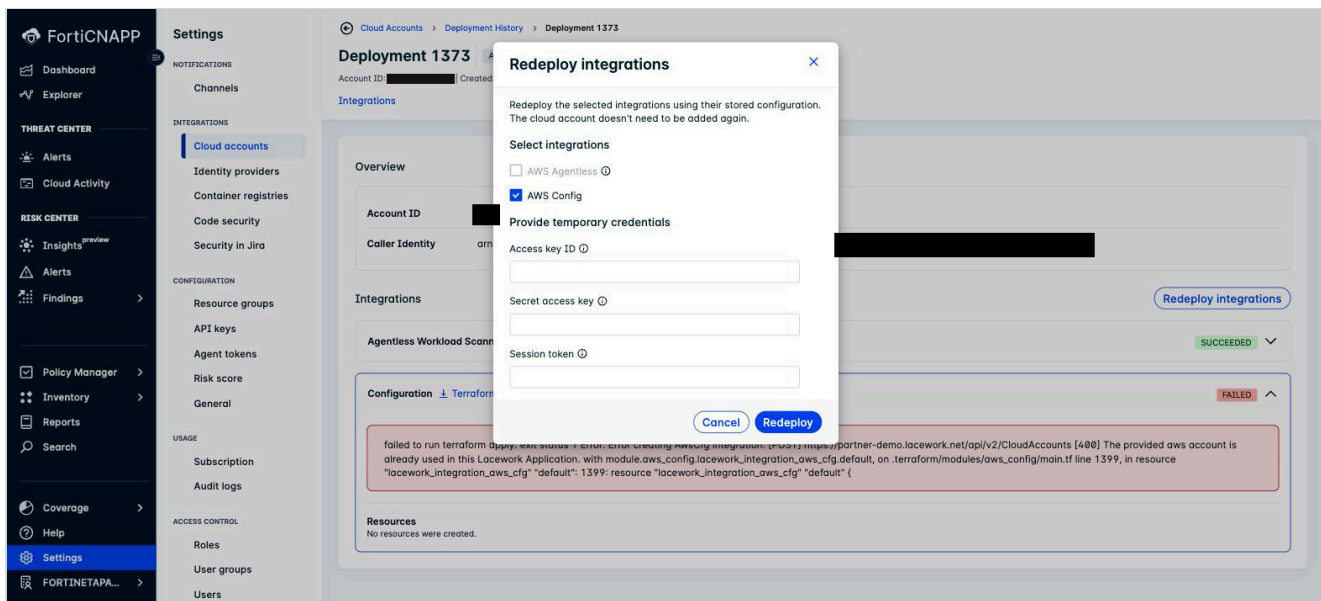
What happens: the run completes its Terraform stage, then rolls back cleanly. Nothing is left behind in AWS.

What to do: delete the existing integration first, then use **Redeploy integrations**.



Retrying without restarting the wizard

The deployment record has a **Redeploy integrations** button. It reuses the stored configuration, so you do not add the cloud account again. Integration types that already succeeded are greyed out. You do have to supply temporary credentials again.



Data timing

Data does not appear instantly.

Data	First appears
CloudTrail events	Within 15 minutes of the trail being created in Lab 12
Resource inventory and compliance	Up to 24 hours on the scheduled cycle
Agentless scan results	After the first scan, on a 24 hour cycle by default

To avoid waiting for the inventory scan, Lab 7 triggers one on demand with the Lacework CLI.

Where the data goes

FortiCNAPP analyses data **inside** your cloud account. File contents and resource contents never leave your network. Only assessment results are sent to the FortiCNAPP platform. Those results are stored as JSON files in a single storage bucket in your own account. You can read them and see exactly what you are sending.

Source: the Authorization Guide panel in the wizard, "Data privacy and security".

What did we do here?

We onboarded an AWS account into FortiCNAPP from one wizard: configuration assessment and agentless vulnerability scanning. Lab 4 adds CloudTrail threat detection with a CloudFormation template.

Compare the work:

	CloudFormation path	Automated configuration
CloudFormation stacks you launch	2	0
Stack parameters you set by hand	Several, including a quota check	0
Preflight permission check	None	Yes, including SCPs and permission boundaries
Behaviour on failure	Stack fails, integration record remains	Full rollback
Cleanup	Delete each stack, then delete the integration record separately	Tag search

The trade is credentials. The CloudFormation path never asks for an AWS credential, because you launch each stack yourself. Automated configuration needs one bounded credential. Usually worth it, because temporary STS credentials keep the exposure contained.

Additional resources

- [FortiCNAPP Administration Guide: Automated configuration](#)
-

Next: [Lab 4: Add CloudTrail with CloudFormation.](#)

Lab 4: Add CloudTrail with CloudFormation

Objectives

Lab 3 onboarded Configuration and Agentless from the wizard and deliberately left CloudTrail off. This lab adds it, using a different method.

CloudTrail is what gives FortiCNAPP **threat detection**: who called which API, from where, and whether that is normal for this account. Configuration tells you a bucket is public. CloudTrail tells you who made it public, at 3am, from an IP the account has never seen.

	Automated Configuration, Lab 3	CloudFormation, this lab
Who builds the resources	FortiCNAPP, from credentials you hand over	You, from a template, in your own console
What it needs from you	Short-lived credentials	Nothing. You are already signed in
Where it runs	FortiCNAPP's side	Your AWS account
Best for	Fast onboarding of a whole account	Estates where CloudFormation is the standard, and any account the wizard cannot read

You will run a CloudFormation template that creates a new CloudTrail trail and registers it with FortiCNAPP in one pass.

Prerequisites

- Completed [Lab 3](#)
- AWS Console access to the same account, with permission to create IAM resources
- FortiCNAPP console access, tenant **FORTINETAPACDEMO**

Why this account needs CloudFormation

Your student account is a member of an AWS Organization, and that organization has its own **organization trail**. You can see it, but you cannot manage it. Run this from CloudShell:

```
aws cloudtrail describe-trails --region ap-southeast-1 \  
--query "trailList[].[Name,IsOrganizationTrail]" --output text
```

```
aws-controltower-BaselineCloudTrail    True
```

`True` means the trail belongs to the organization's management account, not to you. You may see more than one such row.

Automated Configuration inspects existing trails while it works out what to build, and it cannot read that one. So it stops, and nothing gets created:

TrailNotFoundException: Unknown trail: aws-controltower-BaselineCloudTrail

CloudFormation takes a different route. You run the template yourself, in your own console, so there is no inspection step and nothing to be blocked by. The organization trail is simply irrelevant.

This is not a workshop-only situation. Most corporate AWS accounts sit inside an organization, and Control Tower creates an organization trail by default.

Lab steps

Step 1: Open the CloudFormation Method

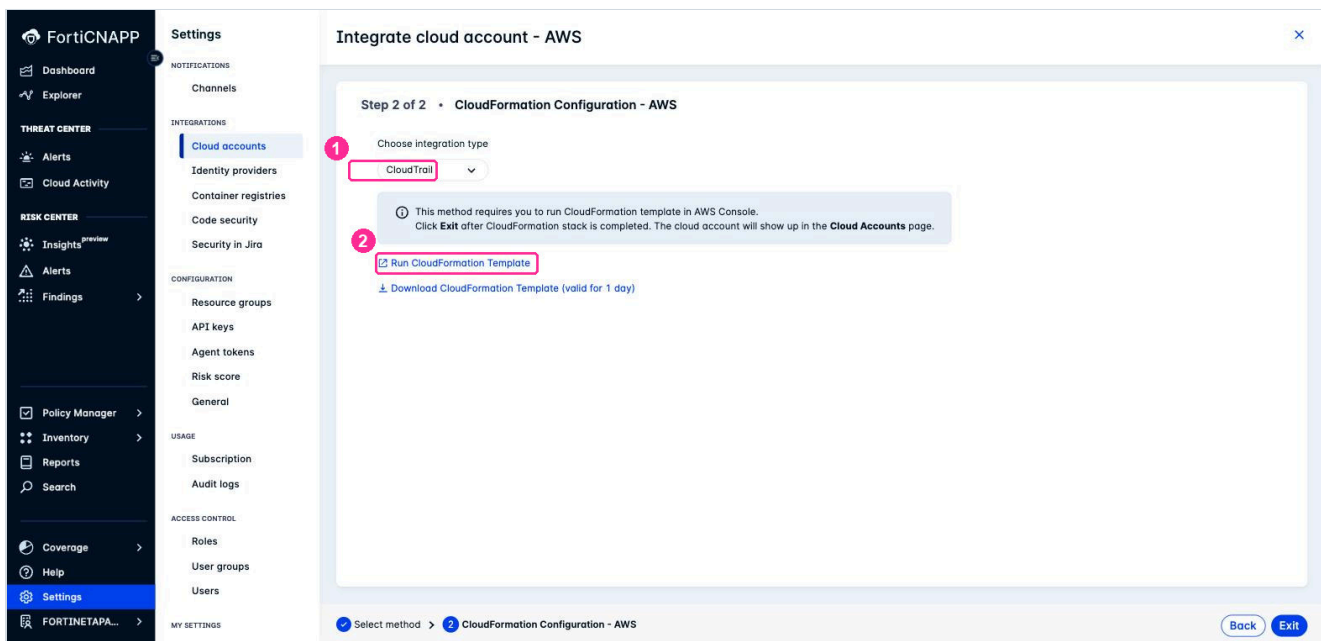
1. In FortiCNAPP, go to **Settings > Integrations > Cloud accounts**.
2. Confirm the tenant selector at the bottom left shows **FORTINETAPACDEMO**.
3. Click **Add New**.
4. Under **Cloud Service Provider**, select **Amazon Web Services**.
5. Under **Integration Method**, select **AWS CloudFormation**.
6. Click **Next**. The header now reads **Step 2 of 2 - CloudFormation Configuration - AWS**.

Step 2: Choose CloudTrail

1. On **CloudFormation Configuration - AWS**, open **Choose integration type**.
2. Select **CloudTrail**.

The list also offers **CloudTrail+Configuration (Control Tower)**. That one is the right answer when you are onboarding a whole Control Tower organization from its management account. Here it would create a second Configuration integration on top of the one Lab 3 already made, so leave it alone.

3. Click **Run CloudFormation Template**.



Important

This opens a new browser tab, in whichever AWS account and region your most recent console session was using. Check both before you go any further.

Step 3: Check the Account and Region

In the new tab, look at the top right of the AWS console.

1. Confirm the **account** is the same one you onboarded in Lab 3.
2. Set the **region selector** to **Asia Pacific (Singapore) ap-southeast-1**.

Checkpoint: the URL contains `region=ap-southeast-1` and the account number matches Lab 3.

Getting this wrong does not fail loudly. The stack builds successfully in the wrong region and your trail quietly logs somewhere nobody is looking.

Step 4: Create the Stack

The template URL is already filled in.

1. On **Create stack**, leave **Choose an existing template** and the prefilled **Amazon S3 URL** as they are. Click **Next**.
2. In **Stack name**, enter `forticnapp-cloudtrail`.
3. Review the parameters. The defaults are correct:

Parameter	Value	What it does
Resource name prefix	your tenant name	Prefixes every resource so names do not collide
Create new trail?	Yes	Creates a new multi-region trail, plus its own S3 bucket and SNS topic
API Token	prefilled	How the stack registers the integration back to FortiCNAPP
Enable Kms Key Rotation	true	Rotates the KMS key that encrypts the logs

Existing Trail Setup stays blank. Those fields are for pointing FortiCNAPP at a trail you already have, which is not what we are doing.

New Trail Options stays blank too. **Log file prefix** only renames the log files.

4. Click **Next**.

Step 5: Acknowledge, Then Submit

On **Configure stack options**, leave every setting alone and scroll to the bottom.

1. Under **Capabilities**, tick **I acknowledge that AWS CloudFormation might create IAM resources with custom names**.
2. Click **Next**.
3. On **Review and create**, scroll to the bottom and click **Submit**.

Warning

The acknowledgement is on **Configure stack options**, not on the review page. It sits below the fold at the bottom of a page that otherwise needs no input, so it is easy to scroll past. Click **Next** without it and the page answers `Please acknowledge all checkboxes before proceeding`.

The stack takes **one to two minutes**. Wait for **CREATE_COMPLETE**.

Checkpoint: the stack status is `CREATE_COMPLETE` and no events show `CREATE_FAILED`.

Step 6: Tell FortiCNAPP You Are Done

1. Return to the FortiCNAPP tab, still showing **Step 2 of 2**.
2. Click **Exit**.

The integration does not appear instantly. The stack registers itself through a callback, so give it **up to a minute**.

Step 7: Verify Both Sides

In **FortiCNAPP**, go to **Settings > Integrations > Cloud accounts**. Your account now carries three integrations, so the **Integrations** column shows two and collapses the rest behind **+1 more**. Open it and confirm **Configuration**, **Agentless** and **CloudTrail** are all listed.

The CloudTrail chip is grey at first, not green. Registration lands before any log data does. It turns green once events start arriving, within about 15 minutes.

In **AWS**, run the same command from the top of this lab:

```
aws cloudtrail describe-trails --region ap-southeast-1 \
  --query "trailList[].[Name,IsOrganizationTrail]" --output text
```

A new row has appeared, and the value in the second column is the point of this lab:

aws-controltower-BaselineCloudTrail	True
fortinetapacdemo-laceworkcws	False

`False` means this one is yours. Your account owns it, and FortiCNAPP can read it.

The trail is named from the **Resource name prefix** parameter, not from the stack name, so it reads `fortinetapacdemo-laceworkcws` rather than `forticnapp-cloudtrail`.

What did we do here?

We added the third integration type by a route that does not care about the organization trail, and we did it without handing FortiCNAPP any credentials.

Worth taking away:

The method matters as much as the product. Same integration, same result, but Automated Configuration and CloudFormation reach it differently, and only one of them works in an account like this. When onboarding stalls, changing method is often faster than fixing the account.

One CloudFormation stack now owns everything it built, including the FortiCNAPP integration record. [Lab 10](#) uses that: deleting the stack removes the AWS resources and deregisters the integration in a single

operation.

Troubleshooting

Next does nothing on Configure stack options

The page shows `Please acknowledge all checkboxes before proceeding` under the **Capabilities** panel. Tick the acknowledgement. See Step 5.

The stack built, but nothing appears in FortiCNAPP

Give it a minute, then click **Exit** in the FortiCNAPP tab if you have not already. The registration happens through a callback from the stack, not from the console.

If it still does not appear, check the stack's **Events** tab for a failure on `LaceworkSnsCustomResource`. That resource is the callback.

The stack is in the wrong region

Delete it and run Step 3 again. Nothing else in the workshop depends on it yet, so this is cheap to fix now and annoying to fix later.

CREATE_FAILED on an IAM resource

You do not have permission to create IAM roles in this account. The template needs it, and the acknowledgement in Step 5 is your confirmation that you expect it.

Reference

- [FortiCNAPP Administration Guide: Automated configuration](#)
 - [AWS: Creating a trail for an organization](#)
-

Next: [Lab 5: Install Linux Agent](#).

Lab 5: Install Linux Agent

Objectives

Lab 3 turned on agentless scanning. That reads your account from the outside: it takes a snapshot of each disk, finds the packages and the secrets, and tells you what is vulnerable. It never touches the running machine.

This lab installs the **agent**, which sits inside the machine and watches it work.

	Agentless, Lab 3	Agent, this lab
Where it runs	Outside, on a copy of the disk	On the host itself
What it sees	What is installed	What is actually happening: processes, connections, file changes, logins
How often	Periodic snapshot	Continuously, reporting hourly
To install	Nothing	One script

Neither replaces the other. Agentless tells you a host **could** be exploited. The agent tells you something **is** behaving strangely on it, which is what a baseline and an anomaly alert are built from.

You will launch a Linux EC2 instance, install the agent on it, and confirm both ends agree it is running.

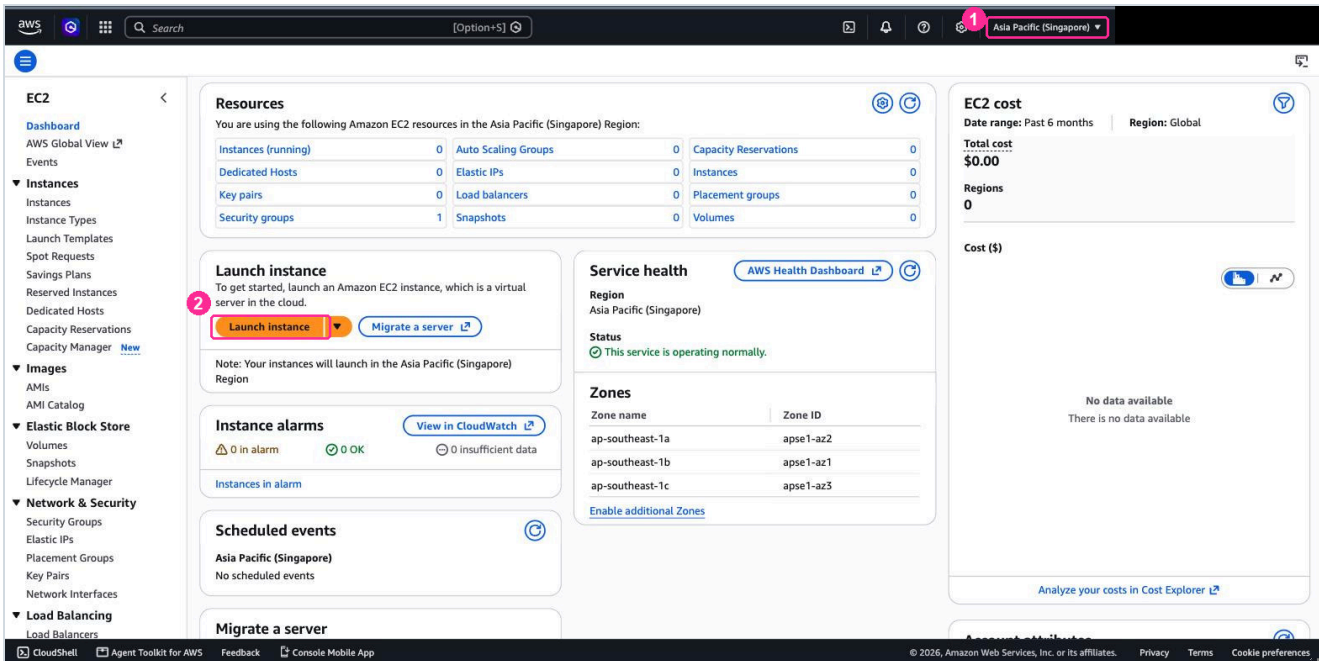
Prerequisites

- Completed [Lab 3](#)
- AWS account with permission to launch EC2 instances
- FortiCNAPP console access, tenant **FORTINETAPACDEMO**

Lab steps

Step 1: Create Linux EC2 Instance

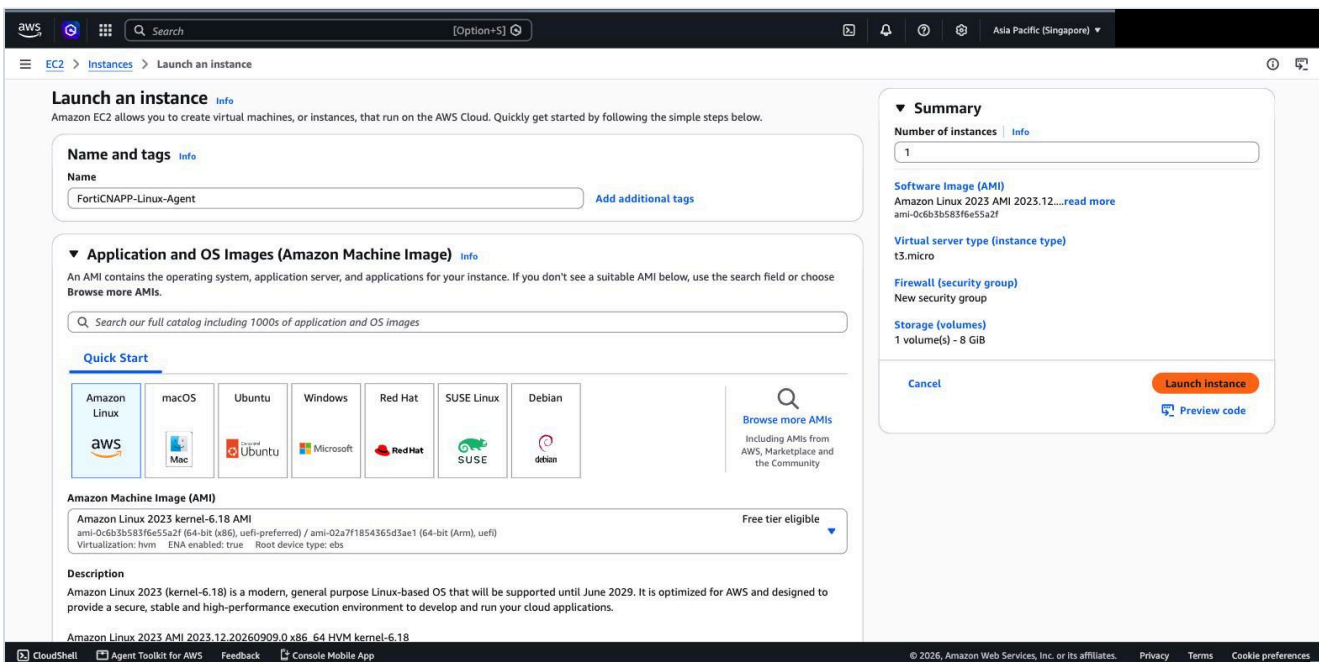
1. Open the **EC2** service. Check the region selector reads **Asia Pacific (Singapore)** before you do anything else.



2. Click **Launch instance**.

3. **Name:** FortiCNAPP-Linux-Agent

4. **Application and OS Images:** leave **Amazon Linux 2023** selected. It is the default, and the agent installer does not care which distribution you pick.



5. **Instance type:** leave the default, **t2.micro**. It is free tier eligible.

6. **Key pair (login):** open the dropdown and choose **Proceed without a key pair (Not recommended)**.

You will connect through EC2 Instance Connect in the browser, which issues its own temporary key. There is nothing for you to download or keep.

aws [Search] [Option+S]

EC2 > Instances > Launch an Instance

Amazon Linux 2023 AMI 2023.12.20260909.0 x86_64 HVM kernel-6.18

Architecture: 64-bit (x86) Boot mode: uefi-preferred AMI ID: ami-0c6b3b583f6e55a2f Creation Date: 2026-09-09 Username: ec2-user Verified provider

▼ Instance type Info | Get advice

Instance type: t3.micro
 Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Windows base pricing: 0.0224 USD per Hour
 On-Demand SUSE base pricing: 0.0132 USD per Hour On-Demand Linux base pricing: 0.0132 USD per Hour
 On-Demand Ubuntu Pro base pricing: 0.0167 USD per Hour On-Demand RHEL base pricing: 0.042 USD per Hour

Free tier eligible ☐ All generations Compare instance types

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required
 Select Create new key pair

Proceed without a key pair (Not recommended) Default value Edit

Network: vpc-05121eff2596b9b1e
 Subnet: No preference (Default subnet in any availability zone)
 Auto-assign public IP

▼ Summary

Number of instances: 1 Info

Software Image (AMI)
 Amazon Linux 2023 AMI 2023.12...read more
 ami-0c6b3b583f6e55a2f

Virtual server type (instance type)
 t3.micro

Firewall (security group)
 New security group

Storage (volumes)
 1 volume(s) - 8 GiB

Cancel Launch instance Preview code

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws [Search] [Option+S]

EC2 > Instances > Launch an Instance

Amazon Linux 2023 AMI 2023.12.20260909.0 x86_64 HVM kernel-6.18

Architecture: 64-bit (x86) Boot mode: uefi-preferred AMI ID: ami-0c6b3b583f6e55a2f Creation Date: 2026-09-09 Username: ec2-user Verified provider

▼ Instance type Info | Get advice

Instance type: t3.micro
 Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Windows base pricing: 0.0224 USD per Hour
 On-Demand SUSE base pricing: 0.0132 USD per Hour On-Demand Linux base pricing: 0.0132 USD per Hour
 On-Demand Ubuntu Pro base pricing: 0.0167 USD per Hour On-Demand RHEL base pricing: 0.042 USD per Hour

Free tier eligible ☐ All generations Compare instance types

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required
 Proceed without a key pair (Not recommended) Default value Create new key pair

▼ Network settings Info Edit

Network: vpc-05121eff2596b9b1e
 Subnet: No preference (Default subnet in any availability zone)
 Auto-assign public IP

▼ Summary

Number of instances: 1 Info

Software Image (AMI)
 Amazon Linux 2023 AMI 2023.12...read more
 ami-0c6b3b583f6e55a2f

Virtual server type (instance type)
 t3.micro

Firewall (security group)
 New security group

Storage (volumes)
 1 volume(s) - 8 GiB

Cancel Launch instance Preview code

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

7. **Network settings:** leave them alone. The wizard creates a security group called `launch-wizard-1` allowing SSH from anywhere, which is what Instance Connect needs.

Warning

This is the step that breaks the lab. Do not switch to **Select existing security group** and pick the VPC's default group. That one only allows traffic between resources that share it, so Instance Connect cannot reach your instance and you get a timeout with no useful error.

8. **Configure storage**: leave the default, 8 GiB gp3.
9. Click **Launch instance**, then **View all instances**.
10. Wait for **Instance state** to read **Running**. Allow about a minute.

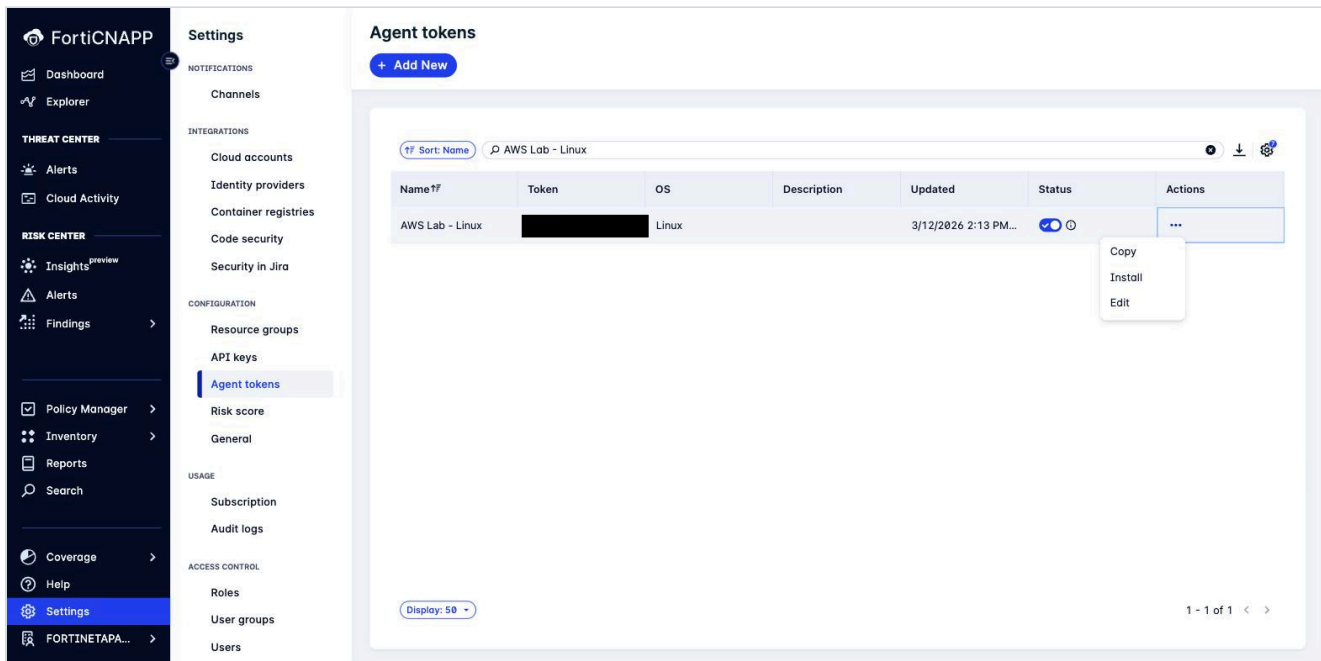
Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic
FortiCNAPP-Linux-Agent	i-09685693596e2a070	Running	t3.micro	Initializing	ap-southeast-1c	ec2-13-229-149-116.ap...	13.229.149.116	-

Checkpoint: one instance, state **Running**, and it has a public IPv4 address.

Step 2: Get the Agent Install URL from FortiCNAPP

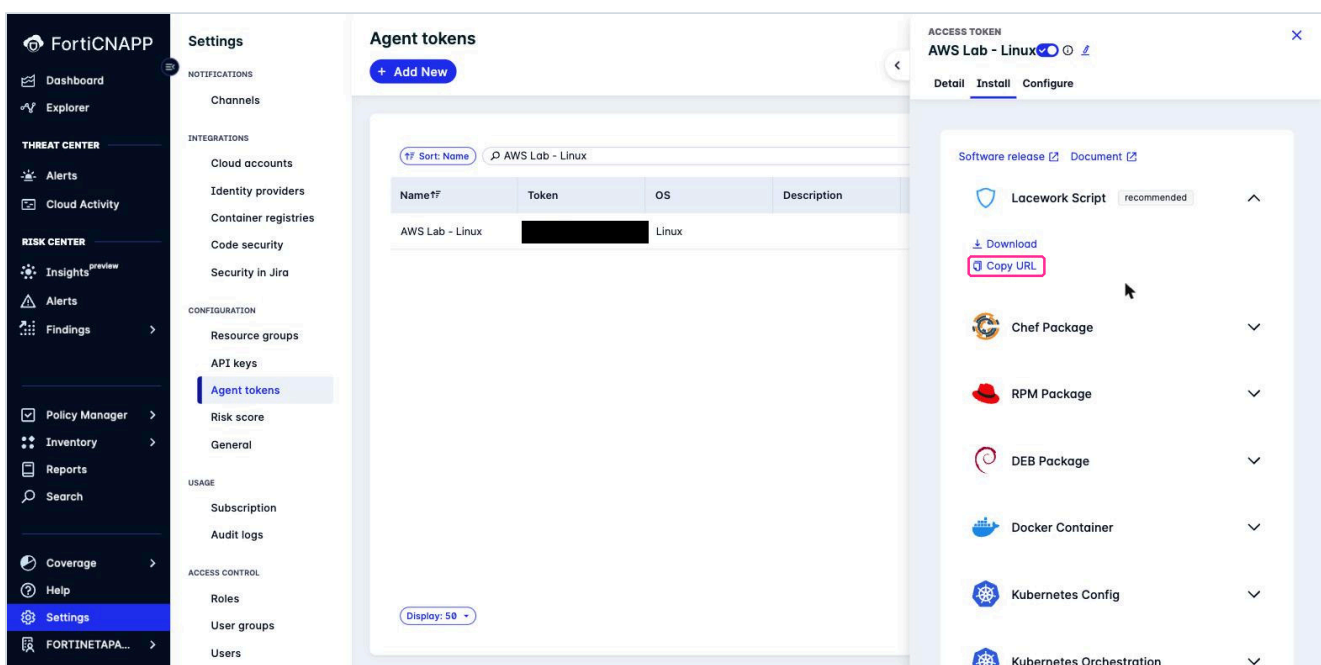
1. Log into FortiCNAPP console at <https://partner-demo.lacework.net/>

2. Ensure tenant is set to **FORTINETAPACDEMO**
3. Navigate to **Settings > Configuration > Agent tokens**
4. Type **AWS Lab – Linux** in the search box. There are dozens of tokens on this tenant, so searching beats scrolling.
5. Click the **Actions** ellipsis (three dots) on that row.



6. Click **Install**. **Lacework Script** is already expanded, so click **Copy URL**.

Copy the URL, not the script. The install command in the next step fetches it.

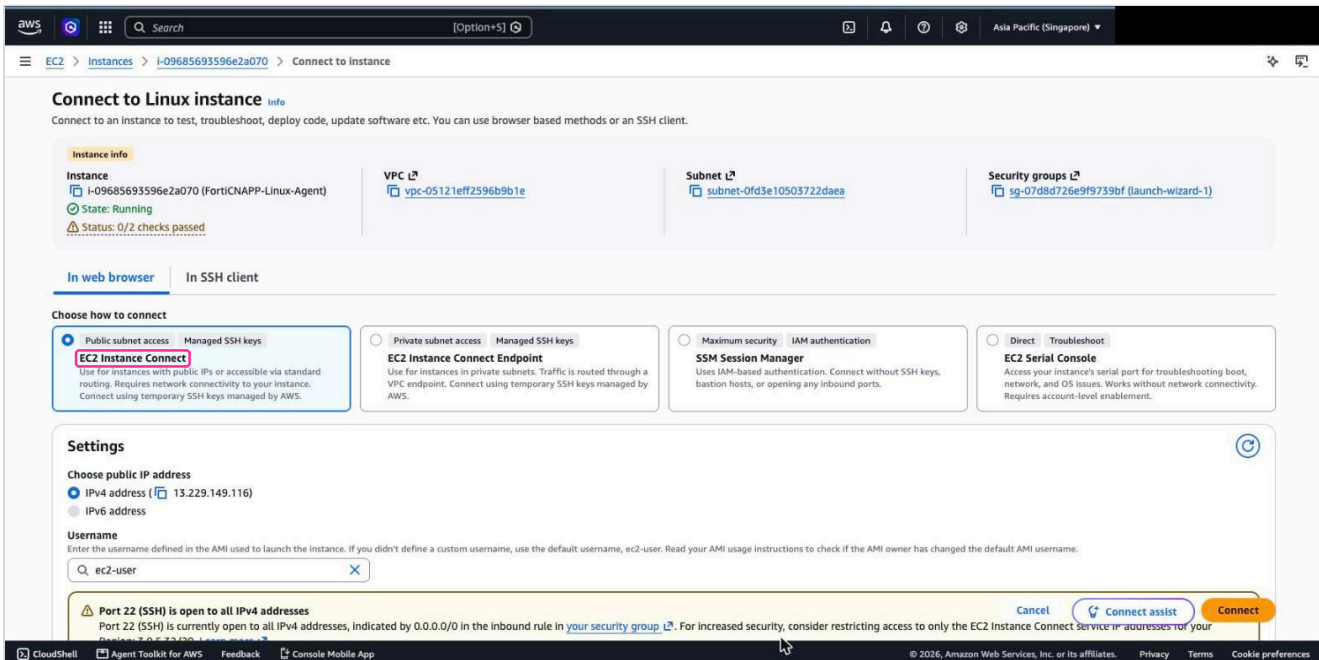


Checkpoint: you have a URL on your clipboard that starts with `https://` .

Step 3: Connect to the Instance

1. Go to **EC2 > Instances**.
2. Tick the checkbox next to **FortiCNAPP-Linux-Agent** .

- Click **Connect** at the top of the page.
- Stay on the **In web browser** tab and leave **EC2 Instance Connect** selected. It is the first of the four cards and it is already chosen.



- Leave **Username** as `ec2-user` , then click **Connect**.

A terminal opens in the browser. No key pair, no SSH client, nothing to install.

- Confirm the shell can do what the installer needs:

```
sudo whoami      # must print: root
ping -c 3 8.8.8.8 # the agent needs to reach the internet
```

Checkpoint: `sudo whoami` prints `root` and the pings come back.

Step 4: Install the Agent

In the EC2 Instance Connect terminal, run the following commands:

Paste the URL from Step 2 where shown. Keep the `-O install.sh` , which names the downloaded file for you rather than trusting whatever the URL ends in.

```
wget -O install.sh "<paste-the-copied-url-here>"
chmod +x install.sh
sudo ./install.sh
```

The script reads the agent token from the URL, so it runs straight through. It checks connectivity, downloads the agent package, installs it, and registers with FortiCNAPP.

Checkpoint: the last line reads `Lacework successfully installed` .

If the script stops at `Please enter access token:` instead, it did not get the token from the URL. Go back to FortiCNAPP, click the **Actions** ellipsis on the same token, and choose **Copy**. Paste the 56-character token at the prompt and press Enter.

Caution

The install URL from Step 2 carries the agent token in its path, so treat it as a secret. Take care if you are sharing your screen.

Tip

Pasting into the browser terminal. Use **Ctrl+V** (**Cmd+V** on a Mac). If the paste arrives empty, your clipboard did not survive the tab switch. Go back and copy again.

Step 5: Verify on the Host

Ask the agent how it is doing:

```
sudo /var/lib/lacework/datacollector -status
```

It answers in JSON. The field that matters is `Status` :

```
{"Version":2,"Datacollector":{"Status":"ACTIVE", "...}}
```

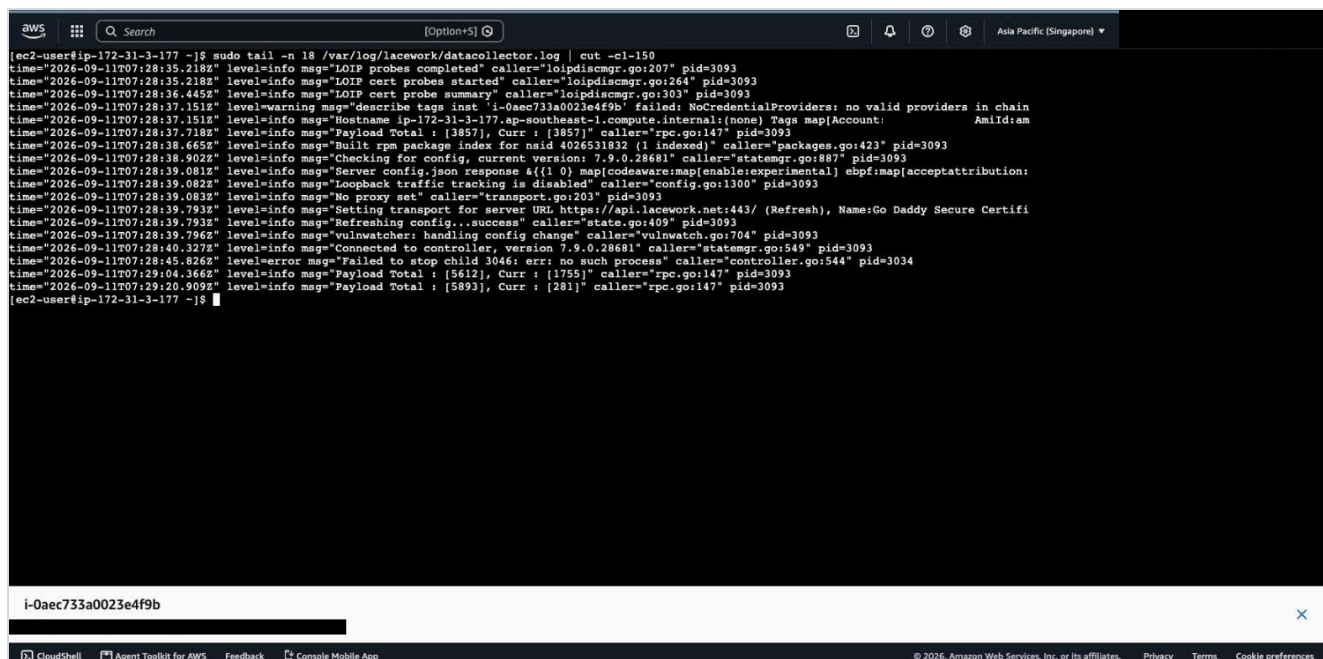
ACTIVE means it is running and talking to FortiCNAPP.

Checkpoint: Status reads ACTIVE .

Look at what it is logging

```
sudo tail -f /var/log/lacework/datacollector.log
```

Ctrl+C stops the tail. The file is readable by `root` only, so the `sudo` matters.



```
aws [Option+S]
[ec2-user@ip-172-31-3-177 ~]$ sudo tail -n 18 /var/log/lacework/datacollector.log | cut -c1-150
time="2026-09-11T07:28:35.218Z" level=info msg="LOIP probes completed" caller="loipdiscmgr.go:207" pid=3093
time="2026-09-11T07:28:35.218Z" level=info msg="LOIP cert probes started" caller="loipdiscmgr.go:264" pid=3093
time="2026-09-11T07:28:36.445Z" level=info msg="LOIP cert probe summary" caller="loipdiscmgr.go:303" pid=3093
time="2026-09-11T07:28:37.151Z" level=warning msg="describe tags inst 'i-0aac733a0023e4f9b' failed: NoCredentialProviders: no valid providers in chain
time="2026-09-11T07:28:37.151Z" level=info msg="Hostname ip-172-31-3-177.ap-southeast-1.compute.internal:(none) Tags map[Account: Amid:am
time="2026-09-11T07:28:37.718Z" level=info msg="Payload Total : {3857}, Curr : {3857}" caller="rpc.go:147" pid=3093
time="2026-09-11T07:28:38.665Z" level=info msg="Built rpm package index for naid 4026531832 (1 indexed)" caller="packages.go:423" pid=3093
time="2026-09-11T07:28:38.802Z" level=info msg="Checking for config, current version: 7.9.0.28681" caller="statemgr.go:587" pid=3093
time="2026-09-11T07:28:39.081Z" level=info msg="Server config.json response &{(1 0) map[codeaware:map[enable:experimental] ebpf:map[acceptattribution:
time="2026-09-11T07:28:39.082Z" level=info msg="Loopback traffic tracking is disabled" caller="config.go:1300" pid=3093
time="2026-09-11T07:28:39.083Z" level=info msg="No proxy set" caller="transport.go:203" pid=3093
time="2026-09-11T07:28:39.793Z" level=info msg="Setting transport for server URL: https://api.lacework.net:443/ (Refresh), Name:Go Daddy Secure Certifi
time="2026-09-11T07:28:39.793Z" level=info msg="Refreshing config...success" caller="state.go:409" pid=3093
time="2026-09-11T07:28:39.796Z" level=info msg="vulnwatcher: handling config change" caller="vulnwatch.go:704" pid=3093
time="2026-09-11T07:28:40.327Z" level=info msg="Connected to controller, version 7.9.0.28681" caller="statemgr.go:549" pid=3093
time="2026-09-11T07:28:45.826Z" level=error msg="Failed to stop child 3046: err: no such process" caller="controller.go:544" pid=3034
time="2026-09-11T07:29:04.366Z" level=info msg="Payload Total : {5612}, Curr : {1755}" caller="rpc.go:147" pid=3093
time="2026-09-11T07:29:20.309Z" level=info msg="Payload Total : {5893}, Curr : {281}" caller="rpc.go:147" pid=3093
[ec2-user@ip-172-31-3-177 ~]$
```

These lines are worth finding:

Line	Means
Setting transport for server URL https://api.lacework.net:443/	Where it sends data
Connected to controller, version 8.2.0.30827	It registered successfully. The version moves, the line does not
Payload Total : [5612], Curr : [1755]	Data leaving the host, every 15 to 20 seconds

Watch for a minute and the `Payload` line repeats. That is your agent reporting.

This log is the agent talking about itself, not a feed of what it sees. The processes, connections and file changes go to FortiCNAPP, not to this file. What the log proves is that the pipe is open.

Several `level=warning` lines look alarming and are not. On a stock Amazon Linux 2023 instance, agent 8.2, you get seven of them:

- `describe tags inst 'i-...' failed: not found` means the instance has no IAM role, so the agent cannot read its own tags. It does not need to
- `Failed to connect to docker engine ... Is the docker daemon running?` means there is no Docker on the host, which is true
- `child death (0: 8.2.0.30827) and lwtime config changed ... requires datacollector restart` are startup noise. The agent restarts itself and carries on
- `CAA: unknown FMODE_NONOTIFY value` is a kernel constant the agent does not recognise

The wording moves between agent versions. Judge by `Status: ACTIVE` and a repeating `Payload` line, not by a clean log.

Step 6: Verify in FortiCNAPP

The agent reports hourly, so it will not appear straight away. That is expected, not a failure. Carry on to Lab 6 and come back to this step later in the session.

In the console, go to **Inventory** and look for the host by its instance ID.

Once you have done [Lab 7](#) you can ask the same question from CloudShell instead:

```
lacework agent list
```

Look for your instance hostname. Nothing there yet means the first check-in has not landed.

The two checks answer different questions. `datacollector -status` on the host says the agent is running. `lacework agent list` says FortiCNAPP has heard from it. You can have the first without the second. That gap is almost always a security group or a route, not the agent.

What did we do here?

The host now reports what it is doing, not just what is installed on it.

Agentless found the vulnerable package. The agent is what notices the package being exploited: a process that has never run before, a connection to somewhere this host has never talked to, a login at the wrong hour. You cannot alert on behaviour you are not watching.

The Windows install differs enough to be worth seeing once.

Next: [Lab 6: Install Windows Agent](#).

Lab 6: Install Windows Agent

Objectives

Same agent, same result, different road to get there. Almost every estate you will meet runs both, so it is worth seeing Windows once rather than assuming it follows.

	Linux, Lab 5	Windows, this lab
Get in	EC2 Instance Connect, in the browser	RDP, from your own machine
Credential	None	A key pair you download, to decrypt the Administrator password
Installer	One shell script	PowerShell script plus an MSI
You need	A URL	Two URLs and an access token
Runs as	<code>datacollector</code>	The <code>LWDataCollector</code> service

The lab has two parts:

- **Part 1, everyone.** Create your own Windows agent token and build the install command from it. In Lab 5 you used a token someone made for you. This time you make it.
- **Part 2, optional.** Launch a Windows instance and run that command over RDP. RDP is slow from a room full of laptops, so you can read this part and skip to Lab 7.

Prerequisites

- Completed [Lab 5](#)
- FortiCNAPP console access, tenant **FORTINETAPACDEMO**
- For Part 2 only:
 - AWS account with permission to launch EC2 instances
 - An RDP client: Remote Desktop Connection on Windows, **Windows App** on a Mac

Part 1: Create a Token and Build the Install Command

Step 1: Create a Windows Agent Token

An agent token is the credential that joins an agent to this tenant. Anyone who holds it can register a host here, so treat it as a secret.

1. In the FortiCNAPP console, confirm the tenant reads **FORTINETAPACDEMO**.
2. Go to **Settings > Configuration > Agent tokens**, and click **Add New**.
3. **Name:** `AWS Lab - Windows - <your initials>`
4. **Operating System:** select **Windows**. Linux is selected by default.
5. Click **Save**.

Create access token

Name:

Description:

Operating System: ☐ Linux ☒ Windows

[Cancel](#) [Save](#)

Important

Put your initials in the name. Everyone in the room makes a token in this step, on a tenant that already holds dozens. Without your initials you cannot tell yours apart.

Checkpoint: the console shows **Token is created**, and your token is in the list with OS **Windows**.

Step 2: Open the Install Panel

1. Type your token's name in the search box.
2. Click the **Actions** ellipsis on your row, then **Install**.

Agent tokens

[+ Add New](#)

Sort: Name AWS Lab - Windows - AB

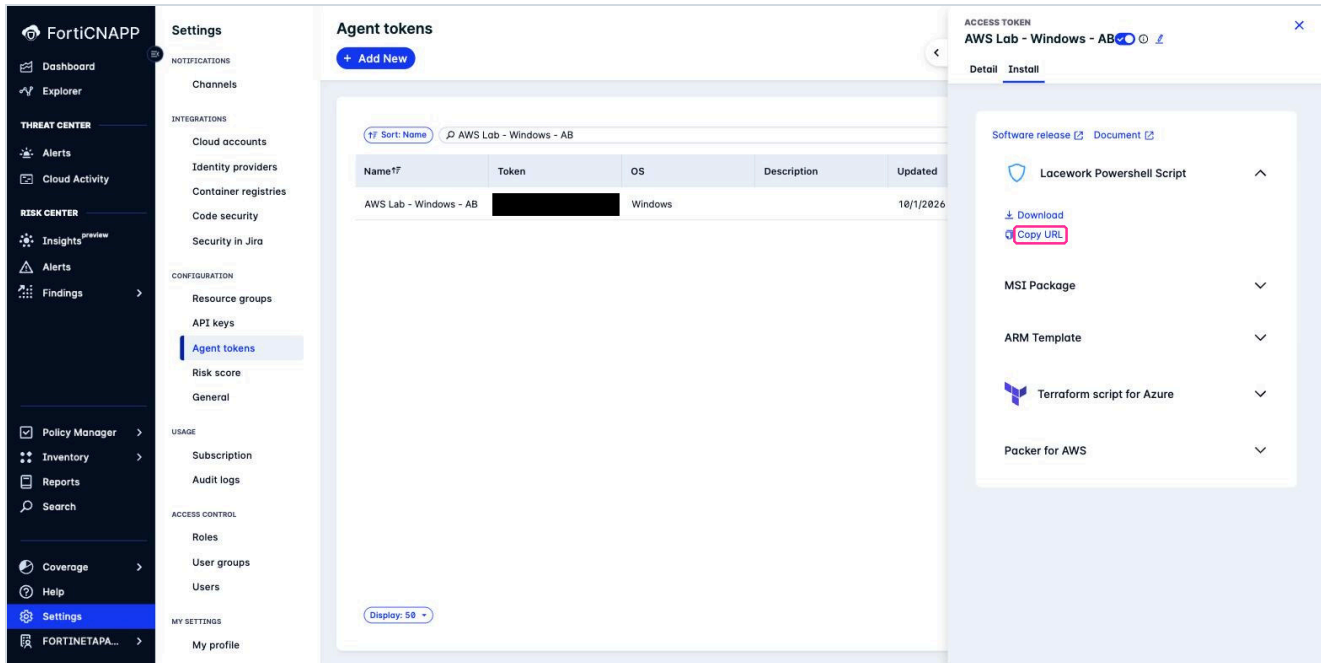
Name	Token	OS	Description	Updated	Status	Actions
AWS Lab - Windows - AB	[REDACTED]	Windows		10/1/2026 11:52 AM	✓	... - Copy - Install - Edit

Display: 50

1 - 1 of 1

Step 3: Collect the Three Install Values

The Windows panel offers different packages to the Linux one. You need three things from it:



Take this	From	Used as
Script URL	Install tab. Lacework Powershell Script , already expanded. Copy URL	what you download
MSI URL	Install tab. Expand MSI Package , then its Copy URL	-MSIURL
Access token	Detail tab, the Token field	-AccessToken

The package sections are an accordion. Expanding **MSI Package** collapses **Lacework Powershell Script**. Take the script URL first, paste it somewhere, then come back for the MSI one.

Paste all three into a text file on your own machine, each one labelled.

Checkpoint: three values, each one labelled so you know which is which.

Step 4: Build the Install Command

The install runs in two parts. The first downloads and unpacks the script bundle:

```
Invoke-WebRequest -Uri "<script-url>" -OutFile "install.zip"
Expand-Archive -Path "install.zip" -DestinationPath "install" -Force
cd install\signed-scripts
```

The second runs the installer with your token and the MSI URL:

```
.\Install-LWDataCollector.ps1 `
  -AccessToken "<access-token>" `
  -ServerURL "https://partner-demo.lacework.net" `
  -MSIURL "<msi-url>"
```

Copy both blocks into your text file and replace each placeholder with its value from Step 3.

These URLs are easily swapped. The script URL ends in `.zip`, the MSI URL in `.msi`. Give the installer them the wrong way round and it fails on a download error rather than telling you they are reversed.

The script pulls the MSI, installs it, and registers the agent against your token. Nothing to configure afterwards.

Checkpoint: both blocks are in your text file, with no angle brackets left.

Part 2 (Optional): Install on a Windows Instance

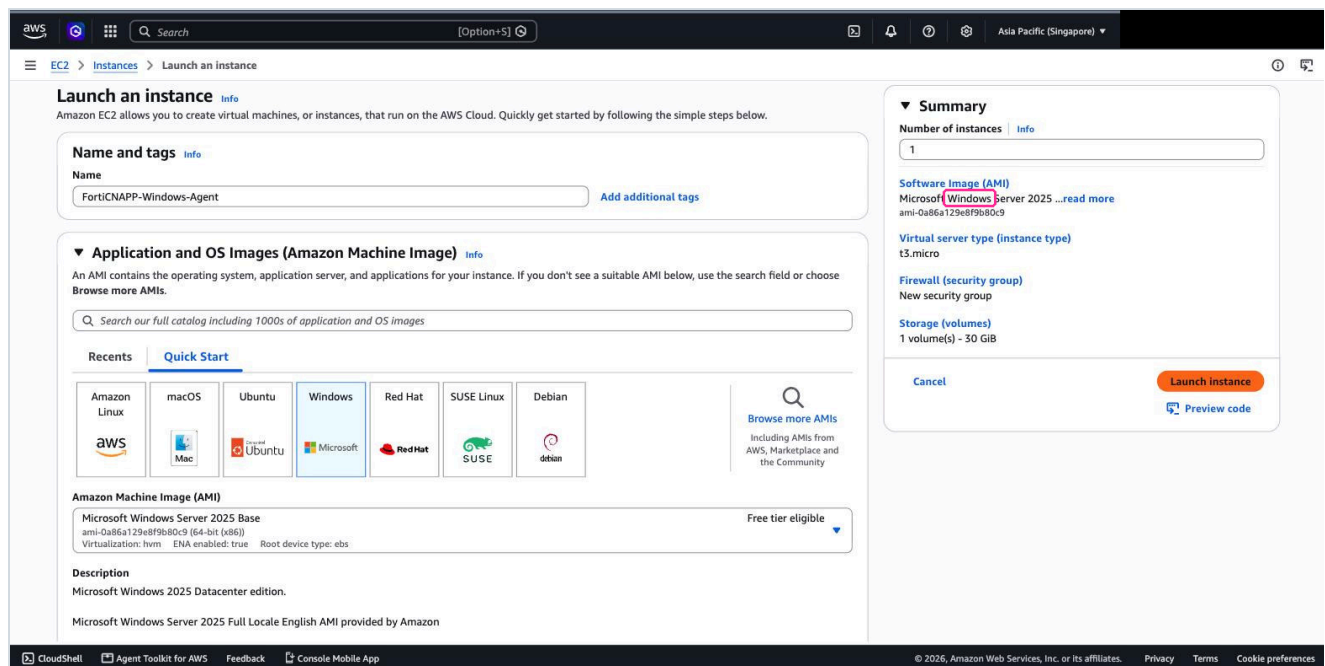
Part 2 launches a Windows instance and runs your command on it over RDP. Windows boots slowly, and RDP is slow on shared WiFi.

To skip it, read the steps below so you know how the install goes, especially [Look at what it is logging](#). Then go to [Lab 7](#).

Step 1: Create Windows EC2 Instance

Same wizard as Lab 5. What differs: the image, and the key pair.

1. Open **EC2** and click **Launch instance**. Check the region still reads **Asia Pacific (Singapore)**.
2. **Name:** FortiCNAPP-Windows-Agent
3. **Application and OS Images:** click the **Windows** tile. The AMI becomes **Microsoft Windows Server 2025 Base**. Storage jumps to 30 GiB on its own.



4. **Instance type:** leave **t3.micro**.
5. **Key pair (login):** this time you do need one.

Windows is different from Lab 5. There is no Instance Connect for Windows. AWS encrypts the Administrator password with your public key, so without the private key you cannot log in at all, and there is no way to recover it later.

- Click **Create new key pair**
- Name it `forticnapp-windows-key-<your initials>`, type **RSA**, format **.pem**
- Click **Create key pair**. The `.pem` file downloads. Keep it, you need it in Step 2.

Important

Put your initials in the name. Key pair names are unique per region, so if anyone has run this workshop in this account before, a plain `forticnapp-windows-key` fails with `InvalidKeyPair.Duplicate`. You cannot reuse the old one either, because its private key was only ever downloadable once.

The screenshot shows the AWS Management Console 'Launch an instance' wizard. The 'Key pair (login)' section is highlighted with a red box around the 'Key pair name' field, which is labeled '- required'. The 'Summary' section on the right shows the configuration: 1 instance, Microsoft Windows Server 2025 Full Locale English AMI, t3.micro instance type, and 1 volume (30 GiB). The 'Launch instance' button is visible.

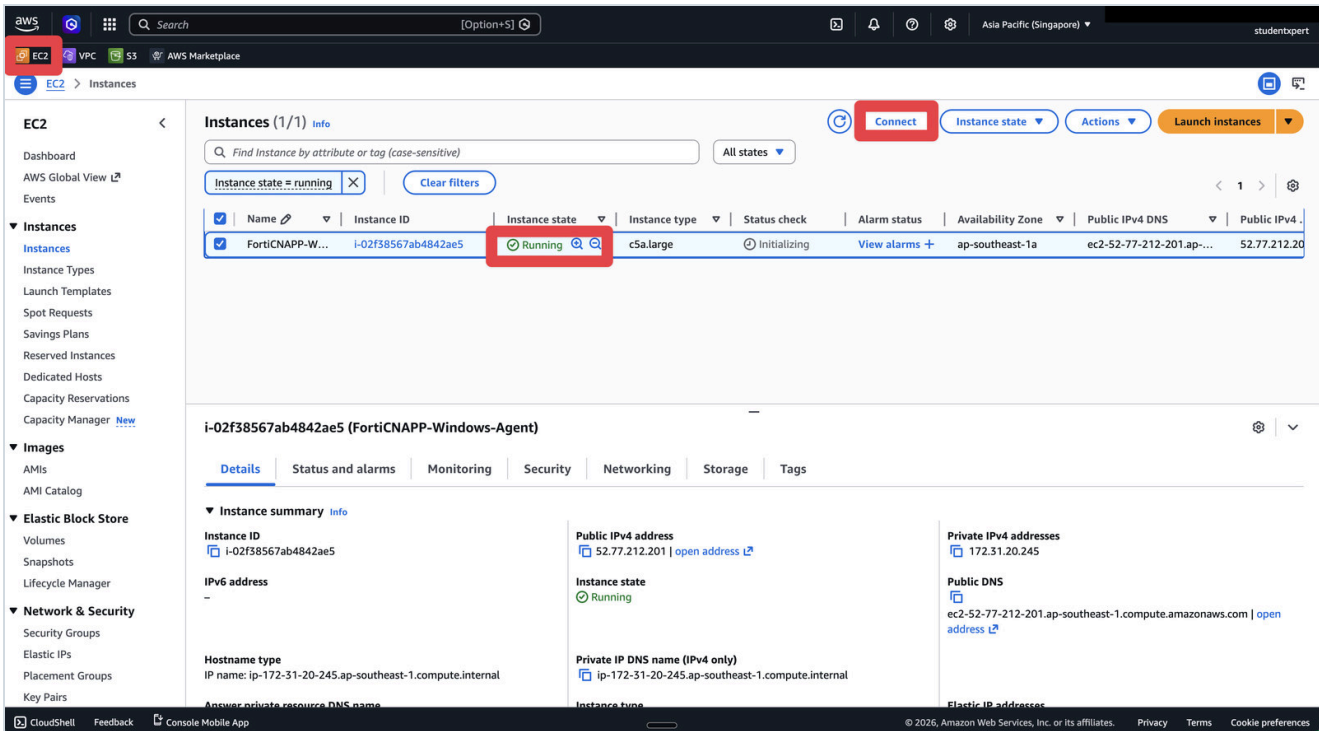
6. **Network settings:** leave them alone. The wizard creates a `launch-wizard-N` group allowing RDP from anywhere.

Warning

As in Lab 5, do not switch to **Select existing security group** and pick `default`. Your RDP client will not reach the instance.

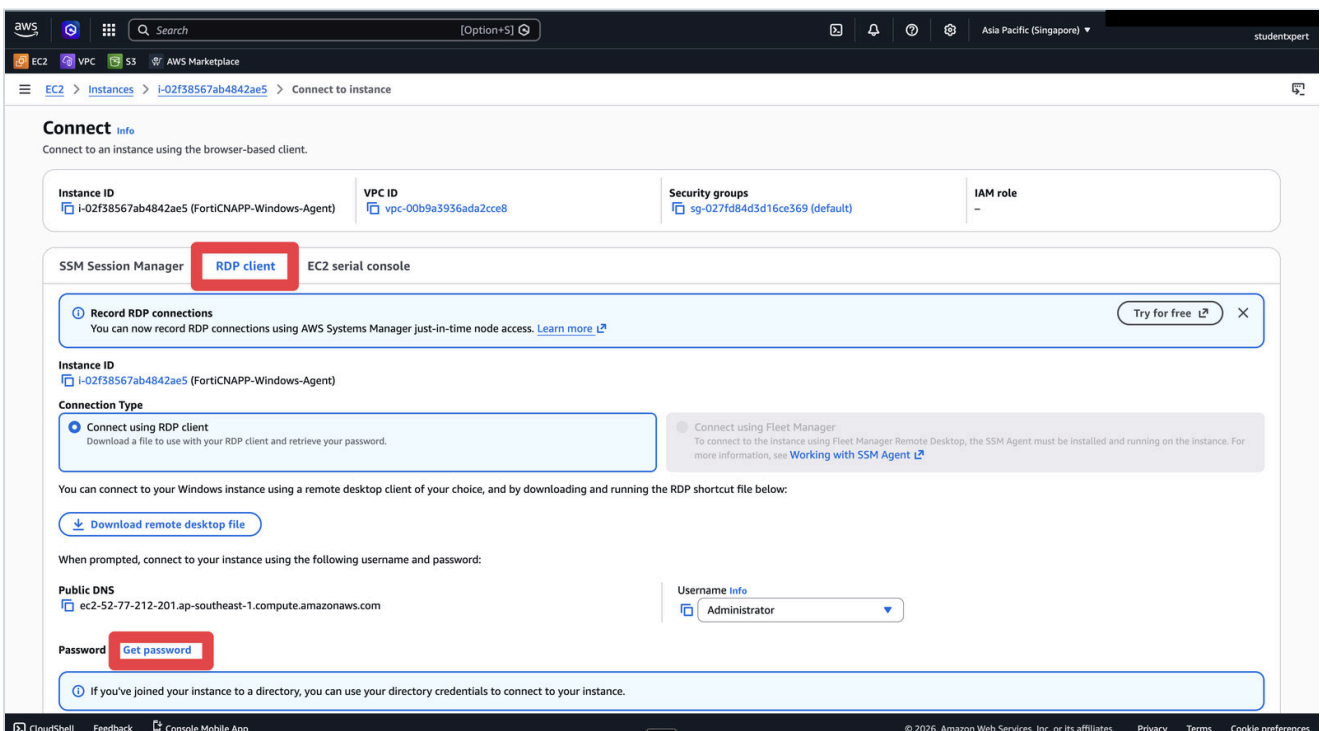
7. **Configure storage:** leave the default, 30 GiB gp3.

8. Click **Launch instance**, then wait for **Instance state** to read **Running**. Windows takes longer to boot than Linux. Allow about four minutes before you try to fetch the password.



Step 2: Get the Password and Connect

1. Go to **EC2 > Instances**, tick your Windows instance, and click **Connect**.
2. Choose the **RDP client** tab.



3. Click **Get password**.
4. Upload the `.pem` file you downloaded in Step 1. AWS uses it to decrypt the password and shows it in the clear.
5. Copy the **Administrator password**.
6. Click **Download remote desktop file**.

7. Open that file:

- **Windows:** double-click it
- **Mac:** right-click and open with **Windows App**

If it does not connect, turn off FortiSASE and try again. FortiSASE blocks RDP by default. So do most corporate VPN and secure-access agents. This is the commonest reason this step fails, and it has nothing to do with AWS or the instance.

8. Username is `Administrator` , capital A. Paste the password.

9. A certificate warning is normal on an EC2 instance. Continue past it.

Checkpoint: you are looking at a Windows desktop in an RDP window.

If you cannot connect

Work down this list. The first two are far more common than anything else.

Symptom	Cause and fix
Times out, or "couldn't connect to the remote PC"	Turn off FortiSASE or your VPN first, as above. If that was not it, outbound TCP 3389 is blocked somewhere else. Confirm with <code>Test-NetConnection -ComputerName <public-ip> -Port 3389 : TcpTestSucceeded : False</code> is your answer. Conference WiFi, hotel WiFi and some ISPs block it too, and tethering to a phone is the quickest way round.
Password rejected	Windows may still be initialising. Wait and click Get password again. Check you used <code>Administrator</code> , capital A, and that the paste did not pick up a trailing space.
No password offered yet	The instance has not finished its first boot. Give it four minutes from Running .

10. Once you are in, open **PowerShell as Administrator**: right-click **Start**, then **Terminal (Admin)** or **Windows PowerShell (Admin)**.

Without **Administrator**, the installer fails partway and leaves no useful message.

11. Open **Notepad** on the Windows desktop and paste in your text file from Part 1. RDP copy and paste is unreliable, so move everything across in one go, not value by value.

Step 3: Install the Agent

In **PowerShell as Administrator**, run the two blocks you built in Part 1, Step 4. Run the download block first, then the installer.

Step 4: Verify on the Host

The agent runs as a Windows service, so ask Windows:

```
Get-Service -Name LWDataCollector
```

Status should read `Running` .

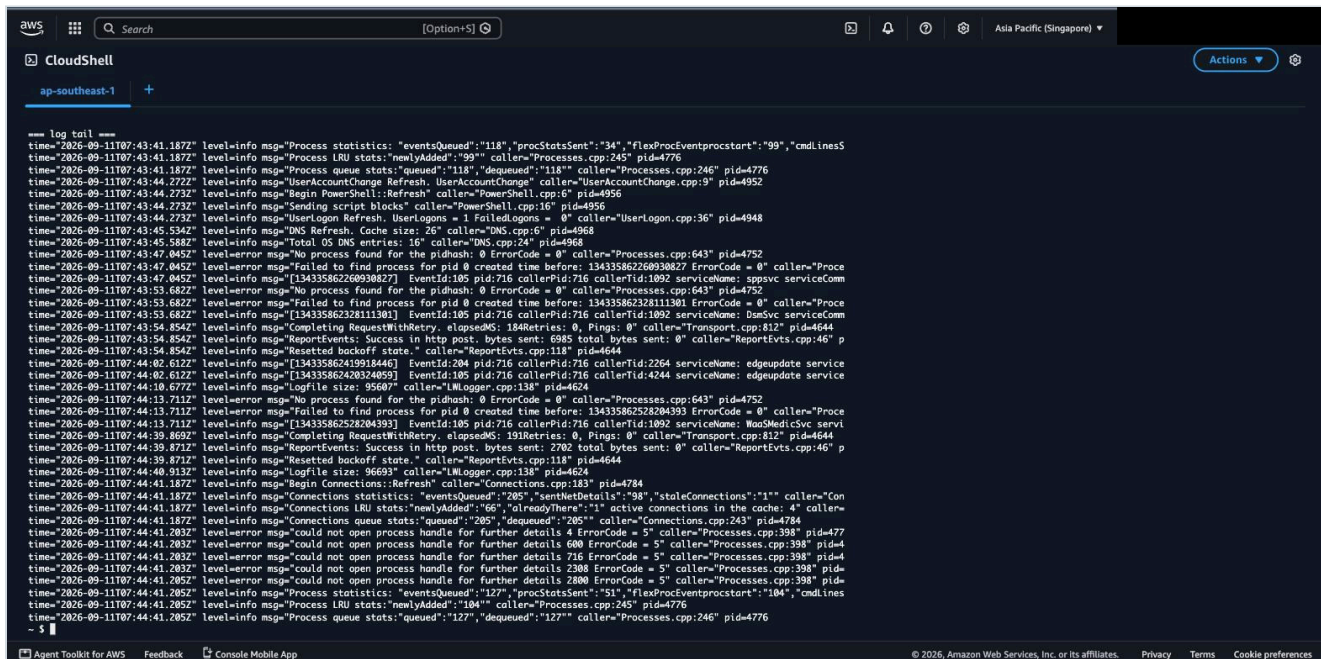
```
ls C:\ProgramData\Lacework\  
ls C:\ProgramData\Lacework\Logs\
```

Checkpoint: `LWDataCollector` is `Running` .

Look at what it is logging

```
Get-Content C:\ProgramData\Lacework\Logs\LWDataCollector_0.log -Tail 40 -Wait
```

Ctrl+C stops it.



This is worth more of your time than the Linux one. The Windows agent writes what it is observing, not just that it is connected:

Line	What it is watching
Process statistics: eventsQueued 118, procStatsSent 34	Processes starting and stopping
Connections statistics: eventsQueued 205, sentNetDetails 98	Network connections
UserLogon Refresh. UserLogons = 1 FailedLogons = 0	Who logged in, and who failed
DNS Refresh. Cache size: 26	Name lookups the host made
Begin PowerShell::Refresh / Sending script blocks	PowerShell being run
ReportEvents: Success in http post. bytes sent: 6985	All of it going to FortiCNAPP

Read that list again. Processes, connections, logons, DNS and PowerShell, from one host, continuously. **That is the polygraph from Lab 1, on a machine you built twenty minutes ago.** It is also why a composite alert can exist: no single line there is suspicious, and the combination can be.

Plenty of lines say `level=error` and are routine:

- No process found for the pidhash: 0 and Failed to find process for pid 0
- could not open process handle for further details ... ErrorCode = 5, which is Windows refusing access to protected system processes

Step 5: Verify in FortiCNAPP

The agent reports hourly, so it will not appear straight away. Carry on and come back.

In the console, go to **Inventory** and look for the host by its instance ID.

After [Lab 7](#) you can ask from CloudShell instead:

```
lacework agent list
```

Both your Linux and Windows hosts should be listed there by the end of the session.

What did we do here?

The same agent, reached a harder way.

You made the token that joins an agent to a tenant, and turned it into an install command. That FortiCNAPP side is where every Windows install starts.

A Windows rollout rarely stalls on the agent itself. It stalls on RDP being blocked, on a lost key pair, on an installer run without Administrator. If you built the instance, you have now hit those in a lab where it costs nothing. If you skipped it, those are the three to plan for.

From here FortiCNAPP treats both hosts identically. The next labs move off hosts entirely and look at the code that builds them.

Next: [Lab 7: Install the Lacework CLI](#).

Lab 7: Install the Lacework CLI

Objectives

Everything so far has been the console. The CLI is the same platform through a different door, and it is the door you use when you want to script something, check twenty accounts at once, or answer a question the console has no page for.

You will install it in **AWS CloudShell**, so there is nothing to install on your own machine and nothing to uninstall afterwards.

Labs 8 and 9 need this CLI, so this lab is not optional if you are continuing.

Prerequisites

- Completed [Lab 3](#), so there are integrations to list
- FortiCNAPP console access, tenant **FORTINETAPACDEMO**

Lab steps

Step 1: Open CloudShell

1. In the AWS console, check the region reads **Asia Pacific (Singapore)**.
2. Click the **CloudShell** icon in the top bar, the cloud with `>_` in it. It also sits at the bottom left of the console.
3. Wait for the prompt. First launch takes a minute.

CloudShell is a free Linux shell with your console credentials already loaded. Nothing to install, nothing to authenticate. It is wiped after long inactivity, apart from your home directory, which is why the next step installs into `$HOME/bin`.

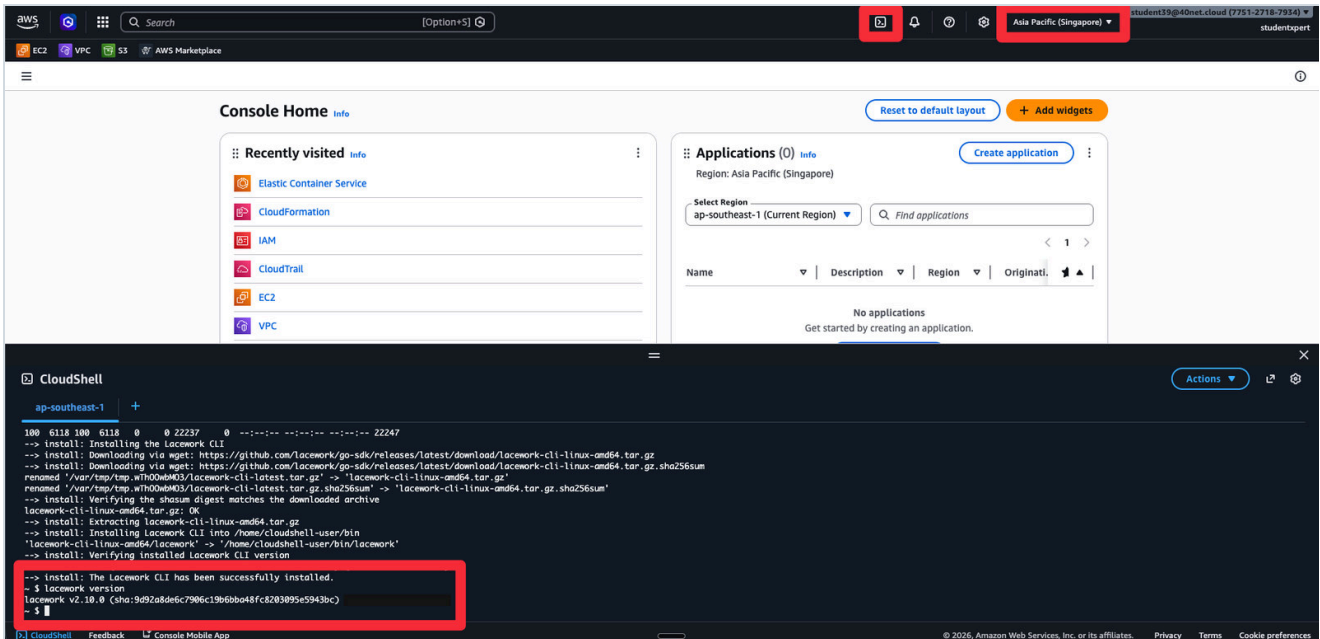
Step 2: Install the CLI

Make a `bin` directory in your home folder and put it on your PATH, so the CLI survives a CloudShell restart:

```
mkdir -p "$HOME/bin"
echo 'export PATH=$HOME/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

Then install:

```
curl https://raw.githubusercontent.com/lacework/go-sdk/main/cli/install.sh | bash -s -- -d
"$HOME/bin"
lacework version
```

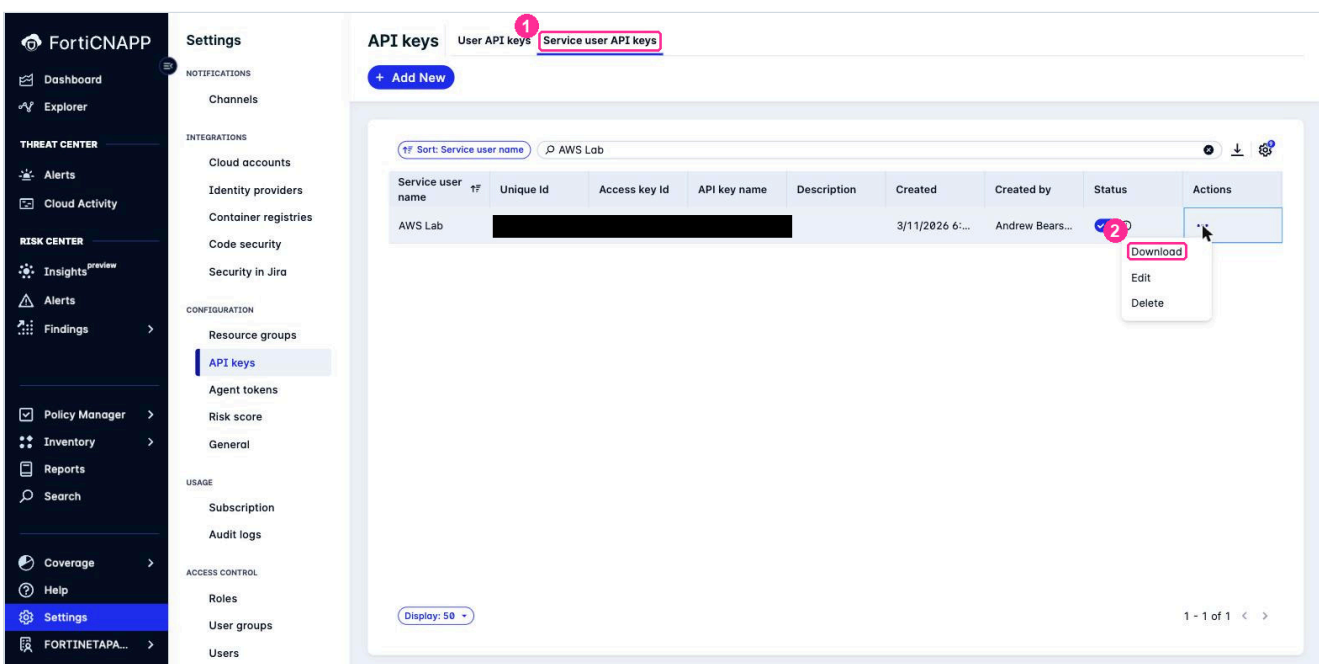


Checkpoint: lacework version prints a version instead of command not found .

Step 3: Download the API Key

An existing service user **AWS Lab** has been pre-configured with the necessary permissions. Download the API key for this user:

1. Log into FortiCNAPP console at <https://partner-demo.lacework.net/>
2. Ensure tenant is set to **FORTINETAPACDEMO**
3. Navigate to **Settings > Configuration > API keys**
4. Click the **Service user API keys** tab. The **User API keys** tab next to it is a different thing. Its keys will not work here.
5. Type **AWS Lab** in the search box
6. Click the ellipsis (three dots) on that row, then **Download**. The key arrives as JSON.



7. Open the JSON file. You need four values from it.

Caution

This file is a live credential. It can read your tenant. Do not paste it into chat, a shared document, or a ticket, and delete it when the workshop ends. Lab 10 revokes the key itself.

Step 4: Point the CLI at FortiCNAPP

```
lacework configure
```

It asks four questions. The answers all come from that JSON file, which looks like this:

```
{
  "keyId": "FORTINET_XXXXXXXXXXXXXXXX",
  "secret": "_XXXXXXXXXXXXXXXXXXXXXXXXX",
  "account": "partner-demo.lacework.net",
  "subAccount": "fortinetapacdemo"
}
```

It asks for	Use the JSON field
Account	account
API Key	keyId
API Secret	secret
Sub-Account	subAccount

The **sub-account** is the tenant, `fortinetapacdemo`. Leave it blank and the CLI talks to the wrong place and shows you nothing.

Step 5: List Your Integrations

```
lacework version
lacework cloud-account list
```

The integration types have internal names, which is the first thing the CLI shows you that the console hides:

CLI name	What you selected in Lab 3
AwsCfg	Configuration
AwsCtSqs	CloudTrail
AwsSidekick	Agentless Workload Scanning

`AwsCtSqs` appears only if CloudTrail deployed. Accounts inside an AWS Organization that already has an organization trail cannot deploy it. See the troubleshooting section in [Lab 3](#).

Checkpoint: your AWS account number appears, with at least `AwsCfg` and `AwsSidekick`.

Step 6: Trigger an Inventory Scan

FortiCNAPP collects resource inventory on its own cycle, up to 24 hours. The CLI can ask for one now.

Warning

Your instructor runs this one, once. Do not all run it.

The scan command behaves unlike anything else in the lab:

- it is **tenant-wide**. There is no per-account form of it, so one person's scan covers every account integrated into this tenant, yours included
- **only one scan runs at a time**. While one is going, every other request is silently ignored. No error, no queue position, nothing
- it takes **one to two hours**

In a room this size that means one scan happens and everyone benefits. Forty of you typing it changes nothing.

This is the command your instructor runs:

```
lacework compliance aws scan
```

So treat this as something you have now **seen**. You will use it on a tenant of your own, where you are the only one driving. Do not expect fresh compliance data to land before the session ends.

Checkpoint: you understand why this is the one command in the workshop you should not all run at once.

What did we do here?

We swapped the console for a terminal, against the same platform and the same data.

That matters more than it sounds. Everything you have done by clicking is an API call, and once you can make those calls yourself you can check twenty accounts as easily as one, put onboarding into a pipeline, and answer questions that have no console page. Labs 8 and 9 take that straight into source code.

The scan command is also a fair warning about shared tenants. Some operations are tenant-wide and serialised. They do not tell you when they are ignoring you.

Next: [Lab 8: Code Security for Infrastructure as Code \(IaC\)](#).

Lab 8: Code Security for Infrastructure as Code (IaC)

Objectives

Labs 3 to 6 found problems in a **running** account. This lab finds them in the Terraform that would have created it, before anything exists.

Shift left means moving the finding earlier, not finding more. A public S3 bucket in production is an incident with a clock on it. The same bucket found in a pull request is a two-line change nobody outside the team hears about.

The scanner reads Terraform, CloudFormation and the other IaC formats, and checks them against the same policies FortiCNAPP uses on live resources.

Prerequisites

- Completed [Lab 7](#), with the CLI working in CloudShell

Lab steps

You should still be in CloudShell from Lab 7. If it timed out, reopen it and run `source ~/.bashrc` to put the CLI back on your PATH.

Step 1: Install the IaC Scanner

The CLI ships small and pulls in what it needs:

```
lacework component install iac
```

Already installed from a previous run? Update it instead:

```
lacework component update iac
```

Checkpoint: the command finishes without an error.

Step 2: Get Some Code to Scan

This repository is deliberately full of bad Terraform:

```
cd ~
git clone https://github.com/andrewbearsley/lacework-iac-scan-example.git
cd lacework-iac-scan-example/example-terraform
```

Step 3: Scan It

```
lacework iac scan
```

It reads every Terraform file below the current directory, checks each against policy, and prints what failed. Uploading to the platform is on by default, controlled by `--upload`.

The command exits non-zero and that is the point. It ends with `violations exceeded a configured exit condition threshold` and an exit code in the hundreds. Nothing went wrong. That exit code is what fails a pipeline build when the code breaks policy.

Step 4: Read the Output

You will get a lot of findings. Do not try to read them all. Work through these three questions instead:

1. **How many are Critical or High?** That is the number anyone reacts to first.
2. **What kinds of problem are they?** Group them in your head: encryption off, access too open, logging missing. Most estates repeat the same few mistakes.
3. **Which file is the worst?** One module usually accounts for a large share, which is where you would start.

Each finding gives you a policy ID, a severity, and a **file and line number**. That last part is what makes this useful: it points a developer at the exact line, in their own editor, in their own language.

Checkpoint: you can name the single most common category of finding in this repo, and the file that carries the most.

Why your scan does not show up in the console. Look under **Risk Center > Findings > Code Security > Infrastructure (IaC)** and you will not find it. That is expected. A CLI scan from CloudShell has no repository behind it.

The **Assessments** view fills up when a repository is onboarded through **Code Security > Add integration** (GitHub, GitLab or Bitbucket), or when the scan runs inside a registered CI/CD pipeline. That is how it runs for real. This lab is the scanner on its own, so you can see what it does.

What did we do here?

We ran the platform's policies against code instead of against cloud.

Same checks, different moment. The scanner found 77 issues in four Terraform files that would have deployed perfectly happily. It named the file and the line for every one.

Wired into a pipeline, these findings arrive as comments on a pull request, before the plan is ever applied. The fix costs a developer five minutes instead of costing you an incident review.

Additional resources

- [Lacework IaC Scanning Documentation](#)
- [Example Repository](#)

Next: [Lab 9: Code Security for Applications \(SCA\)](#).

Lab 9: Code Security for Applications (SCA)

Objectives

Lab 8 scanned the code that builds your infrastructure. This lab scans the application that runs on it. Different question: **what did you inherit?**

Modern applications are mostly other people's code. A team writes a few thousand lines and imports a few hundred thousand. Software Composition Analysis reads what you pulled in and tells you which of it is vulnerable, which of it carries a licence you cannot live with, and whether anyone has committed a credential by accident.

You will also produce an **SBOM**, a parts list for your application. When the next Log4j-scale vulnerability lands, the only question anyone asks is "are we affected?" An SBOM turns that from a week of archaeology into a search.

Prerequisites

- Completed [Lab 7](#), with the CLI working in CloudShell

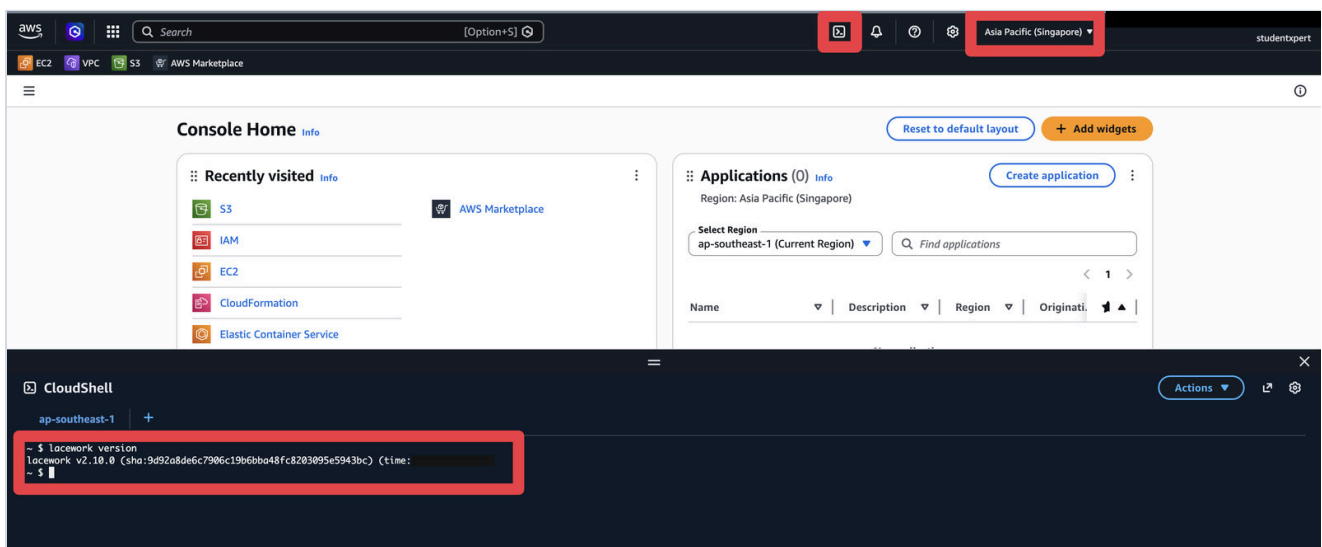
Lab steps

Still in CloudShell from Lab 8. If it timed out, reopen it and run `source ~/.bashrc`.

Step 1: Install the SCA Scanner

```
lacework component install sca
```

Already installed? `lacework component update sca` instead.



Step 2: Get Some Code to Scan

A JavaScript and TypeScript application, deliberately full of problems:

```
cd ~
git clone https://github.com/andrewbearsley/lacework-sca-scan-example.git
cd lacework-sca-scan-example
```

Step 3: Scan It

```
lacework sca scan .
```

SCA does not upload by default. IaC does. If you want the results sent to the platform, ask for it:

```
lacework sca scan . --save-results=true
```

That needs git metadata, which the cloned repo has.

As in Lab 8, results still will not appear under **Risk Center > Findings > Code Security > Applications** until the repository is onboarded through **Code Security > Add integration**, or the scan runs in a registered CI/CD pipeline.

Step 4: Read the Output

The findings come back in kinds worth telling apart, because different people fix them:

Finding	What it means	Who fixes it
Vulnerabilities	CVEs in packages you imported, direct or pulled in by something else	Usually a version bump
Weaknesses	CWEs in code that was written here: SQL injection, hard-coded credentials, weak auth	A developer, properly
Secrets	A credential committed to the repo	Rotate it first, then remove it
Licences	A dependency whose terms may not suit a commercial product	Legal, not engineering

Work through the same three questions as Lab 8:

1. How many Critical and High vulnerabilities?
2. Are they in packages you chose, or in packages **those** packages chose? Transitive dependencies are the ones teams are surprised by.
3. Were any secrets found? That is the finding to act on today.

The scan reports **19** third-party vulnerabilities and **12** critical secrets, the secrets sitting in `data/users.yml` and `index.js`. Two of the secrets are recognised as AWS keys.

Checkpoint: you can say whether the worst vulnerability is in a direct or a transitive dependency. Both kinds are present, so the answer is worth working out rather than guessing.

Step 5: Generate an SBOM

```
lacework sca scan ./ -f cdx-json -o sbom.json
```

That writes CycloneDX JSON, one of the two formats regulators and auditors ask for.

Important

The repository already ships an `sbom.json` at its root, and this command overwrites it. That is fine here. It does mean that if the scan fails you can still read a file called `sbom.json` and think it worked, so check the timestamp inside it.

Count what is in there, and what the application actually asked for:

```
jq '.components | length' sbom.json
jq '.dependencies | length' swiss-cheese-app/package.json
```

The application declares **2** dependencies, `express` and `lodash`. The SBOM lists **53** components.

Checkpoint: the component count is far larger than the number of packages the application directly imports. That gap is the point of the exercise.

What did we do here?

We inventoried what the application actually contains, rather than what its authors wrote.

The vulnerability count matters less than where it comes from: most of the risk arrives through dependencies nobody chose deliberately. That is why an SBOM is worth generating before you need it. The day a Log4j-scale CVE is announced, the teams that can answer "are we affected" in minutes are the ones that already had the parts list.

Additional resources

- [Lacework SCA Scanning Documentation](#)
- [Example Repository](#)

Next: [Lab 10: Clean Up Workshop Resources](#).

Lab 10: Clean Up Workshop Resources

Objectives

Leaving workshop resources running in AWS costs money. In this lab, we'll remove everything the workshop created: the integrations from Labs 3 and 4, the resources they built in AWS, the EC2 instances from Labs 5 and 6, and your copies of the shared API key from Lab 7.

How you remove something depends on how it was made. That is the real lesson here, and it is why this lab has three routes rather than one.

What made it	Route	Removes both sides?
Lab 4, CloudFormation	A. Delete the stack	Yes. The stack owns the integration record as well as the AWS resources
Lab 3, the wizard	B. Console and tag search	No. Two separate jobs, done by hand
Lab 3, the wizard, if you did Lab 11	C. Terraform destroy	Yes. One command per integration

Start with **Route A**, which clears Lab 4 in a single step. Then use **Route B** for what Lab 3 built, or **Route C** if you have Terraform installed.

Important

Do this before Lab 12. One AWS account carries one integration of each type per tenant, so Lab 12 cannot deploy CloudTrail as code while Lab 4's CloudTrail is still registered.

Prerequisites

- Access to the AWS Console
- FortiCNAPP console access, tenant **FORTINETAPACDEMO**

First, find what was created

Whichever route you take, start here.

1. Go to **Settings > Integrations > Cloud accounts**.
2. Select the **Deployment History** tab.
3. Open the deployment you created in Lab 3.
4. Expand each integration to see its **Resources** table.

The screenshot shows the FortiCNAPP Settings page. On the left is a navigation menu with options like Dashboard, Explorer, Threat Center, Risk Center, Policy Manager, Inventory, Reports, Search, Coverage, Help, and Settings. The main content area is titled 'Settings' and shows the 'Cloud accounts' integration. It displays the Account ID, Caller Identity, and a list of resources. The resources are tagged with lacework_tag and lacework_integration.

Resource ID	Resource Name	Resource Type
arn:aws:iam::[redacted]:policy/lwaudit-policy-8f2fd5d2	lacework_audit_policy	aws_iam_policy
arn:aws:iam::[redacted]:policy/lwaudit-policy-8f2fd5d2...	lacework_audit_policy_2025_1	aws_iam_policy
FORTINET_7F2EF93753408DABBS16674B254CDB947092A...	default	lacework_integration_aws_cfg
arn:aws:iam::[redacted]:role/lw-iam-9b64622b	lacework_iam_role	aws_iam_role

Each integration lists every resource by ARN, name and type, and states the exact tags applied, for example:

All cloud resources are tagged with `lacework_tag: self-deployment` and `lacework_integration: aws_config`

Warning

Read the tag values from this screen, and search on the tag key. The values are not stable. The administration guide documents `lacework_tag: self-deployment` and `lacework_integration: configuration`. A 2026 deployment record shows `lacework_integration: aws_config`. Resources deployed in March 2025 carry `lacework_tag: lacework-self-deploy`. All of these are real, seen in the same tenant. The console and the AWS tag can even disagree on the same deployment: one September 2026 run showed `aws_agentless` on the deployment record and `agentless` on the resources.

Search on the key `lacework_tag` with any value. Filter on a value and you will miss resources and leave them running.

Keep this page open. It is your checklist either way.

Route A: Delete the CloudFormation Stack

This removes everything Lab 4 created, on both sides, in one operation.

1. In the AWS Console, go to **CloudFormation > Stacks**.
2. **Check the region selector** matches where you created the stack, `ap-southeast-1`. A stack is only visible in its own region.
3. Select **forticnapp-cloudtrail**.
4. Click **Delete stack**.
5. Type the stack name to confirm, then click **Delete stack**.

Deletion takes a minute or two. Wait for the stack to leave the list.

Checkpoint: in FortiCNAPP, **Settings > Integrations > Cloud accounts** no longer lists a CloudTrail integration for this account. You did not delete it there. The stack deregistered it on the way out, through the same

callback that registered it in Lab 4.

In AWS, the trail is gone too:

```
aws cloudtrail describe-trails --region ap-southeast-1 \
  --query "trailList[].[Name,IsOrganizationTrail]" --output text
```

Only the organization trails remain, the ones you never owned.

This is not how CloudFormation always behaves with FortiCNAPP. Where an integration is created in the console first and CloudFormation only builds the AWS side, the stack does not own the record, and deleting the stack leaves the integration behind polling a bucket that no longer exists. Lab 4's template registers the integration itself, which is why it can also remove it. Check which pattern you have before assuming a stack delete is enough.

Route B: Console and tag search

No tooling required. Deleting the integration does not touch AWS, so you clean up both sides by hand.

Step 1: Delete the Integrations in FortiCNAPP

1. Go to **Settings > Integrations > Cloud accounts**.
2. Select the **Cloud Accounts** tab.
3. Find your AWS account ID in the list.
4. Delete each integration on that account: Configuration, CloudTrail and Agentless.

Important

Use Delete, not the toggle. The toggle beside each integration only disables it. A disabled integration stays on the shared tenant until an instructor finds it and removes it. After the Bangkok class, 6 disabled integrations were still there the next day.

Step 2: Delete the AWS Resources by Tag

1. Log into the AWS Console.
2. Select the region you used in Lab 3.
3. Go to **Resource Groups & Tag Editor > Tag Editor**.
4. Set **Regions** to your workshop region, or **All regions** to be thorough.
5. Set **Resource types** to **All supported resource types**.
6. Add this tag filter:
 - **Tag key:** lacework_tag
 - **Tag value:** leave empty, so the search matches every value
7. Click **Search resources**.

Work through the result and delete each resource, cross-checking the deployment record. Expect IAM roles and policies, S3 buckets, KMS keys, SNS topics, SQS queues, a CloudTrail trail, an ECS cluster and Lambda functions.

Agentless scanning also tags its networking `LWTAG_LACEWORK_AGENTLESS`, which the `lacework_tag` search does **not** return. Run a second search on that key to catch the VPC, subnet, route table, internet gateway and

security group in every scanned region.

Checkpoint: both tag searches come back empty, or return only the resources listed under "a tag search will still return a few resources" at the end of this lab.

Important

Order matters. Empty an S3 bucket before deleting it. Delete resources that depend on an IAM role before the role. A KMS key can only be scheduled for deletion, minimum seven days.

Route C: Terraform destroy

Better. The only route that removes both sides at once. The bundle FortiCNAPP gives you contains **full Terraform state**, including the FortiCNAPP integration itself, so `terraform destroy` deregisters the integration and deletes the AWS resources in one pass.

You need Terraform. CloudShell does not ship it, so complete [Lab 11](#) first, or install it now.

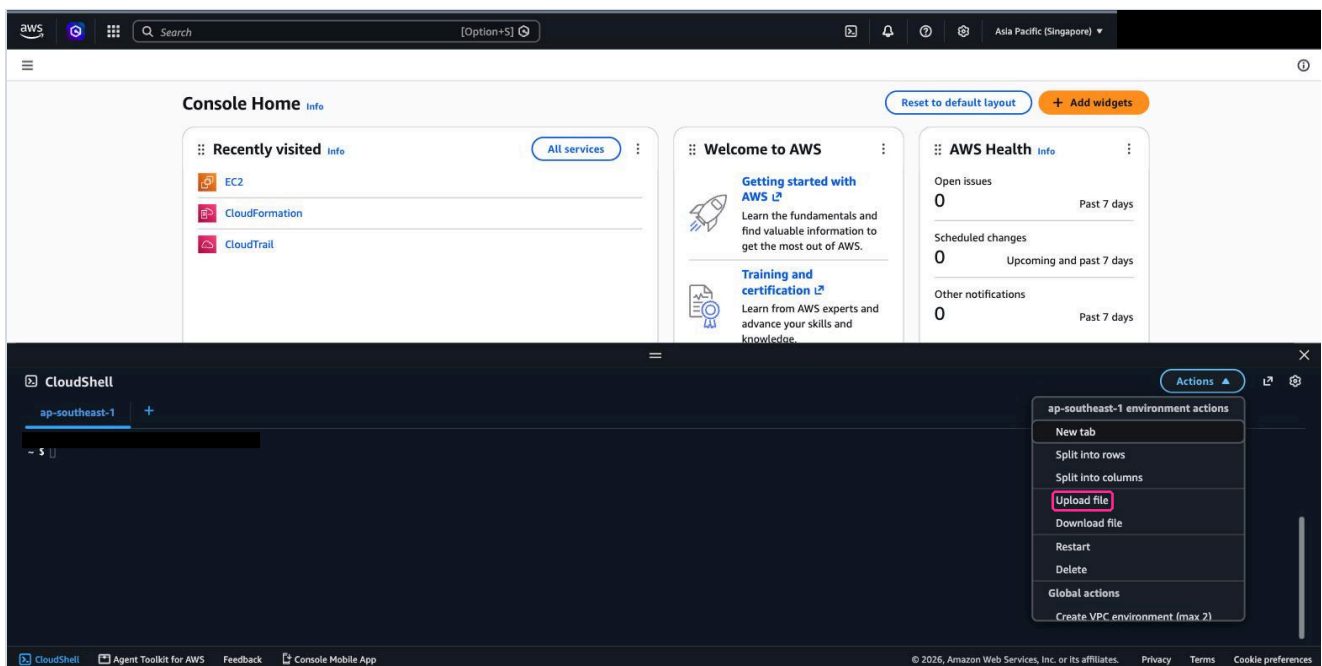
Step 1: Download the Terraform bundle

On the deployment record, click **Terraform files** next to an integration. Repeat for each integration; **the bundle is per integration**, so three integrations means three bundles.

You cannot `curl` this URL. It is authenticated by your browser session, not by an API token. Fetching it without a browser returns `401` and an HTML login page. Download it in the browser, then upload it to CloudShell.

Step 2: Upload it to CloudShell

1. Open CloudShell.
2. Choose **Actions** > **Upload file**.



3. Select the `tf-files.tar.gz` you just downloaded.

Step 3: Destroy

Get fresh credentials the same way you did in [Lab 2](#), then:

```
mkdir -p ct && tar -xzf tf-files.tar.gz -C ct && cd ct

terraform init

terraform plan -destroy \
  -var access_key="$AK" -var secret_key="$SK" -var token="$ST"

terraform destroy \
  -var access_key="$AK" -var secret_key="$SK" -var token="$ST"
```

Read the plan before you apply it. It should list the AWS resources **and** a `lacework_integration_*` resource. That last one is the integration record. It is why this route cannot leave an orphan.

Checkpoint: `terraform destroy` ends with `Destroy complete!` and a resource count.

Repeat for each bundle. A full three-integration teardown took about four minutes in testing, most of it the agentless VPC and ECS cluster. Terraform prints `Still destroying...` every ten seconds, so it is working, not stuck.

For reference, a full run on one account destroyed:

Bundle	Resources
Agentless	41
CloudTrail	29
Configuration	19

Remove the CLI credential

Both routes need this. [Lab 7](#) put a downloaded **API key** onto your CloudShell home directory. Nothing above removes it.

CloudShell keeps your home directory for 120 days, so the key outlives the workshop.

In CloudShell:

```
rm -f ~/.lacework.toml
rm -f ~/*-api-key.json          # the file you downloaded in Lab 7
rm -rf ~/bin/lacework ~/.config/lacework
```

Check the second line matches before you run it. `rm ~/.*.json` would take anything else you had in there.

Caution

Do not delete the AWS Lab service user or its key. You did not create it. Lab 7 had you download a key that already existed and is shared by everyone in the room. Deleting it revokes it for every other student at the same time.

Only delete a key in the console if **you** created it. If you are unsure, you did not.

The key is shared because **FortiCNAPP caps how many API keys a tenant can hold**. One key for the room is a deliberate choice, not an oversight, and it cannot be replaced with one key per student.

Removing the local copies above is the part that matters to you. The key is shared, so the risk you control is the copy sitting in your own CloudShell home directory, which survives for 120 days.

A leaked API key is worth more to an attacker than any resource in this lab. It reads the whole tenant. Treat deleting your local copies as mandatory, not optional tidying.

Terminate the EC2 Instances

Both routes need this. The agents in Labs 5 and 6 run on instances the integrations do not own.

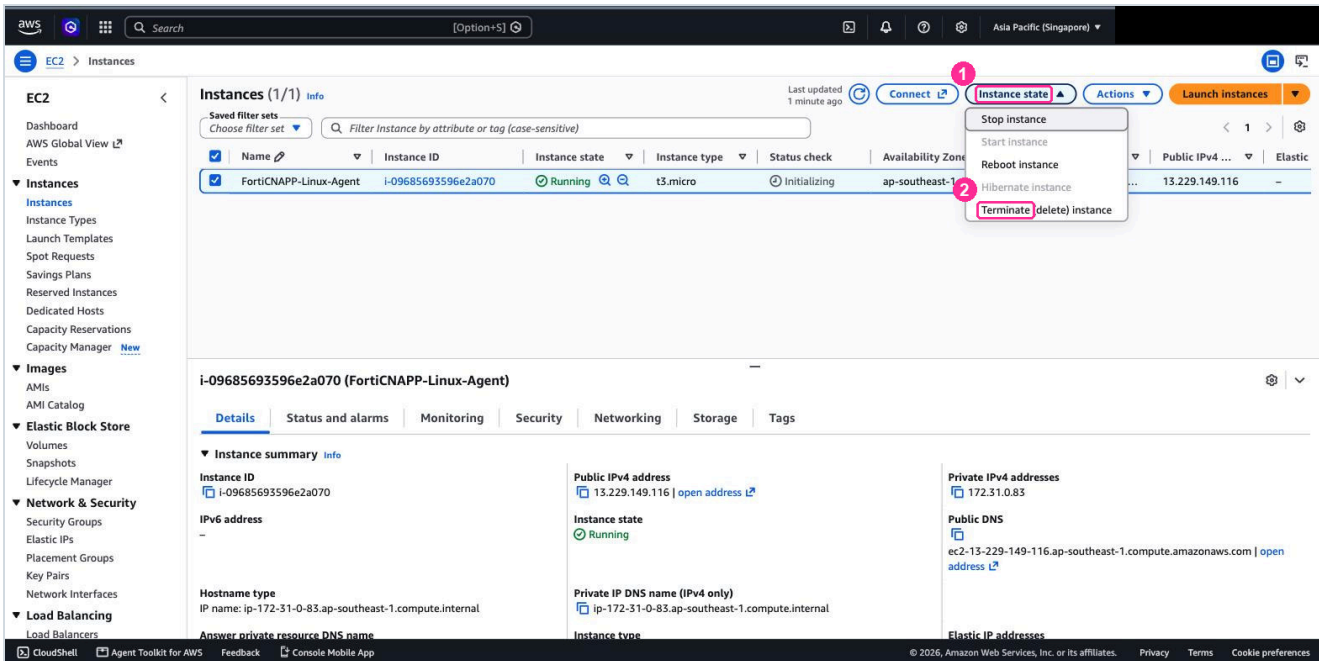
Warning

Check every region you have opened today, not just the workshop one. The console opens in whatever region you last used, and past runs of this workshop have left instances running in Ireland and the US because nobody changed the selector before launching. Those keep billing.

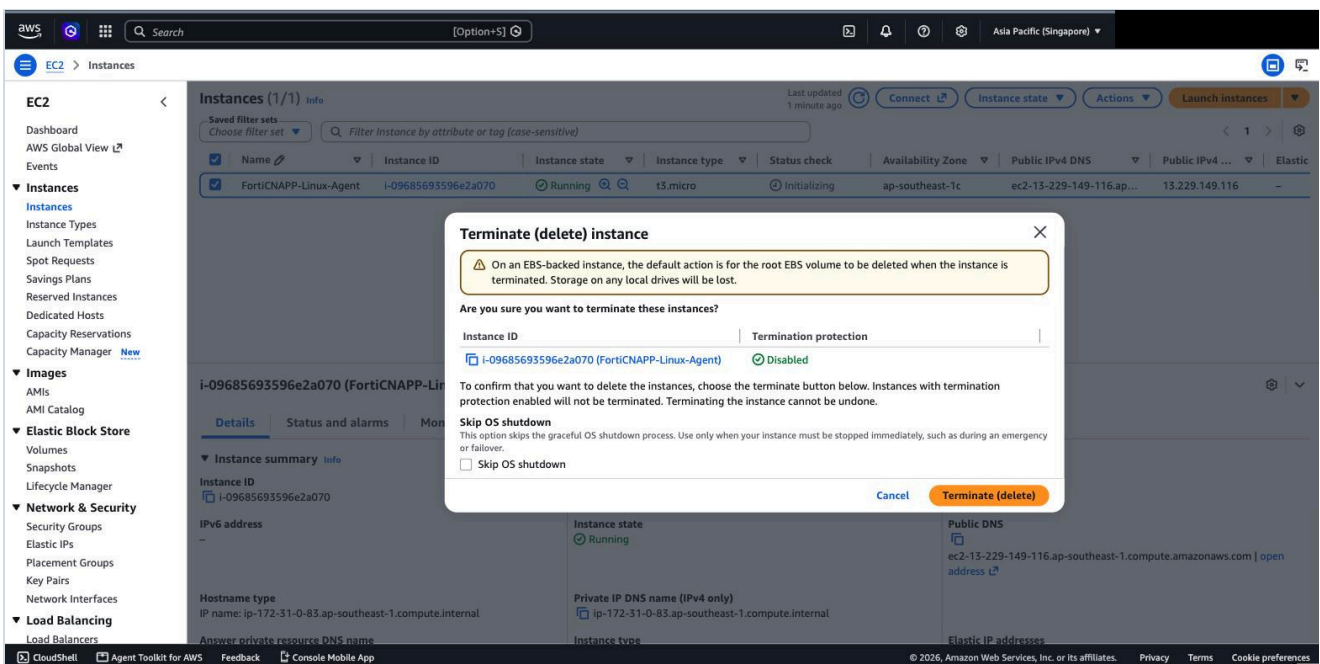
The quickest way to be sure, from CloudShell:

```
for r in $(aws ec2 describe-regions --query 'Regions[].RegionName' --output text); do
  ids=$(aws ec2 describe-instances --region "$r" \
    --filters Name=instance-state-name,Values=running,stopped \
    --query 'Reservations[].Instances[].InstanceId' --output text)
  [ -n "$ids" ] && echo "$r: $ids"
done
```

1. Go to the **EC2** service.
2. Find the instance from Lab 5, for example `FortiCNAPP-Linux-Agent`.
3. Tick its checkbox, then **Instance state > Terminate (delete) instance**.



4. Confirm in the dialog. The root EBS volume goes with the instance, so there is no separate volume to clean up afterwards.



5. If you built the Windows instance in Lab 6, repeat for it.

Verify

1. In FortiCNAPP, confirm your AWS account no longer appears under **Cloud accounts**, not even as disabled.
2. In CloudShell, confirm your copies of the Lab 7 API key are gone. This command must print nothing:

```
ls ~/.lacework.toml ~/*-api-key.json 2>/dev/null
```

The key itself stays in FortiCNAPP. Everyone in the room shares it. See [Remove the CLI credential](#).

3. Confirm your EC2 instances show **terminated**: the Linux one, and the Windows one if you built it.
4. In AWS, check that nothing is still running:

```
aws cloudtrail describe-trails --query "trailList[].Name" --output text
aws ecs list-clusters --query clusterArns --output text
aws events list-rules --query "Rules[?contains(Name, 'lacework')].Name" --output text
aws s3api list-buckets --query "Buckets[?contains(Name, 'lacework')].Name" --output text
```

All four should come back empty. The EventBridge one matters most, because that is the hourly trigger.

A tag search will still return a few resources, and that is normal

Re-run the Tag Editor search and you will still see a dozen or so entries, plus the IAM policies in the global index. Check their state before chasing them:

Resource	Expected state after cleanup
KMS key	PendingDeletion , scheduled 7 to 30 days out. AWS does not delete keys immediately.
Secrets Manager secret	Deleted, inside its recovery window
ECS cluster	INACTIVE
ECS task definition	INACTIVE . Deregistered task definitions stay in the account permanently.
IAM roles and policies	Still there. Nothing in this lab deletes IAM.
Security group, subnet, route table, VPC	Gone once the agentless stack is removed. The tag index lags behind by minutes.

```
aws kms describe-key --key-id <key-id> --query 'KeyMetadata.[KeyState,DeletionDate]' --
output text
aws ecs describe-clusters --clusters <name> --query 'clusters[0].status' --output text
aws events describe-rule --name <rule-name> --query 'State' --output text
```

Warning

Two of these tell you the cleanup did not finish. An ECS cluster reading **ACTIVE** , or an EventBridge rule reading **ENABLED** , means the agentless scanner is still scheduled. That is the one thing on this page worth acting on straight away. Everything else is inert.

Nothing else here is running or scanning. The one to watch on cost is the KMS key, which carries a small monthly charge until its deletion date passes.

Why this lab matters more than it looks

Orphaned agentless infrastructure keeps running.

The agentless scanner is driven by an **EventBridge rule on an hourly schedule**. Deleting the integration in FortiCNAPP does not delete that rule. It keeps firing, keeps starting an ECS task, and keeps costing money, while reporting to an integration that no longer exists.

We found exactly this in a real Fortinet demo account: an agentless deployment from March 2025 still running `rate(1 hour)` in September 2026, pointing at an integration GUID that had been deleted from the tenant.

Nothing in the FortiCNAPP console showed it, because there was no integration left to show.

Orphaned storage grows quietly too. The CloudTrail bucket left behind by that same deployment held **816,721 objects** by the time we emptied it, eighteen months of logs nobody was reading.

Check the EventBridge rule in every region you scanned:

```
aws events list-rules --region <region> \
  --query "Rules[?contains(Name, 'lacework')]. [Name,State,ScheduleExpression]" --output text
```

What did we do here?

We took it all down, both sides, and that turned out to be the harder half.

Deleting an integration in the console does not delete anything in AWS. It removes the record. The roles, buckets, queues and scheduled triggers carry on without it. We found an agentless deployment in a Fortinet account still firing every hour eighteen months after its integration was deleted, with over 800,000 objects in its CloudTrail bucket.

Each route answers that gap differently:

Route	Who owns what	Result
A, delete the stack	The CloudFormation stack owns the AWS resources and the integration record	One operation, nothing left behind
B, console and tag search	FortiCNAPP owns the record, AWS owns the resources, nothing spans both	Two jobs by hand, and the gap orphans fall through
C, terraform destroy	The state file spans both	One command per integration

Route B is where you can see the seam, which is why it is worth doing once even though it is the slowest. Routes A and C both work because something owns both sides at once.

When you onboard an account, agree the teardown at the same time. It is a much easier conversation before anything is deployed than after somebody finds the bill.

That is the core path finished.

Still curious? The optional track onboards the same account as code instead of through the wizard: [Lab 11: Install Terraform](#).

Lab 11: Install Terraform

Objectives

In Lab 4 you used CloudFormation to deploy an integration, which is one infrastructure as code approach. But many enterprises prefer Terraform since it's not AWS-specific and works across multiple cloud providers. In this lab, we'll install the two tools needed for the Terraform approach: the Lacework CLI (to generate Terraform configuration) and Terraform (to deploy it). This sets up everything for the next lab.

Prerequisites

- AWS account access
- FortiCNAPP account credentials

Note: You installed and configured the Lacework CLI in [Lab 7](#), so skip Steps 1-6 and go straight to Step 7 (Install Terraform).

Lab steps

Step 1: Log into AWS Console and Open CloudShell

1. Navigate to <https://aws.amazon.com/>
2. Click **Sign into console**
3. After logging in, change to your local region (e.g., **Asia Pacific (Singapore)**) using the region selector in the top right of the AWS Console
4. Click the **CloudShell** icon in the top navigation bar (cloud icon with `>_` symbol)
5. Wait for CloudShell to initialize (this may take a minute the first time)
6. Once CloudShell opens, you'll have a Linux-based terminal environment ready to use

Step 2: Set Up Installation Directory

Create a bin directory in your home folder and add it to your PATH:

```
mkdir -p "$HOME/bin"
echo 'export PATH=$HOME/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

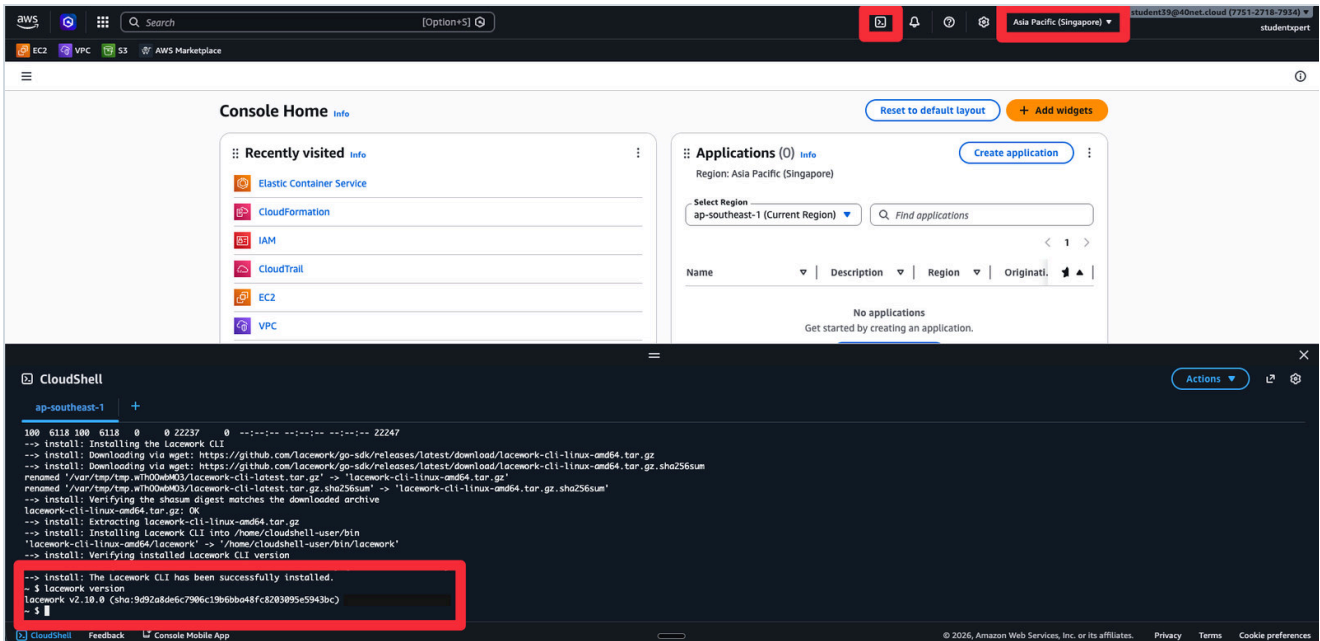
This directory will be used for both Lacework CLI and Terraform installations.

Step 3: Install Lacework CLI

In the CloudShell terminal, run:

```
curl https://raw.githubusercontent.com/lacework/go-sdk/main/cli/install.sh | bash -s -- -d "$HOME/bin"
```

This will download and install the Lacework CLI tool to your home bin directory.

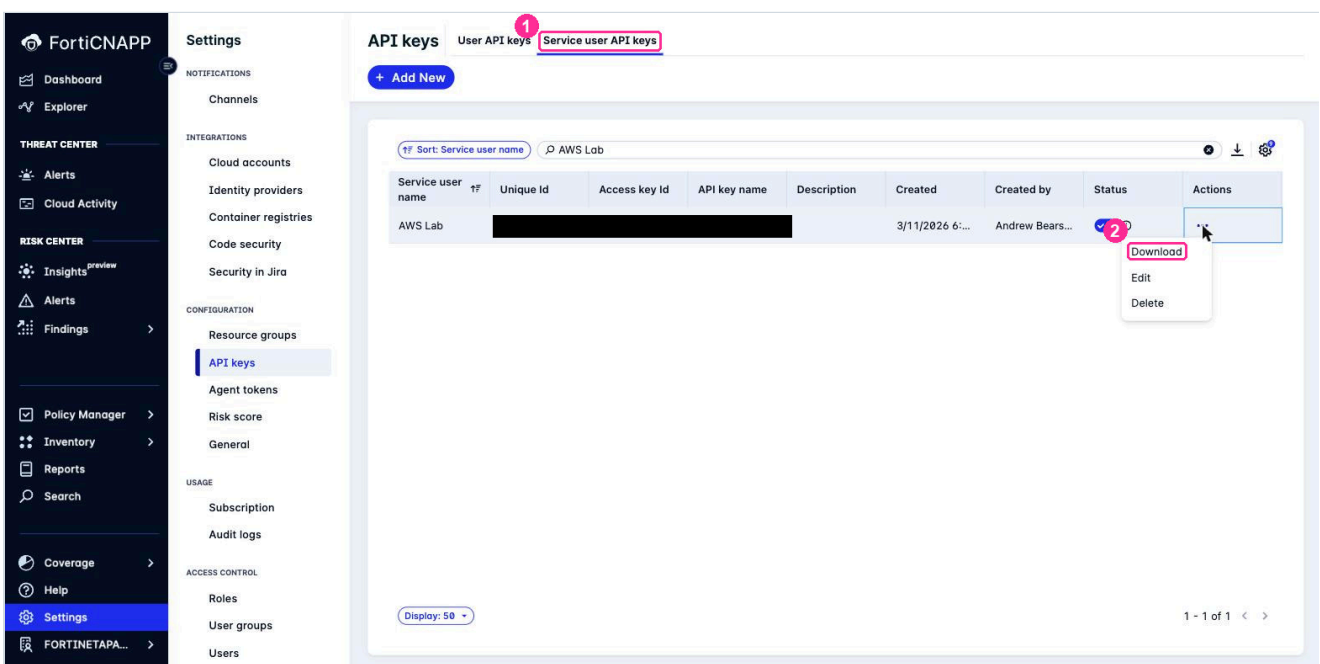


Note: For Windows installation (if needed outside CloudShell), download from the [Lacework CLI releases page](https://github.com/lacework/go-sdk/releases/latest/download/lacework-cli-linux-amd64.tar.gz).

Step 4: Download API Key from FortiCNAPP

An existing service user **AWS Lab** has been pre-configured with the necessary permissions. Download the API key for this user:

1. Log into FortiCNAPP console at <https://partner-demo.lacework.net/>
2. Ensure tenant is set to **FORTINETAPACDEMO**
3. Navigate to **Settings > Configuration > API keys**
4. Click on the **Service user API keys** tab
5. Find the API key for the **AWS Lab** service user
6. Click on the ellipsis (three dots) next to the API key and select **Download** to download the key as JSON



7. Open the downloaded JSON file and note the values. Keep these credentials ready for the next step

Step 5: Configure CLI

In CloudShell, run:

```
lacework configure
```

Enter your FortiCNAPP account credentials when prompted. These values are taken from the JSON file downloaded in the previous step:

```
{
  "keyId": "FORTINET_5DFDAF3B...",
  "secret": "_f1b528...",
  "account": "partner-demo.lacework.net",
  "subAccount": "fortinetapacdemo"
}
```

- **Account:** partner-demo.lacework.net (the account value from the JSON file)
- **API Key:** The keyId value from the JSON file
- **API Secret:** The secret value from the JSON file
- **Sub-Account** (if prompted): The subAccount value from the JSON file

Step 6: Verify CLI Installation

```
lacework version
```

This should return the installed version, confirming the CLI is ready to use.

Note: If you completed [Lab 10](#), the integrations from Labs 3 and 4 have been cleaned up. We'll redeploy CloudTrail using Terraform in the next lab.

Step 7: Install Terraform

Install Terraform in AWS CloudShell:

1. Download Terraform (Linux amd64) and unzip it to your bin folder:

First, get the latest Terraform version:

```
TERRAFORM_VERSION=$(curl -s https://api.github.com/repos/hashicorp/terraform/releases/latest | grep '"tag_name":' | sed -E 's/.*"v([^\"]+)"\.*/\1/')
```

Then download and install that version:

```
wget -O terraform.zip
"https://releases.hashicorp.com/terraform/${TERRAFORM_VERSION}/terraform_${TERRAFORM_VERSION}_linux_amd64.zip"
unzip terraform.zip
mv terraform "$HOME/bin/"
rm terraform.zip
# Configure Terraform to use the temporary directory for cache to prevent running out of
cloudshell storage space
echo 'export TF_PLUGIN_CACHE_DIR="/tmp"' >> ~/.bashrc
source ~/.bashrc
```

Alternative: If you prefer to check manually, visit [HashiCorp's Terraform releases page](#) and replace `${TERRAFORM_VERSION}` in the URL with the latest version number (e.g., 1.9.5).

What did we do here?

We set up the tooling needed for infrastructure as code. The Lacework CLI connects to FortiCNAPP's API, and Terraform lets us define and deploy cloud resources from code instead of clicking through consoles.

In the next lab you use both together: the CLI generates Terraform configuration, and Terraform deploys it. This is the production-ready approach to FortiCNAPP integration: repeatable, version-controlled, and auditable.

Next: [Lab 12: Install Integrations via Terraform](#).

Lab 12: Install Integrations via Terraform

Objectives

You have now onboarded this account twice, by two different methods. Lab 3 used the console wizard. Lab 4 used a CloudFormation template. [Lab 10](#) then removed both.

This lab does it a third way, and it is the one most teams end up standardising on: **Terraform**. We'll use the Lacework CLI to generate the code, read it, then apply it.

The difference is not the result, it is what you are left holding. A wizard leaves you with a working integration. Terraform leaves you with a working integration **and a file you can review in a pull request, commit, and apply again next quarter to a different account**. That is why the same integration is worth building a third time.

Like CloudFormation in Lab 4, Terraform talks to the FortiCNAPP API directly, so the organization trail on your account is not a factor here either.

Prerequisites

- Completed [Lab 11: Install Terraform](#)

Tip

Shortcut worth knowing. If you already onboarded with automated configuration in Lab 3, you can download the Terraform FortiCNAPP generated instead of generating your own. Go to **Settings > Integrations > Cloud accounts > Deployment History**, open your deployment, and click **Terraform files** on any integration. This lab generates the code from scratch so you can see how the Lacework CLI does it.

- AWS account with appropriate permissions
- FortiCNAPP account access with API key configured

Lab steps

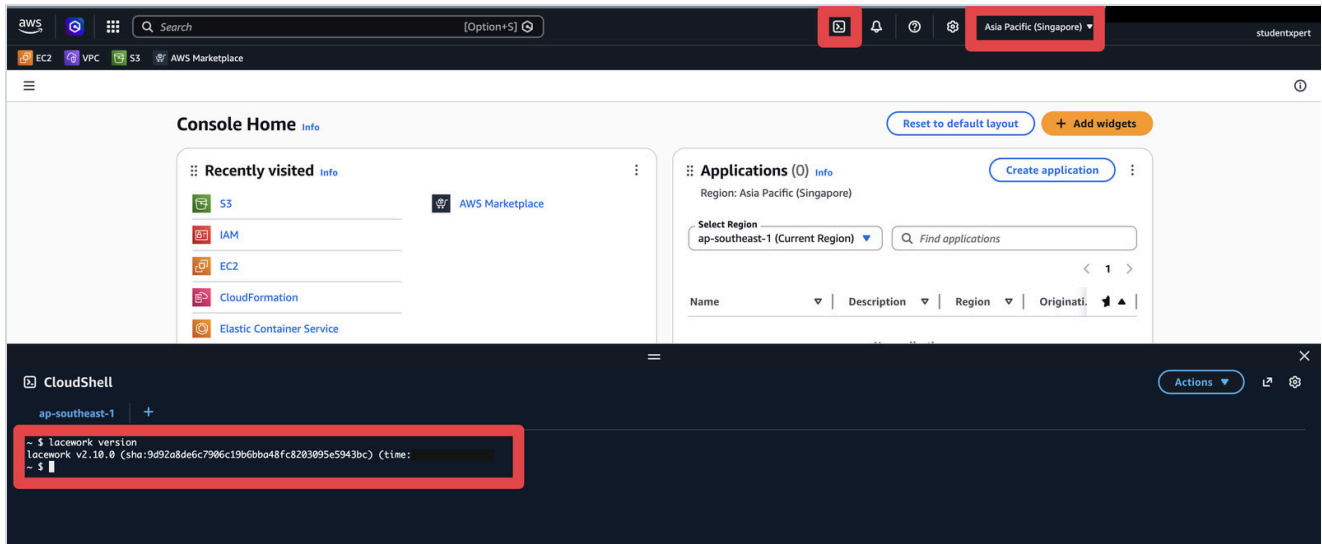
Step 1: Open AWS CloudShell

- Navigate to <https://aws.amazon.com/>
- Click **Sign into console**
- After logging in, change to your local region (e.g., **Asia Pacific (Singapore)**) using the region selector in the top right of the AWS Console
- Click the **CloudShell** icon in the top navigation bar (cloud icon with `>_` symbol)
- Wait for CloudShell to initialize

Step 2: Verify Lacework CLI Configuration

Verify that the Lacework CLI is configured and working:

```
lacework version
```



Step 3: Generate Terraform Configuration for AWS Integration

Use the Lacework CLI to generate Terraform code for the CloudTrail integration:

```
lacework generate cloud-account aws \
  --cloudtrail --noninteractive \
  --aws_region ap-southeast-1
```

Important

Do not add `--config` here, and only run this after [Lab 10](#) has cleaned up.

One AWS account can carry one integration of each type per tenant. If Lab 3's Configuration or Lab 4's CloudTrail is still in place, Terraform builds around 18 AWS resources, has the registration call rejected with "*The provided aws account is already used in this Lacework Application*", and then destroys them all again. Nothing is left behind, but you wait through the whole cycle to learn something that was knowable up front.

Parameters explained:

- `--cloudtrail` : Enable AWS CloudTrail integration
- `--noninteractive` : Run without prompts (uses defaults)
- `--aws_region ap-southeast-1` : Specify the AWS region (Asia Pacific - Singapore)

This command generates Terraform files in the `~/lacework/aws` directory.

Step 4: Review Generated Terraform Files

Navigate to the generated Terraform directory and review the files:

```
cd ~/lacework/aws
ls -la
```

The CLI generates a single `main.tf` file that uses pre-built Terraform modules from the Lacework registry. All the complexity (IAM policies, S3 buckets, CloudTrail setup) is handled by the modules with sensible defaults.

Review the generated configuration:

```
cat main.tf
```

Step 5: Initialize Terraform

CloudShell home is capped at 1 GB and the AWS provider alone is ~700 MB, so with the Lacework CLI (~50 MB) and Terraform binary (~150 MB) already in `~/bin`, there's barely enough room. CloudShell's split mount layout (`/home` on disk, `/tmp` on tmpfs) also trips up git when Terraform downloads modules. Set three env vars to work around both before running `init`:

```
mkdir -p /tmp/tfcache $HOME/tmp
export TF_PLUGIN_CACHE_DIR=/tmp/tfcache
export TMPDIR=$HOME/tmp
export GIT_DISCOVERY_ACROSS_FILESYSTEM=1
terraform init
```

What each one does:

- `TF_PLUGIN_CACHE_DIR=/tmp/tfcache` puts the provider cache on tmpfs (several GB) instead of home (1 GB).
- `TMPDIR=$HOME/tmp` keeps git's temp working dir on the same filesystem as `.terraform/modules/`, so module clones don't fail crossing the `/home` mount boundary.
- `GIT_DISCOVERY_ACROSS_FILESYSTEM=1` silences git's mount-crossing complaint if it does happen.

Troubleshooting

- `Error while installing ... it is still not detected in /tmp; this is a bug in Terraform: CloudShell home is full (the "/tmp" message is misleading).` Run `df -h $HOME` to confirm, then `rm -rf ~/lacework/aws/.terraform ~/lacework/aws/.terraform.lock.hcl /tmp/tfcache` and retry.
- `Error while installing ...: text file busy: a stale terraform process is holding the provider binary open.` Run `kill -f terraform`, then `rm -rf /tmp/tfcache` and retry.
- `Could not download module ...: not a git repository (or any parent up to mount point /home): the env vars above (TMPDIR and GIT_DISCOVERY_ACROSS_FILESYSTEM) fix this.` Make sure both are exported in the same shell as `terraform init`.

Step 6: Review Terraform Plan

Review what Terraform will create before applying:

```
terraform plan
```

This shows you:

- Resources that will be created (IAM roles, CloudTrail, S3 buckets, etc.)
- Any changes that will be made
- Output values that will be generated

The plan output will display in your terminal. Review it carefully to understand what will be deployed.

What will be created (29 resources):

AWS CloudTrail Integration:

- 1 CloudTrail - AWS CloudTrail for API activity logging

- 2 S3 Buckets - One for CloudTrail logs, one for CloudTrail log delivery
- 2 S3 Bucket Policies - Access policies for the buckets
- 2 S3 Bucket Versioning - Enable versioning on both buckets
- 2 S3 Bucket Encryption - Server-side encryption configuration
- 2 S3 Bucket Public Access Block - Block public access to buckets
- 2 S3 Bucket Ownership Controls - Bucket ownership settings
- 1 S3 Bucket Logging - Access logging configuration
- 1 S3 Bucket ACL - Access control list for log bucket
- 1 KMS Key - Encryption key for CloudTrail logs
- 1 SNS Topic - Topic for CloudTrail notifications
- 1 SNS Topic Policy - Access policy for SNS topic
- 1 SNS Topic Subscription - Subscription to forward notifications
- 1 SQS Queue - Queue to receive CloudTrail notifications
- 1 SQS Queue Policy - Access policy for SQS queue
- 1 IAM Policy - Cross-account policy for CloudTrail access
- 1 IAM Role Policy Attachment - Attaching policy to IAM role
- 1 Lacework Integration (`lacework_integration_aws_ct`) - CloudTrail integration
- 1 IAM Role - Cross-account role for FortiCNAPP to read the trail
- 1 Lacework External ID - Security identifier for the IAM role
- 2 Random IDs - Unique identifiers for resource naming
- 1 Time Sleep - Wait period for resource propagation

No Configuration or Agentless resources appear in this plan, and none should. Lab 10 removed everything Labs 3 and 4 built. This plan creates CloudTrail and nothing else.

After this applies the account carries one integration, `AwsCtSqs` . Check **Settings > Integrations > Cloud accounts** before applying if you want to confirm you are starting from empty.

Optional: Save the plan to a file:

If you want to save the plan output for later review or documentation:

```
terraform plan > plan-output.txt
```

Then view it with:

```
cat plan-output.txt
```

Step 7: Apply Terraform Configuration

If the plan looks correct, apply the Terraform configuration to deploy the integration:

```
terraform apply
```

When prompted, type `yes` to confirm the deployment.

Note: This takes two to three minutes. It ends with `Apply complete! Resources: 29 added, 0 changed, 0 destroyed.` and creates:

- An IAM role and policy for the CloudTrail integration
- CloudTrail with encryption and logging
- Two S3 buckets, one for the trail and one for its access logs, both with versioning, encryption and public access blocked
- A KMS key for encryption
- An SNS topic and SQS queue for CloudTrail notifications
- One Lacework integration, `lacework_integration_aws_ct`
- Supporting resources: random IDs, an external ID, and a time delay for propagation

Step 8: Verify Integration Deployment

After the Terraform apply completes successfully, verify the integration:

1. Using Lacework CLI:

```
lacework cloud-account list
```

You should see one entry for this account:

- `AwsCtSqs` (CloudTrail integration), named **TF cloudtrail**

It reads `Pending` at first, not `Ok`. The integration exists before any log data has arrived through the queue.

2. In FortiCNAPP Console:

- Log into FortiCNAPP console at <https://partner-demo.lacework.net/>
- Ensure tenant is set to **FORTINETAPACDEMO**
- Navigate to **Settings > Integrations > Cloud accounts**
- Verify that your AWS account appears with the CloudTrail integration

Step 9: Clean Up Resources

After completing the lab, clean up the resources created by Terraform:

1. Review what will be destroyed:

```
terraform plan -destroy
```

This shows you what resources will be removed.

2. Destroy the resources:

```
terraform destroy
```

When prompted, type `yes` to confirm the destruction.

Note: This will:

- Remove the FortiCNAPP integrations from your AWS account
- Delete IAM roles and policies created by Terraform
- Remove S3 buckets created for CloudTrail (if created by Terraform)
- Remove SNS topics created for notifications (if created by Terraform)
- **Note:** If you're using an existing CloudTrail, it will not be deleted, only the integration will be removed

3. Verify cleanup:

```
lacework cloud-account list
```

The AWS integrations should no longer appear in the list.

4. Clean up Terraform files:

```
rm -rf ~/lacework/aws
```

What did we do here?

We built the CloudTrail integration a third way, with Terraform we own rather than a console wizard or a template someone else launched. The Lacework CLI generated 25 lines of `main.tf`, and `terraform apply` turned that into 29 resources: an IAM role, S3 buckets, KMS encryption, SNS and SQS notifications, and the FortiCNAPP integration itself.

This is how you'd do it in production. The Terraform configuration can be checked into version control, reviewed in pull requests, and deployed through CI/CD pipelines. Need to integrate 50 AWS accounts? Fortinet provides organization-level Terraform modules that deploy across all accounts in your AWS Organization in one go. And when you're done, `terraform destroy` cleanly removes everything.

Additional resources

- [FortiCNAPP Documentation: AWS Integration Terraform from AWS CloudShell](#)

Next: [Lab 13: Scripted Cleanup of All Workshop Resources](#).

Lab 13: Scripted Cleanup of All Workshop Resources

This lab provides a cleanup script that removes every workshop resource, on both the FortiCNAPP side and the AWS side.

Caution

This script assumes a disposable workshop AWS account. It sweeps every enabled region and removes all EC2 instances, key pairs and non-default security groups in the account, not only the ones the workshop created, along with every FortiCNAPP integration pointing at that AWS account. Do not run it in an account that hosts anything you want to keep.

Caution

Run it in CloudShell, not on your own machine. Step 5 clears the home directory it runs in. It deletes `~/.lacework.toml`, `~/.lacework`, `~/.terraform.d`, `~/bin/lacework` and any top-level folder whose name contains `Terraform`. In CloudShell that is exactly what you want. On a laptop it removes the Lacework CLI configuration you use for every other tenant.

It does not touch IAM. The roles in these accounts belong to whoever runs the account pool, and some of them drive cost management and automated cleanup. The workshop's own cross-account role is owned by a CloudFormation stack and goes when the stack does.

Why both sides need cleaning

Who owns the integration record decides what a delete actually removes, and the three methods in this workshop do not agree.

Created by	Deleting the AWS side removes the FortiCNAPP record?
Lab 3, the wizard	No. The record is held in FortiCNAPP and nothing in AWS points back at it
Lab 4, CloudFormation	Yes. Its template registers the integration itself, through <code>LaceworkSnsCustomResource</code> , and deregisters it on delete
Lab 12, Terraform	Yes. Terraform holds the integration in state, so <code>terraform destroy</code> removes both sides

Lab 3 is the case that strands. Remove its AWS resources and the integration stays, polling on schedule, finding a bucket that no longer exists, and reporting:

```
Data loading error: Unable to access scan results within storage bucket
```

So the AWS sweep is not enough on its own. Any integration FortiCNAPP still owns has to be deleted through FortiCNAPP, which is what Step 2 of the script does.

Instructions

1. Open AWS CloudShell in your browser.
2. Copy the entire contents of [cloudshell_cleanup.sh](#) using the copy button.



```
1 #!/bin/bash
2 # CloudShell Cleanup Script
3 # This script cleans up CloudShell artifacts created during labs 4, 8, 10, and 11
4 #
5 # To use: Copy and paste this entire script into AWS CloudShell, or save as a file and run:
6 #   bash cloudshell_cleanup.sh
7
8 set -e
9
10 echo "==== FortiCNAPP Workshop CloudShell Cleanup ==="
11 echo ""
12
```

3. Paste the script into the CloudShell terminal and press Enter.
4. Wait for the script to complete. It may take several minutes, especially when deleting CloudFormation stacks and S3 buckets. Sweeping every region adds time even when most regions are empty.
5. **Verify in the FortiCNAPP console.** Navigate to **Settings > Integrations > Cloud accounts** and confirm your AWS account is no longer listed. If anything remains, select it and click **Delete**.

What the script does, in order

The order matters. The FortiCNAPP integrations are removed while the Lacework CLI and its credentials still exist, and the local CloudShell artifacts are only cleaned up at the very end.

Step	Action
1	<code>terraform destroy</code> for the Lab 12 deployment, which removes its own integrations and AWS resources
2	Delete any remaining FortiCNAPP cloud integrations for this AWS account
3	Per region: terminate EC2 instances, delete CloudFormation stacks, delete CloudTrail trails, delete key pairs, delete non-default security groups

Important

Step 3 does not cover agentless scanner infrastructure. It leaves the ECS cluster, its EventBridge schedule, the scanner VPC and the Secrets Manager secret in place. In the normal order that does not matter, because Lab 10 removed them before you got here. If you skipped Lab 10, check the two that keep costing you:

```
aws ecs describe-clusters --clusters <cluster-name> --query 'clusters[0].status' --output text
aws events describe-rule --name <rule-name> --query 'State' --output text
```

ACTIVE and ENABLED mean the scanner is still scheduled. Go back to [Lab 10](#) and finish the tag sweep. | 4 | Empty and delete Lacework-related S3 buckets, resolving each bucket's own region | 5 | Remove CloudShell artifacts: Terraform and Lacework binaries, cloned repos, config files, `.bashrc` edits |

Notes

- The script runs non-interactively and disables the AWS CLI pager to prevent prompts.
- It continues past individual failures rather than aborting, so one stuck resource does not strand the rest of the cleanup.
- CloudFormation stack deletions wait for completion before proceeding.

- Stack matching is case-insensitive, handles names containing spaces, and deletes every matching root stack rather than just one. Lab 3 lets you name the stack freely, and a redeploy after a failed attempt leaves two.
- Nested stacks are skipped, since deleting the root stack removes them.
- S3 bucket deletion handles versioned objects and delete markers.
- The default security group is preserved.

Troubleshooting

An integration is still listed in FortiCNAPP after the script runs. The script needs the Lacework CLI (installed in Lab 7) and valid credentials to deregister integrations. If you skipped those labs, or the CLI could not authenticate, delete the integration manually: **Settings > Integrations > Cloud accounts > select > Delete**.

A CloudFormation stack failed to delete. Usually a non-empty S3 bucket blocking it. The script retries while retaining the blocked resource, then Step 4 sweeps the bucket. Re-run the script once to clear the remainder.

A security group would not delete. It is still attached to an instance that has not finished terminating. Wait a minute and re-run the script.

Other failures. Check AWS permissions for the CloudShell user, and verify the resources exist before cleanup. Some resources need manual deletion if dependencies prevent automated removal.

That is everything. Back to the [workshop overview](#).